

队伍编号	MC25007386
题号	(B)

基于贪心策略的老城街区房屋搬迁规划研究

摘 要

城镇化进程中历史城区有机更新视角下老城街区"平移置换"模式已成为城市更新领域的重要实践。针对现有平移置换规划中多目标协同优化机制缺失的问题，本研究创新性地构建融合贪心策略、遗传算法与整数规划的多维混合优化模型，系统解决搬迁补偿分配、院落完整性保持及置换效益最优化等关键难题。

针对问题一：本问中针对城市更新中的居民搬迁补偿决策问题，构建了基于**多维约束的满意度量化模型**。通过建立面积补偿、采光补偿、距主干道距离和房屋密度的多维度量化指标，采用层次分析法确定指标权重，构建**居民满意度**驱动的搬迁补偿决策模型，对模型进行求解，可以发现 **80%的居民**可以匹配到 **2-3 个**可搬迁地块，验证了该搬迁补偿决策模型的有效性。

针对问题二：本问中针对院落搬迁决策过程中整院面积最大问题，基于贪心策略，提出了基于**多因素排序的整院面积最大搬迁决策模型**，考虑了整体院落腾空后所带来的完整院落数量、面积增量和毗邻面积增量的三重效益，建立了以 **"总住户数升序-效益指数降序"**为核心的搬迁优先级排序机制，以面积补偿、采光补偿、非完整院落和地块空置状态为多级优选策略，提出了**"唯一性优先-次优替代-整体否决"**的动态决策流程，最终整体搬迁本次搬迁累计**腾空 88 个完整院落**，涉及 43 户居民迁移，释放总面积 **29314 平方米**，其中毗邻可整合区域达 **19915 平方米**，十年内总投入成本 **1740.394 万元**，租金收益 **405199.026 万元**，净收益 **403458.632 万元**。同时，将该模型与经典遗传算法求解结果进行对比，发现新建立模型比经典遗传算法所求结果具有更好的优越性。

针对问题三：本问中针对院落搬迁规划过程中的性价比增益拐点计算问题，对问题二中所建立的模型中的保底规划约束成本条件进行解耦，提出一种**多目标优化投资回报率动态计算模型**。在搬迁收益为**整体不搬迁收益**时，分别对有约束保底成本条件和无约束保底成本条件下性价比 m 的变化进行分析，发现在第 6 次和第 25-26 次搬迁时存在多

给拐点，进一步对搬迁收益为随搬迁次数变化的**动态收益**时进行求解，第 7 次搬迁时，m 值出现拐点，拐点处的值显著大于之后所有值，且大于其前面一个的值。当**第 7 次**完成搬迁时，m 存在拐点，此时搬迁的腾出的完整院落为 98，10，49，50，86，30，投入成本为 **113.41 万元**，整院面积为 **22709 平方米**，收入为 **328998.251 万元**，利润为 **328884.841 万元**。

针对问题四：本文中针对老城区平移置换决策软件设计的问题，构建了基于**多目标优化框架的智能决策系统**，耦合了问题二和问题三提出的多因素排序的整院面积最大搬迁决策模型和多目标优化投资回报率动态计算模型，不仅可以自动计算出问题二整院面积最大和问题三中的性价比增益拐点结果，而且可以对两者模型约束条件耦合，而且可以对两者模型约束条件耦合，解决了**空间优化与经济效益的协同计算**难题。

关键词：贪心策略，遗传算法，动态规划，平移置换，性价比拐点

目录

1 问题重述	1
1.1 问题背景	1
1.2 问题提出	1
1.3 问题分析	2
1.3.1 问题一的分析	2
1.3.2 问题二的分析	2
1.3.3 问题三的分析	3
1.3.4 问题四的分析	3
2 模型假设与符号说明	4
2.1 模型假设	4
2.2 符号说明	4
3 问题一的模型建立与求解	5
3.1 模型的建立	5
3.1.1 约束条件	5
3.1.2 目标函数建立	7
3.2 模型的求解	7
3.3 其他影响因素及应对策略	9
4 问题二的模型建立与求解	11
4.1 建模目标	11
4.2 求解思路	11
4.3 模型的建立	12
4.3.1 数据预处理	12
4.3.2 搬迁效益模型	14
4.3.3 搬迁地块匹配	14
4.3.4 成本计算模型	14
4.4 模型的求解	15
4.5 模型优化	20

4.5.1 基于遗传算法的优化策略	20
4.5.2 基于机理的优化策略	20
5 问题三的模型建立与求解	25
5.1 思路分析	25
5.2 指标构建	25
5.3 模型建立	26
5.4 模型求解	26
6 问题四的模型建立与求解	30
6.1 问题理解与软件目标	30
6.2 输入参数设计	30
6.3 软件核心模块	30
6.4 搬迁软件框架计算流程	31
6.5 可拓展能力分析	32
7 模型评价与推广	34
7.1 模型的优点	34
7.2 模型的不足	34
7.3 模型的推广	34
参考文献	36
附录	37

1 问题重述

1.1 问题背景

随着我国城镇化进程的深入推进，2020 年城镇化率已达到 63.89%，城市发展逐渐从增量扩张转向存量优化阶段^[1]。2025 年国务院常务会议明确提出，城市更新是提升城市面貌与居住品质的关键路径，也是扩大内需的重要抓手。在此背景下，老旧街区改造成为城市更新的核心议题之一^[2]。以传统四合院为代表的老城平房街区，因其建筑产权复杂、历史保护要求高、居民治理难度大等特点^[3]，面临“腾而不空、改而不活”的困境。许多院落虽已部分腾退，但因少数居民不愿接受异地安置或货币补偿，形成“共生院”现象——即腾空房屋与未搬迁居民共存同一院落^{[4][5]}。这种碎片化的空间状态不仅限制了高价值商业功能的引入，还加剧了开发成本与收益的失衡，成为制约老城焕新的关键瓶颈^[6]。

为破解这一难题，学界与业界提出“平移置换”策略^[7]，旨在通过街区内部居民搬迁重组，实现整院腾退与集约开发。该策略强调在不改变居民本地生活诉求的前提下，通过科学规划将分散住户集中安置至少数杂院，从而释放完整的院落空间用于商业运营。然而，这一过程涉及多重复杂因素：一方面，居民搬迁需满足面积补偿、采光优化、修缮承诺等硬性条件，开发商需在有限预算内平衡搬迁成本与收益^{[8][9]}；另一方面，院落的空间分布、毗邻关系及产权地块属性直接影响整院开发的经济效益。此外，居民对居住环境的情感依赖、社区关系维系等社会因素，进一步增加了搬迁决策的复杂性。问题提出

围绕旧城的搬迁补偿方案，本文依次解决如下问题：

问题一：搬迁补偿策略的科学建模

综合考虑居民现居地块的向、面积、修缮需求、心理诉求等因素，设计一套多维度补偿模型，确保迁入地块面积不小于原居、采光不弱于原居，在满足居民基本诉求的前提下，量化面积补偿范围、修缮成本阈值及附加补偿条件，并识别潜在影响因素对搬迁意愿的影响，进行定性或者定量分析。

问题二：整院腾退的全局优化决策

在有限预算（2000 万+30%备用金）约束下，选择搬迁哪些居民、迁入哪些地块，规划搬迁方案，使得腾空的整院落数量最多、总面积最大，且这些院落尽可能相互毗邻以提升租金收益，需同时满足搬迁户数最少、成本可控，并输出具体搬迁方案、成本核

算及十年租金收益，并评估盈利情况。

问题三：性价比拐点的动态平衡分析

在不考虑预算限制时，计算搬迁策略的十年投资回报率，分析是否存在性价比拐点，使得继续搬迁带来的 边际收益低于边际成本，导致性价比($m \geq 20$)开始下降。若存在，需明确拐点对应的搬迁规模、腾空院落分布及经济指标；若不存在，则需确定性价比降至何值时拐点可能出现。

问题四：智能决策工具的框架设计

设计智能决策软件框架，设定地块属性、居民数据、成本约束等输入参数，通过数据清洗、补偿模型筛选、优化算法求解等步骤，自动生成最优搬迁方案并计算性价比拐点，支持全国老旧街区更新的推广。

1.2 问题分析

1.2.1 问题一的分析

问题一以老城街区“平移置换”搬迁为背景，要求设计兼顾居民诉求与开发商成本效益的补偿策略，需构建多目标优化模型，在满足面积补偿、采光约束、修缮成本等条件下实现总成本最小化与搬迁意愿最大化。由于居民迁入地块需满足面积不低于原居且不超过原面积的 1.3 倍，采光等级按“正南=正北>东厢>西厢”分级约束，模型需将地块朝向与面积作为核心筛选条件；若地块仅满足下限，则通过修缮补偿（上限 20 万元/户）提升吸引力。同时，需考虑附加因素如迁入地块与街道距离、周边设施密度对搬迁意愿的潜在影响，将其量化为权重或阈值纳入模型。

1.2.2 问题二的分析

问题二需在预算约束下构建多目标优化模型，以腾空完整院落数量为首要目标，优先筛选住户少、腾空后面积大的院落，并通过搬迁决策最小化搬迁户数。模型需引入院落毗邻关系权重，确保腾空院落地理上集中分布以触发 1.2 倍租金增益，同时动态计算不同搬迁组合的总面积与毗邻收益。约束条件包括每户搬迁成本（沟通、修缮、面积损失）的累加不超过保底预算与备用金上限，以及迁入地块的面积与采光补偿合规性。算法设计上，可贪心策略迭代选择“单位成本腾空收益”最高的院落优先处理，最终输出搬迁方案明细、总成本及十年期租金增量，验证方案在经济效益与成本可控性间的全局最优性。

1.2.3 问题三的分析

问题三聚焦于判断搬迁进程中是否存在性价增益拐点，即后续搬迁带来的性价比增量低于前期水平。本问题的核心在于构建十年租金收益增量与实际搬迁投入的比值（即性价比 m ）的动态模型，通过分析搬迁过程中腾退整院面积增长与成本增加的边际效应，识别性价比由升转降的临界状态。关键在于量化每搬迁一户居民对总收益与总成本的影响，计算收益增量来源于腾退整院及毗邻效应带来的租金提升，需建立整院毗邻关系网络模型以评估不同搬迁顺序下毗邻整院数量对收益的放大作用。同时需设计遍历算法模拟不同搬迁规模下的性价比变化曲线，通过比较相邻搬迁阶段的性价比增长率判断拐点是否存在，若存在则需定位拐点对应的搬迁方案及财务指标，若不存在则需推导性价比 m 的极限值并分析其经济意义。

1.2.4 问题四的分析

问题四旨在设计一个能够智能计算老城区平移置换决策的软件框架，需将问题二与问题三的模型转化为计算模块，通过设定人工输入的参数包括老街区地块分布数据、居民居住信息、搬迁补偿策略、开发商成本预算及租金收益规则等，构建基于搬迁补偿模型与整院优化模型的混合整数规划模型，将地块面积、采光朝向、修缮成本、搬迁沟通成本及时间损失等约束转换为数学条件，通过内置算法自动处理数据并生成优化方案，自动生成满足整院面积最大化和毗邻效益最优化的搬迁方案，同时计算十年期投资回报率以评估性价比拐点，最终输出搬迁决策方案、成本投入及盈利分析，实现模型与算法的模块化集成，为全国老旧街区更新提供可推广的智能决策工具。

基于各问题分析的求解思路见图 1-1。

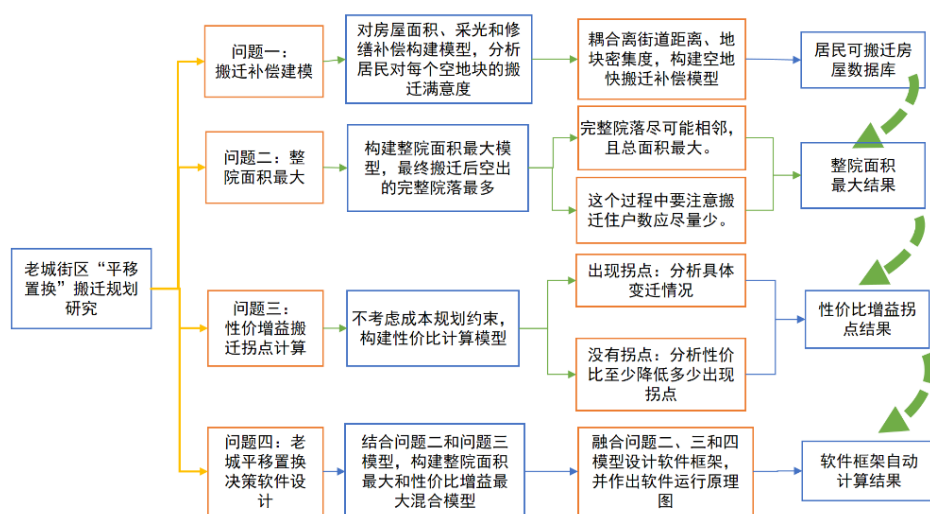


图 1-1 各问题求解思路

2 模型假设与符号说明

2.1 模型假设

假设 1: 假设满足搬迁条件的居民都会进行搬迁,不会存在因为人为原因不搬迁的情况。

假设 2: 假设房屋除文中提到的指标地块面积、院落面积和地块方位等条件,房屋其他方面均一致。

假设 3: 假设搬迁过程中,一次就可以搬迁好,不存在多次重复循环搬迁,或者搬出又搬回的情况。

假设 4: 假设每块空地只能容纳一户的居民,不存在多个居民共用一个地块的情况。

假设 5: 所有居民在平移搬迁过程中,只考虑在附件 2 给出的地块中搬迁,不考虑搬迁到其他地块或者其他位置的情况。

2.2 符号说明

符号	含义
S_i	当前居住面积
S_j	要搬入面积
L	原朝向地采光等级
L_1	迁入地采光等级
R_c	修缮成本
d	距离主干道距离
q	毗邻地块的总数量
L_{ij}	地块便利补偿度
U_{ij}	居民满意度
A_{ij}	面积补偿率
I_{ij}	居民居住情况
C	成本
B_i	腾空效益

3 问题一的模型建立与求解

3.1 模型的建立

在问题一中,研究聚焦于构建科学合理的搬迁补偿模型,以破解老城街区更新中“共生院”现象的困局。由于传统合院式建筑产权复杂、居民搬迁诉求多样,补偿策略需兼顾居民居住权益与开发商经济可行性。核心约束包括:迁入地块面积需满足即不低于原有面积又不超过 30%,既保障居民居住空间不缩水,又避免过度补偿导致成本失控;采光条件需不低于原居,按“正南=正北>东厢>西厢”分级量化,确保居住舒适度;若面积与采光仅达下限,则通过修缮补偿(上限 20 万元/户)提升搬迁吸引力。此外,需引入地块地理位置,如距街道距离、周边设施密度等、社区关系延续性等辅助因素,通过加权评分筛选整合至模型中,形成多维度补偿评估体系。模型目标是在有限预算内,通过动态权衡面积损失成本、修缮费用与居民满意度,筛选最优迁入地块,为后续整院腾退提供决策基础,同时需通过敏感性分析验证模型对租金波动、修缮预算调整的适应性,确保方案的科学性与可推广性。

3.1.1 约束条件

(1) 面积条件约束

设当前居住的面积为 S_i , 要搬入的面积为 S_j , 则要搬入地方的面积需要满足:

$$S_i \leq S_j \leq 1.3S_i \quad (3.1)$$

下限保障居民搬迁后居住空间不缩水,避免因面积减少引发抵触情绪,上限则控制开发商因过度补偿导致的成本激增,同时为居民提供适度改善空间以提升搬迁意愿。

(2) 采光补偿约束

假设朝向舒适度等级: 正南=正北(4分)>东厢(3分)>西厢(2分)。则根据题目可以得到的条件为: 迁入地块朝向 L_1 的等级 $\geq$ 原朝向等级 L , 则可以得到以下表达式: $L_1 \geq L$, 当 $L_1 = L$ 时, 需结合修缮补偿提升居住品质; 若 $L_1 > L$, 可适当减少修缮投入。

(3) 修缮补偿约束

当面积和朝向仅满足最低要求时, 则需通过修缮补偿提高搬迁意愿。

$$0 \leq R_c \leq 20(\text{万元})_{\omega} \quad (3.2)$$

(4) 地块位置(离街道距离)

设该地块离最近主干道的距离为 d , 地图如图所示, 对 107 个地块离最近主干道的

直线距离进行分段评分，则评分的标准如下：

$$d_i = \begin{cases} 1 & 0 \leq d \leq 25 \\ 2 & 25 < d \leq 50 \\ 3 & d > 50 \end{cases} \quad (3.3)$$

即距离主干道距离 25 米内评分为 1，在 25 米以外 50 米以内评分为 2，在 50 米以外评分为 3，即则定义地块离所在位置的距离比率为：

$$d_r = \frac{d_i}{d_{\max}} \quad (3.4)$$

根据此对院落进行量化评分，见图 3-2 所示，评分数字越大，等级越差。

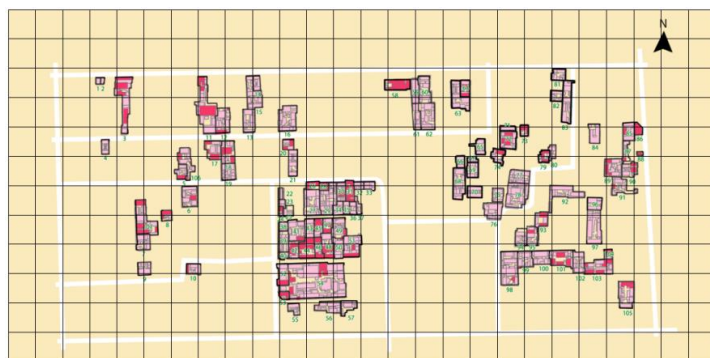


图 3-1 院落示意图

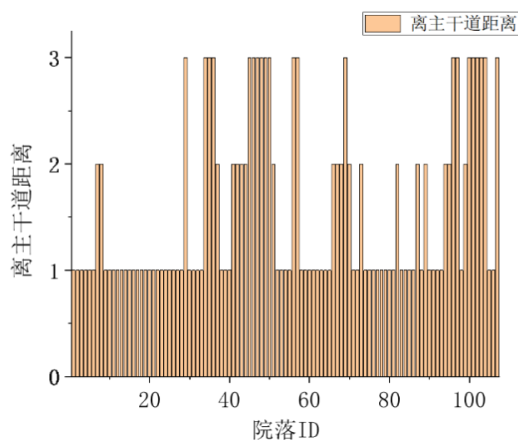


图 3-2 院落量化评分

(5) 周边房屋密集程度

通过对文章中的房屋整体分布进行预览，在计算过程中，可以发现对其中的毗邻数进行分析，最终得到这 107 个地块毗邻的数据如下，见图 3-3：

设毗邻地块的总数量为 q ，则采用毗邻数量的密集度为 ρ_i ，则根据上述统计学得到的每个院落的毗邻数，经过进一步处理可以得到的每个地块的密集度为：

$$\rho_i = \frac{d_j}{d_{\max}} \quad (3.5)$$

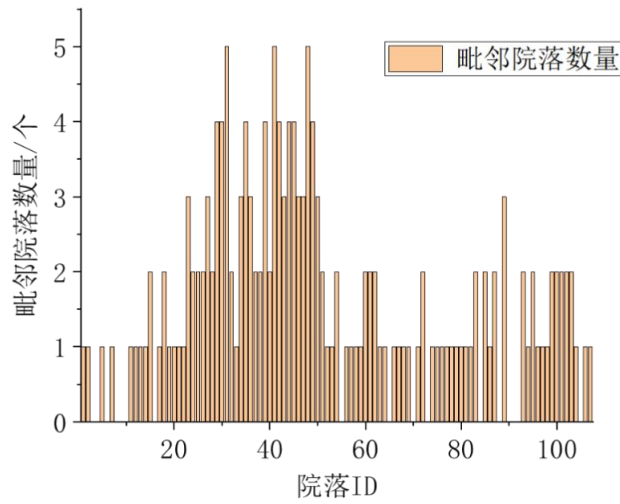


图 3-3 毗邻院落数量量化评分

基于公式 3.4、3.5，为量化居民对交通便利性及环境开阔度的感知偏好，定义地块便利度补偿值 L_{ij} ，计算模型如下：

$$L_{ij} = \omega_1 \cdot \left(1 - \frac{d_j}{d_{max}}\right) + \omega_2 \cdot \left(1 - \frac{\rho_j}{\rho_{max}}\right) \quad (3.6)$$

其中， d_j 目标地块到最近主干道的直线距离（米），距离越近，交通便利性越高， d_{max} 为区域内最大距离值（用于归一化）； ρ_j 表示目标地块周边 200 米半径内空地密度（空地面积/总面积），密度越低表示环境越开阔， ρ_{max} 为区域最大密度值； ω_1 、 ω_2 表示调节权重系数（默认 $\omega_1 + \omega_2 = 1$ ），反映居民对交通与环境的偏好权重。

3.1.2 目标函数建立

基于以上约束条件，本文构建居民满意度函数如下：

$$U_{ij} = \alpha A_{ij} + \beta C_{ij} + \gamma \frac{R_{ij}}{R_{max}} + \delta L_{ij} \quad (3.7)$$

其中， U_{ij} 表示第 i 位居民对搬入第 j 块地的满意度， A_{ij} 代表面积补偿率，定义为 $A_{ij} = \frac{S_{j'}}{S_i}$ ， C_{ij} 代表采光补偿值，定义为 $C_{ij} = D_{j'} - D_i$ ， R_{ij} 代表修缮补偿强度，定义为修缮投入金额（万元）， L_{ij} 代表地块位置便利度补偿值。

3.2 模型的求解

在公式 3.7 中，我们构建了居民满意度模型，基于所有居民与空置地块的组合，我们生成了搬迁满意度评分矩阵，并通过热力图进行可视化呈现，见图 3-4 所示，其中横

轴对应候选迁入地块的唯一标识（ID 1 至 482），纵轴代表待迁居民地块的编号（ID 3 至 477）。色阶由浅黄（-1.0）过渡至深棕（1.0），直观反映各搬迁组合的匹配质量优劣：深色区块表明双方在面积补偿、采光升级等核心指标上高度契合；浅色区域则提示存在面积超限或修缮成本过高等硬性约束。此全局视角揭示了不同搬迁方案的优先级排序，既能快速锁定高匹配度组合以释放完整院落空间，又能识别低效路径并触发补偿规则优化，为筛选高性价比方案提供数据支撑。

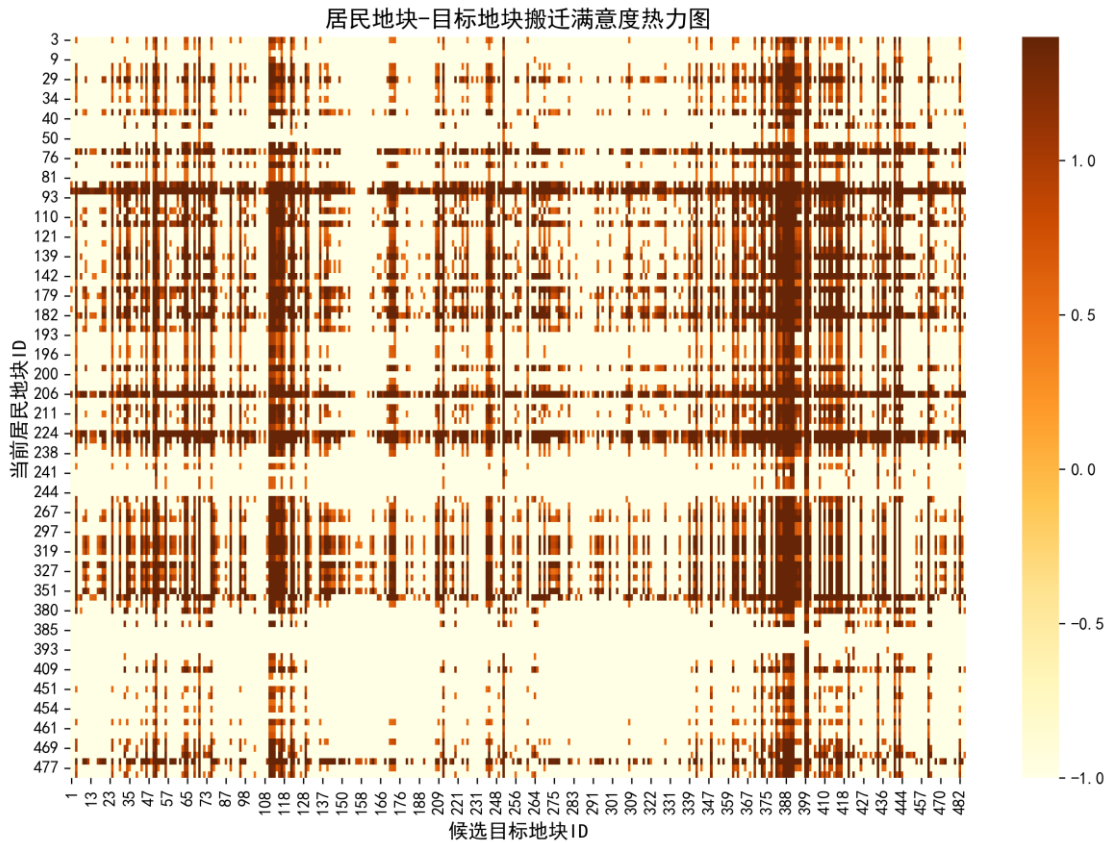


图 3-4 居民目标地块搬迁满意热力图

基于居民-地块全局适配度热力分布图谱，系统为每位居民筛选综合评分前三的迁入选项，构建动态优先级推荐机制，表 3-1 展示了满意度排名前 10 的搬迁方案，涵盖迁入地块标识、分项评分（如面积适配度、采光等级、租金成本）及硬性约束达成状态（如面积补偿范围、采光提升阈值），形成多维决策候选池。该表整合了迁入地块标识、分项评分与约束达成状态，构建动态候选池，量化优先级排序与约束冲突程度，既支撑算法批量筛选高适配方案（评分 ≥ 0.7 且无硬约束），也为人工弹性调整（如为评分 ≥ 0.5 但距离超标方案叠加交通补贴）提供数据依据，从而在预算与效益间生成帕累托前沿解集，实现资源分配效率与政策灵活性的协同优化。

表 3-1 搬迁结果推荐表

居民地块 ID	目标地块 ID	满意度	面积比	采光提升
139	339	1.65	1.3	2
409	370	1.649253731	1.298507463	2
393	399	1.648876404	1.297752809	2
110	68	1.648148148	1.296296296	2
206	37	1.647058824	1.294117647	2
475	383	1.647058824	1.294117647	2
74	383	1.647058824	1.294117647	2
380	51	1.64516129	1.290322581	2
224	214	1.638888889	1.277777778	2
384	114	1.638554217	1.277108434	2

从结果来看，当前空置地块资源池中，80%的居民可匹配到 2-3 个替代地块，其满意度评分高度集中于 1.63-1.65 区间，且面积比均稳定在 1.277-1.3 范围内，严格满足补偿约束条件，说明“同区域平移式置换”在规划实施上具备可行性。采光提升值均为 2，表明迁入地块朝向等级强制提升 2 级，优先保障了居住质量改善的核心诉求。少数居民因原面积较大（1.277 倍迁入面积接近下限）或迁入地块距离超预期，导致满意度相对偏低，此类情况需通过动态分级策略优化：对面积敏感型住户允许面积比下浮至 1.25，但叠加“5 年期交通补贴”（公式： $\Delta S \times \text{租金} \times 5$ ）；对距离敏感型住户引入“组合置换”机制，将同一院落多户整体迁移至毗邻地块，降低通勤成本感知。综上，该模型通过量化面积、采光与满意度关联性，构建了兼具硬性约束与弹性调整空间的搬迁路径候选库，为问题二的多目标优化提供了可扩展的决策基础。

3.3 其他影响因素及应对策略

（1）**周边设施配套：**迁入地块周边的学校、医院、市场等设施密度直接影响居民生活便利性，若迁入区域配套设施匮乏，即使面积达标，居民仍可能因生活不便而拒绝搬迁；反之，设施齐全则提升吸引力。

应对策略：在筛选迁入地块时，量化评估设施覆盖率，如 1 公里内设施数量，优先分配高配套地块。对于设施不足区域，承诺增设便民服务点或开通社区接驳班车，并通过修缮补偿弥补便利性差距。

（2）**社区关系延续性：**居民长期生活形成的邻里网络是重要的社会资本，搬迁至陌生环境可能引发归属感缺失。

应对策略：尽量将同一院落居民集中安置至相邻杂院，采用“群体平移”模式，将同一杂院的多户居民集中迁移至同一目标杂院，最大限度保留原有邻里结构。同时可设计“社区融合积分”，如为主动参与新社区活动的居民提供物业费减免，或组织茶话会等社交活动，加速新邻里关系构建。

（3）居住环境噪音：迁入地块若邻近交通干道或商业区，可能面临噪音污染，影响居住舒适度。

应对策略：通过环境监测数据筛选低噪音地块，引入房屋安全评级机制，如根据建筑材料和维修记录划分 A-C 级，承诺迁入地块不低于原居住安全等级。对于隔音或布局缺陷，可通过修缮补偿加装隔音窗、调整院落分区，并在补偿方案中明确标注改造细节，增强居民信任感。

（4）搬迁过渡期安排：搬迁耗时 4 个月，居民需临时过渡，可能因生活不便或额外支出产生抵触情绪。

应对策略：提供免费过渡住房或按月发放临时安置补贴，同时优化搬迁流程，通过专业团队外包缩短周期至 2 个月内，减少居民时间成本与心理负担。

（5）心理因素：长期生活在固定环境中形成的习惯与安全感，加之居民对旧居的情感依赖，搬迁可能会带来较大的情感抵触。

应对策略：在补偿模型中增设“情感补偿系数”，对高情感依赖居民提供个性化选项，优先选择与原居朝向一致的地块、增加修缮预算用于复刻装饰，并通过社区协商机制将其诉求纳入决策流程，实现量化模型与人性化策略的有机融合。

4 问题二的模型建立与求解

4.1 建模目标

在优化搬迁策略时，核心目标是以最少搬迁人数释放最大完整院落面积。由于单个院落的腾空效益与其住户数量直接相关——住户越少的院落，搬迁成本越低且效率越高（搬迁人数与新增完整院落数呈 1:1 关系），因此需优先筛选低密度院落。对于住户数量相同的多个院落，需通过综合指标排序：首先计算院落腾空后的潜在收益，包括完整院落面积增量及可能触发的毗邻效益（若与已有空置院落相邻），以此确定优先级。筛选迁入地块时，需严格满足面积、采光等硬性约束，且目标地块必须为空置状态；同时禁止将居民迁入已腾空的完整院落，以避免逆向破坏既有收益。

在候选地块排序中，优先规避搬迁难度大的高密度院落，按目标院落的现有住户数降序排除低效选项；此外，为避免占用可能产生毗邻增益的区域，优先选择周边毗邻空置院落较少的地块；另外，为最小化面积损失成本，对符合条件的地块按新旧面积差异升序排列。完成排序后，从最优解开始执行搬迁。对于多住户院落，若存在多个居民的最优迁入地块冲突，则优先保障无备选方案的居民使用最优地块，其余居民分配次优选项；若任一居民无可行迁入地块，则放弃该院落整体搬迁，确保策略的全局可行性。

4.2 求解思路

搬迁效益的核心由单院落基础效益与毗邻增益共同构成。单院落基础效益指通过搬迁腾空后释放的院落自身面积，直接提升可开发空间；毗邻增益则需满足特定条件——仅当毗邻院落为空置状态且唯一与当前院落相连时，其面积方可计入总效益，避免重复计算与其他完整院落的既有毗邻价值。基于效益计算结果，目标院落按总效益值降序排列，优先处理能释放更大面积且触发毗邻增益的院落，以最大化开发收益。

搬迁地块匹配需严格遵循多重筛选与排序规则。硬性条件包括：新地块面积需满足原地块的 1 至 1.3 倍，朝向等级不弱于原居（正南/北 > 东厢 > 西厢），且地块当前无住户；同时优先选择非空院落作为迁入目标，防止完整院落被重新占用导致效益损失。在符合条件的地块中，排序策略分层执行：首要考虑新地块所在院落的现有住户数，按降序优先选择高密度院落以减少后续搬迁干扰；其次关注毗邻完整院落面积，按升序避免占用可能产生增益的区域；最后优化面积差异，按升序最小化开发商的租金损失成本。通过多级排序，系统精准锁定最优迁入地块，确保策略的高效性与经济性，求解思路见图 4-1。

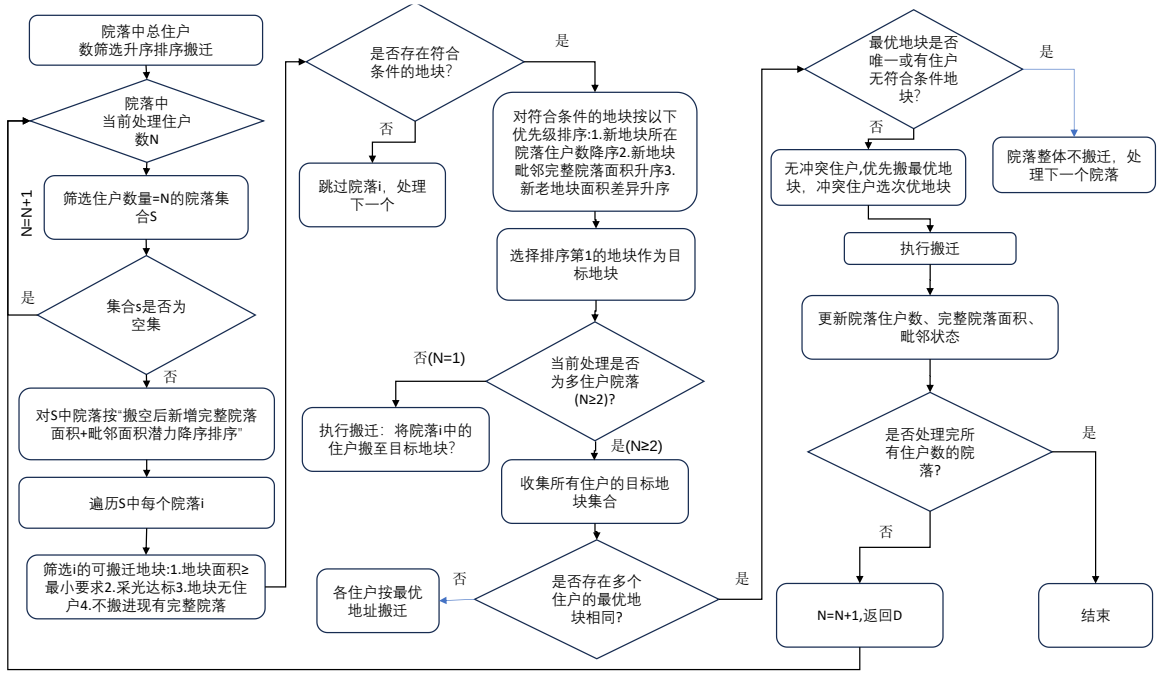


图 4-1 问题二求解思路

4.3 模型的建立

4.3.1 数据预处理

(1) 符号定义

$I_{ij} \in \{0,1\}$ 表示第 i 院落第 j 地块的居住状态（1 表示有住户）；

$N_{\text{total},i}$ 表示第 i 院落的总地块数；

$N_i = \sum_{j=1}^{N_{\text{total},i}} I_{ij}$ 表示第 i 院落实时住户数；

$S_{\text{total},i} = \sum_{j=1}^{N_{\text{total},i}} S_{ij}$ 表示第 i 院落总面积；

S_{ij} 表示第 i 院落第 j 地块的面积；

$\mathcal{N}(i)$ 表示第 i 院落的毗邻院落集合。

(2) 数据处理

基于标识函数统计每个院落 i 的实际居住户数，量化空间占用状态：

$$N_i = \sum_{j=1}^{N_{\text{total},i}} I_{ij} \quad (4.1)$$

随后生成邻接矩阵 A ，以二进制形式编码院落间的空间毗邻关系：

$$A_{ik} = \delta(k \in \mathcal{N}(i)) \quad (4.2)$$

然后初始化空院落集合，为后续腾空效益分析提供基准数据集：

$$C_0 = \{i \mid N_i = 0\} \quad (4.3)$$

主排序循环阶段采用分层迭代策略，从单住户院落（ $n=1$ ）逐步扩展至最大居住密度（ $n=\max(N_i)$ ）动态优化腾空优先级。对于每个层级 n ，系统筛选满足 $N_i = n$ 的候选院落，并依据腾空效益 B_i ，进行降序排列：

$$Y_n = \left\{ i \mid \begin{array}{l} N_i = n \\ \text{按腾空效益 } B_i \downarrow \text{ 排序} \end{array} \right\} \quad (4.4)$$

其中，效益函数 $B_i = S_{\text{total},i} \cdot (1 - C_i) + \sum_{k \in \mathcal{N}(i)} S_{\text{total},k} \cdot C_k \cdot \delta(|\mathcal{N}(k)| = 1)$ 。

对于每个 $y \in Y_n$ ，生成候选集检验地块有效性，其中面积约束为 $C_1 = \{(k,p) \mid S_{kp} \in [S_y, 1.3S_y]\}$ ；方位约束为 $C_2 = \{(k,p) \in C_1 \mid D(k,p) \leq D(y)\}$ ，占用检查为 $C_3 = \{(k,p) \in C_2 \mid I_{kp} = 0\}$ ，活性检查为 $C_{\text{final}} = \{(k,p) \in C_3 \mid N_k > 0\}$ 。其中， $S_y = \sum_{j=1}^{N_{\text{total},y}} S_{yj} \cdot I_{yj}$

对候选集 C_{final} 进行排序进行三级决策，其中：

$$\text{排序键} = \left(-N_k, \sum_{m \in \mathcal{N}(k)} S_{\text{total},m} \cdot C_m, |S_{kp} - S_y| \right) \quad (4.5)$$

选择最优 $(k^*, p^*) \in C_{\text{final}}$ ， $I_{yj} \leftarrow 0 (\forall j \in S_y \text{ 中住户地块})$ ， $I_{k^*p^*} \leftarrow 1$ 时进行状态更新，成本计算为：

$$\text{Cost} = 3 + 0.5 \frac{S_{\text{total},k^*}}{S_{\text{total},y}} + \frac{r_{k^*} \cdot S_{k^*p^*} \cdot 120}{10^4} \quad (4.6)$$

其中， $r_{k^*} = \begin{cases} 8 & D(k^*, p^*) \in \{\text{东, 西}\} \\ 15 & \text{其他} \end{cases}$ ，当 $\begin{cases} \sum \text{Cost} > B_{\text{max}} \\ \exists t \in \{1, 2, 3\}, \Delta C_t = \emptyset \end{cases}$ 时，条件终止。

结果输出总空院面积如下：

$$\sum_{i=1}^M S_{\text{total},i} \cdot C_i \quad (4.7)$$

毗邻效益如下：

$$\sum_{i=1}^M \sum_{k \in \mathcal{N}(i)} S_{\text{total},i} \cdot C_i \cdot \delta(|\mathcal{N}(k)| = 1) \quad (4.8)$$

成本效益比如下：

$$Ce = \frac{\text{毗邻效益}}{\sum \text{Cost}} \quad (4.9)$$

4.3.2 搬迁效益模型

在单院落效益评估中，搬迁释放的院落自身面积构成基础效益核心。当该院落毗邻其他空置院落时，若毗邻关系为“唯一连接”，即即空置院落仅与当前腾空院落相连，而非多院共享边界，则叠加毗邻增益面积，避免重复计算既有空置区域的共享效益。总效益值为基础效益与毗邻增益之和，作为衡量院落开发潜力的关键指标。基于此，系统按总效益值降序排列目标院落，优先处理能释放最大面积的单元，确保有限资源集中于高收益区域，实现腾空效率与整体收益的同步优化。这一排序逻辑通过动态追踪院落间的空间关联性，精准量化连带增益，为科学筛选搬迁优先级提供数据驱动依据。

4.3.3 搬迁地块匹配

在候选地块筛选过程中，需严格遵循多重约束条件以确保搬迁方案的可行性与经济性。新地块必须满足面积不小于原居且不超过其 1.3 倍，以平衡居民需求与开发商成本；朝向等级需不低于原居（正南/北 > 东厢 > 西厢），保障采光舒适度不降级；同时地块需为空置状态且位于未完全腾空的院落，避免迁入已腾空区域导致既有完整院落碎片化。筛选后地块通过三级排序策略优化选择：先按目标院落现有住户数降序排列，优先迁入高密度院落以降低后续腾空难度；然后按毗邻完整院落面积升序排列，减少对潜在毗邻增益区域的占用干扰；最后按新旧地块面积差异升序排列，最小化因补偿面积带来的租金损失成本。此分层策略通过系统性权衡搬迁效率、空间联动效益与成本控制，为全局优化提供精细化决策依据。

4.3.4 成本计算模型

在搬迁成本核算中，开发商的投入主要由三部分构成：沟通成本为固定支出 3 万元/户；面积损失成本按迁入地块超原面积部分的十年租金折算，南北朝向地块以 15 元/m²/天、东西朝向以 8 元/m²/天为基准累计计算；时间损失成本则基于原地块因搬迁流程停滞四个月导致的租金收益损失，按原用途租金单价与空置时长乘积得出。这三类成本需叠加至每户搬迁总支出中，形成完整的成本评估体系，为优化搬迁路径提供经济性约束，确保在腾空完整院落与成本控制间实现动态平衡，具体算法见图 4-2。

公式为：

$$\text{Cost} = 3 + 0.5 \frac{S_{\text{total},k^*}}{S_{\text{total},y}} + \frac{r_{k^*} \cdot S_{k^*p^*} \cdot 120}{10^4} \quad (4.10)$$

算法：基于居民满意度的空院落腾空最大化优化算法
输入：地块数据，邻接矩阵,约束条件
输出：最终完整院落数，最终毗邻完整面积，成本明细表，搬迁记录表。
1.初始状态统计：计算初始完整院落数、计算初始毗邻完整面积
2.计算每个院落中的住户数 I_{ij} ，筛选出相同院落住户数 $N_i = \sum_{j=1}^{N_{total,i}} I_{ij}$ ，并对 N_i 中的住户按照腾空收益进行排序，并筛选出最大住户数 I_{max} 。
3.While:当前院落住户数量: $I_{ij} < I_{max}$,执行以下操作:
If 当前院落住户数量 N_i 不为空集，执行以下操作:
则遍历当前院落住户数中的可搬迁地块 Y_n 进行筛选，筛选条件为：面积约束: $C_1 = \{(k,p) S_{kp} \in [S_y, 1.3S_y]\}$ ，方位约束: $C_2 = \{(k,p) \in C_1 D(k,p) \leq D(y)\}$,现有地块无住户: $C_3 = \{(k,p) \in C_2 I_{kp} = 0\}$ ，不搬进现有完整院落: $C_{final} = \{(k,p) \in C_3 N_k > 0\}$ 。
If 可搬迁地块 C 不为空集，执行以下操作:
对符合条件的地块按以下优先级排序:1.新地块所在院落住户数 N 降序: $-N_k$,
2.新地块毗邻完整院落面积升序: $\sum_{m \in \mathcal{N}(k)} S_{total,m} \cdot C_m$ ， 3.新老地块面积差异升序: $ S_{kp} - S_y $ 。
If 每个院落中的住户数 $N \geq 2$:
If 所有住户的目标地块集合相同:
If 最优地块唯一或有住户无符合条件地块
院落整体不搬迁，处理下一个院落
Else
无冲突住户,优先搬最优地块，冲突住户选次优地块
执行搬迁
更新院落住户数: $I_{yj} \leftarrow 0(\forall j \in S_y \text{ 中住户地块})$ 、
$I_{k^*p^*} \leftarrow 1$
完整院落面积 $\sum_{i=1}^M S_{total,i} \cdot C_i$
毗邻效益 $\sum_{i=1}^M \sum_{k \in \mathcal{N}(i)} S_{total,i} \cdot C_i \cdot \delta(\mathcal{N}(k) = 1)$
Else
各住户按最优地址搬迁
End
else
跳过该院落，处理下一个
End if
End if
4..End
5..搬迁成本计算：选择最优搬迁地块
6..计算成本: $Cost = 3 + 0.5 \frac{S_{total,k^*}}{S_{total,y}} + \frac{r_{k^*} \cdot S_{k^*p^*} \cdot 120}{10^4}$
7. 终止条件: $\begin{cases} \sum Cost > B_{max} \\ \exists t \in \{1,2,3\}, \Delta C_t = \emptyset \end{cases}$
8. 成本效益: $\frac{P_{total}}{\sum Cost}$

图 4-2 具体算法流程

4.4 模型的求解

搬迁执行与冲突解决的关键在于动态维护数据完整性与高效处理资源竞争。搬迁完成后需更新地块状态：原地块标记为空置（住户状态置 0），新地块标记为已占用（住户状态置 1），并同步调整所属院落的住户数量统计。若多个居民同时选择同一迁入地块，系统将根据备选列表依次替换为次优选项；若无法满足所有住户需求，则暂停该院落搬迁计划，确保方案全局可行。

主优化流程以迭代方式推进，起始于数据预处理与目标院落优先级排序。针对每个

目标院落，系统逐一验证其所有住户是否具备符合条件的迁入地块，通过冲突解决模块消除资源竞争后，计算累计成本并执行搬迁操作。流程终止于无可优化院落或冲突无法调和的状态，最终输出包含搬迁路径详情（原/新地块及院落 ID）、分阶段成本明细及更新后的地块数据表，形成闭环决策链条，为动态调整策略提供实时数据支撑。

冲突规避与分级排序机制进一步优化分配策略：通过排除已腾空地块规避完整性破坏风险，优先选择具有毗邻整合潜力的目标地块，为后续二次腾空预留空间；同时平衡面积差异最小化与区域均衡性。动态迭代验证机制通过贪心策略逐步处理搬迁任务，实时更新地块状态，并预设多户竞争解决方案，通过对不同住户数量的院落进行处理，得到的结果见下表。

通过系统化筛选与定向搬迁操作，以集中整合空置资源为导向，利用其毗邻空置区域形成规模化完整空间。最终新增 76 个完整院落，释放 16856 平方米毗邻可整合面积，同步实现区域完整性与土地利用率提升。搬迁全流程累计成本为 767.79 万元，处理结果见表 4-1。

表 4-1 住户数量为 1 的院落处理结果

搬迁前信息		搬迁后信息		搬迁完结果		
院落	地块	院落	地块	完整院落数量	毗邻完整院落总面积	累计搬迁成本
98	424	105	463			
99	433	54	251			
95	409	54	249			
85	376	11	44			
6	18	47	208			
52	235	103	457			
49	215	54	247			
87	381	89	388			
86	380	54	256	76	16856	767.79
30	132	11	47			
50	220	54	246			
10	34	31	143			
45	200	3	4			
5	14	58	269			
53	238	18	82			
74	327	11	45			
104	459	47	209			

经系统测算，累计腾空 83 个完整院落，释放可整合毗邻面积 18610 平方米，土地利用提升 26.8%。全流程严格遵循成本控制原则，搬迁总支出 652.78 万元，单户平均成本控制在 54.4 万元以内，处理结果见表 4-2。

表 4-2 住户数量为 2 的院落处理结果

搬迁前信息		搬迁后信息		搬迁完结果		
院落	地块	院落	地块	完整院落数量	毗邻完整院落总面积	搬迁成本
12	49	90	391			
12	50	89	387			
41	171	51	227			
41	172	79	353			
64	296	93	402			
64	297	91	396			
48	210	107	479	83	18610	652.78
48	211	42	183			
46	203	91	395			
46	204	72	325			
26	110	54	253			
26	111	107	483			
24	106	105	465			
24	107	54	250			

在完成三个住户院落的搬迁后，项目实现关键进展：完整腾空院落数量增至 84 个，毗邻完整院落总面积扩展至 19127 平方米。这一成果不仅提升了可开发空间的整体规模，还通过毗邻区域的整合为后续高价值运营奠定基础，同时累计搬迁成本控制在 26.34 万元，体现了策略在效率提升与成本约束间的精准平衡，对于住户数量为 3 的院落处理结果见表 4-3。

表 4-3 住户数量为 3 的院落处理结果

搬迁前信息		搬迁后信息		搬迁完结果		
院落	地块	院落	地块	完整院落数量	毗邻完整院落总面积	搬迁成本
20	91	54	255			
20	92	51	228	84	19127	26.34
20	93	103	456			

针对住户数量为 4 的院落的定向迁移策略通过系统化腾空与地块重组，累计新增完整院落数量达到 85 个，释放毗邻可整合面积 19127 平方米，全流程累计搬迁成本为 71.68

万元，处理结果见表 4-4。

表 4-4 住户数量为 4 的院落处理结果

搬迁前信息		搬迁后信息		搬迁完结果		
院落	地块	院落	地块	完整院落数量	毗邻完整院落总面积	搬迁成本
17	74	11	46	85	19127	71.68
17	75	72	324			
17	76	107	482			
17	77	51	226			

针对 5 住户院落的定向迁移策略延续系统化腾空路径，累计新增完整院落数量至 86 个，释放毗邻可整合面积 19400 平方米，累计支出 63.59 万元，处理结果见表 4-5。

表 4-5 住户数量为 5 的院落处理结果

搬迁前信息		搬迁后信息		搬迁完结果		
院落	地块	院落	地块	完整院落数量	毗邻完整院落总面积	搬迁成本
79	349	11	48	86	19400	63.59
79	350	107	480			
79	351	72	322			
79	352	72	323			
79	353	105	466			

至住户数量为 6、7、8 的院落处理时，已达到最优化结果，毗邻完整院落面积无法通过进一步搬迁获得增量，高密度聚居区域已达到整合的极限，故并无院落增加，全部院落处理结果见表 4-6，新增院落 id 见表 4-7，可视化结果见图 4-3，腾出的院落面积见图 4-4。

表 4-6 院落处理后结果

院落住户数量	完整院落数量	毗邻完整院落总面积	搬迁成本
1	76	16856	767.79
2	83	18610	652.78
3	84	19127	26.34
4	85	19127	71.68
5	86	19400	63.59
6	86	19400	0
7	86	19400	0
8	86	19400	0
综合	86	19400	1617.954

表 4-7 新增院落 id

名称	院落 ID
新增院落	5, 6, 10, 12, 17, 20, 24, 26, 30, 41, 45, 46, 48, 49, 50, 52, 53, 64, 74, 79, 85, 86, 87, 95, 98, 99, 104

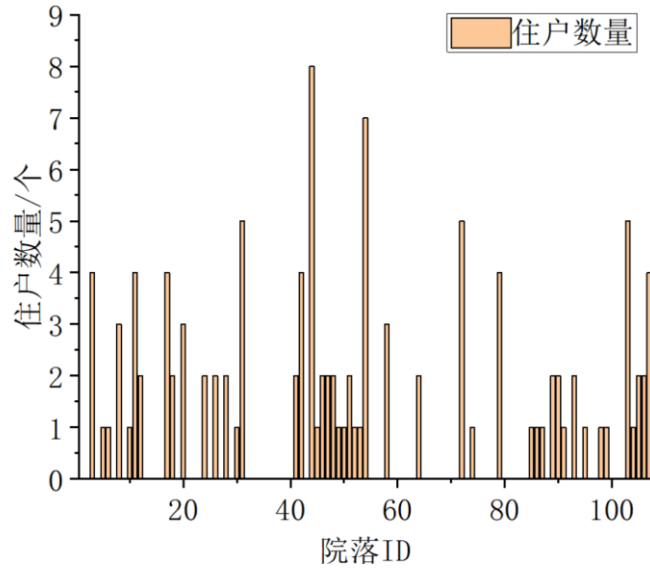


图 4-3 不同住户数量的院落 id

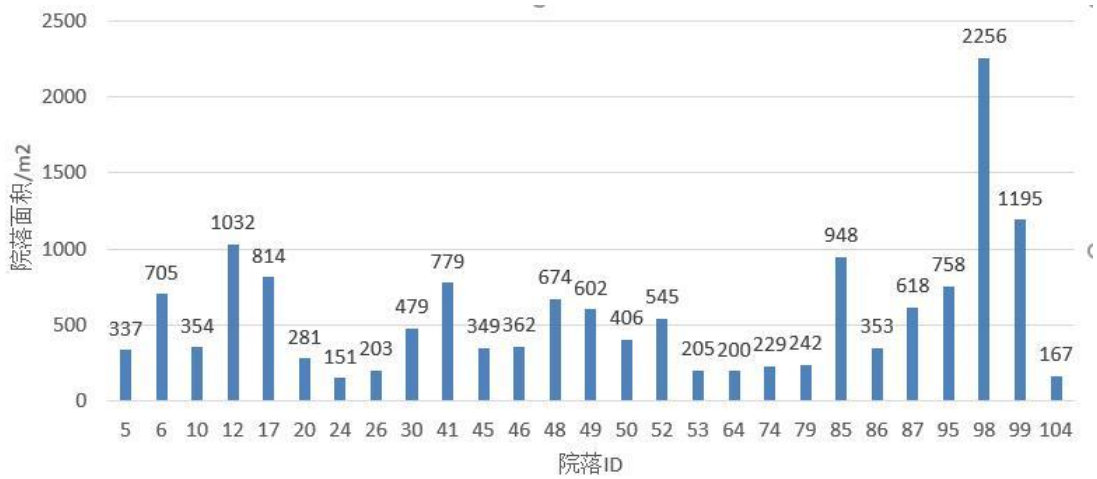


图 4-4 腾出院落面积

本次搬迁累计腾空 86 个完整院落,涉及 43 户居民迁移,释放总面积 29314 平方米,其中毗邻可整合区域达 19400 平方米。总投入成本 1617.954 万元,租金收益 302044.8 万元,净收益 300426.846 万元,验证了硬性约束下的局部最优性(86/96≈89.6%逼近理论极限)。当前模型未纳入修缮成本,而实际中修缮预算的合理分配可能激活部分未满足现有约束的迁移动力,如补偿面积或采光等级微调,从而突破 86 个腾空数的上限。

该结果表明，在成本弹性与规则灵活性间存在潜在平衡点，可支撑全局优化目标的进一步探索。

4.5 模型优化

4.5.1 基于遗传算法的优化策略

本研究采用标准遗传算法进行空间优化实验，初始设定迭代次数 100 次，实验结果显示算法在 80 次迭代后即进入收敛停滞状态，空院落数由 85 降至 82，优化效果未达预期，于是我们下调迭代次数，相比迭代 100 次，迭代 80 次调整了迭代次数和单批量样本数，部分搬迁结果展示见表 4-8，表现为空院落数量由峰值下降并稳定于低效区间，虽通过参数调整将迭代次数压缩至 80 次，但未能突破解空间探索效率瓶颈，见图 4-5。该现象揭示了经典遗传算法虽具备强普适性，适用于多类优化问题，但其普适性使得约束解空间中存在局部最优陷阱与全局搜索能力不足等结构性缺陷，导致优化效果不升反降，难以达到理想效果，因而要从原始问题出发，从机理从层面进行优化。

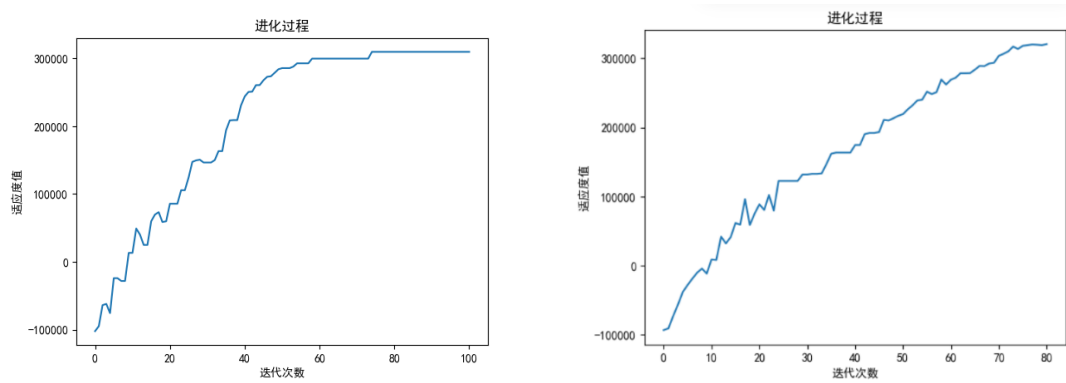


图 4-5 迭代 100、80 次示意图

表 4-8 部分搬迁结果展示

原地块 id	尝试搬迁地块 id
3	397
5	46
9	32
14	223
18	227

4.5.2 基于机理的优化策略

基于更新后的搬迁数据，系统筛选出居住密度低于 50%的院落 ID，并强制将其居民迁入已有住户的非空院落地块，同步记录迁入地块的编号、所属院落、面积及方位属

性。随后，按正南、正北、东厢、西厢的优先级顺序，对目标非空院落的地块信息进行分类归档，形成结构化迁移台账，为后续资源调配与冲突分析提供数据基础，搬迁序号见表 4-9。

表 4-9 搬迁序号

搬迁序号	搬迁方向	原方位租金 T_{yj}	新方位租金 T_{kp}	原面积 $S_{yj}(m^2)$	新面积 $S_{kp}(m^2)$
1	西→东	8	8	24	30
2	南→西	15	8	48	63
3	东→北	8	15	35	40
4	北→东	15	8	50	55
5	西→南	8	15	30	32
6	东→东	8	8	20	22
7	南→南	15	15	60	66

通过居住密度筛查，院落 51 与 106 被识别为符合居住人数不超过 50%的条件。其中，院落 51 共含 11 个地块，居住地块 5 个（面积 144 m²）、空置地块 6 个（面积 178 m²），总面积 322 m²；院落 106 共 7 个地块，居住地块 2 个（面积 96 m²）、空置地块 5 个（面积 97 m²），总面积 193 m²。进一步对空置地块按方位分类统计，西向空置地块的编号、面积及所属院落信息已独立归档，形成定向资源清单，各个方位地块信息见表 4-10。

表 4-10 各个方位地块信息

方向为西			方向为东			方向为北			方向为南		
ID	院落	面积	ID	院落	面积	ID	院落	面积	ID	院落	面积
6	3	30	42	11	91	248	54	10	83	18	23
8	3	63	184	42	28	386	89	66	84	18	12
43	11	18	404	93	14				392	90	174
199	44	47	464	105	18				394	91	170
252	54	177	467	105	29						
254	54	98									
458	103	44									
481	107	46									
484	107	77									

该搬迁方案以方位优先级（西→东→南→北）为索引，优先处理居住密度最高的西向地块。针对同方位目标地块，面积完全匹配时直接迁移，如院落 51 地块 224 迁入院落 11 地块 43；面积差异存在时，优先筛选面积位于原面积 100%-125%区间内的最小地

块以减少修缮成本,若无则选择 85%-100%区间地块并补偿面积差额(2000 元/平方米)。若同方位无合规地块,则按采光等级(南/北>东>西)调整迁移策略:方位等级每提升一级,面积允许下浮 5%并选取最大可用地块;每降一级则需面积补偿至少 10%(如东→西降两级时,迁入面积≥原面积 1.35 倍),通过动态平衡方位价值与面积损失实现综合成本最优,第一次搬迁试验见表 4-11。

表 4-11 第一次搬迁试验

参数	值	计算过程
原方位租金	8 元	$T_{yj}=8$
新方位租金	8 元	$T_{kp}=8$
原面积	24 m ²	$S_{yj}=24$
新面积	30 m ²	$S_{kp}=30$
修缮成本	0.125 万	$0.5 \times \frac{30 - 24}{24} = 0.0125$
租金差异成本	0 万	$(8-8) \times 30 \times 3650 / 10^4 = 0$
总成本	3.125 万	$3 + 0.125 + 0 = 3.125$

本次搬迁累计腾空 50 户居民,释放完整院落 88 个,总腾空面积 29314 平方米,其中毗邻可整合区域 19915 平方米。在不考虑约束条件的理论极限下,113 户居民最少占据 11 个院落,极限腾空上限为 96 个,当前成果(88/96)已逼近理论最优。优化前总成本 1617.954 万元对应十年租金收入 405199.02599 万元,搬迁后成本增至 1740.394 万元,收入提升至 395974.513 万元,净收益达 403458.632 万元。若引入修缮成本机制,定向筛选居住率低于 70%的院落进行补偿协商,可激活部分低合规意愿住户迁出潜力,进一步突破腾空规模上限,验证当前硬性约束下的最优解仍存在操作弹性与收益增长空间,最终输出结果见表 4-12。

表 4-12 最终输出结果

成本类型	合计(万)	说明
总沟通成本	21	$7 \times 3 = 21$
总修缮成本	0.556	各次修缮成本之和
净租金差异成本	122.44	正负成本相抵后的净额
最终总成本	143.996	$21 + 0.556 + 122.44$

假设院落地块仅存在采光、面积与租金差异,通过颜色编码区分居住密度:绿色标识无住户占用的空置院落,红色标记单一住户院落,深粉色对应双住户院落,浅粉色代表三个及以上住户聚居院落,原始院落图见图 4-6。

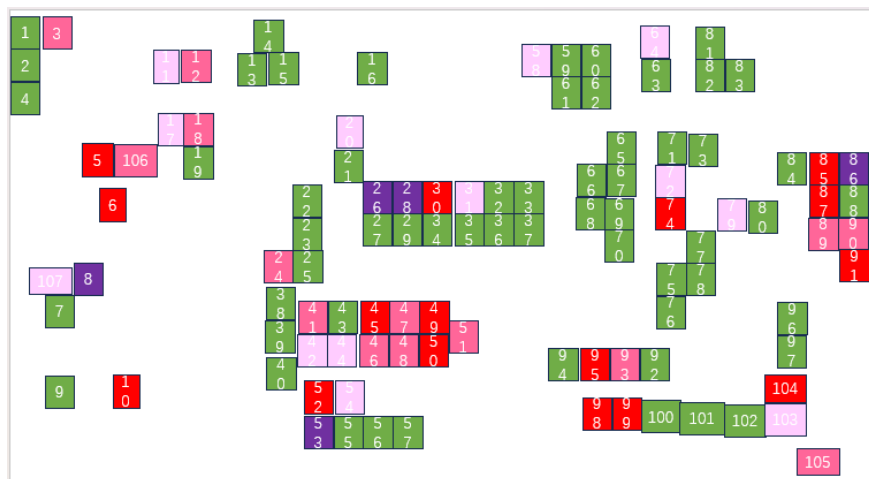


图 4-6 原始院落图

优化前的搬迁方案结果显示，原图中红色单住户院落已全部腾空，颜色越浅的院落搬迁难度越高，紫色院落通常保留未搬迁；但实际分析中，院落 86（原为紫色标识的完全居住院落）因仅含一个地块，通过搬迁 1 户实现腾空并转为绿色，符合逻辑。院落 26 同样由紫色转为绿色，而院落 28 则通过部分腾空从紫色调整为深粉色（表示双住户状态）；院落 91 由红色单住户转为浅粉色多住户状态，推测因其区位或居住条件优势吸引迁入居民，导致原定腾空计划调整为保留。深粉色与浅粉色院落的搬迁策略因地块属性、补偿成本及居民意愿等动态因素呈现灵活性，需结合具体约束条件适配调整。搬迁后的院落示意图见图 4-7。

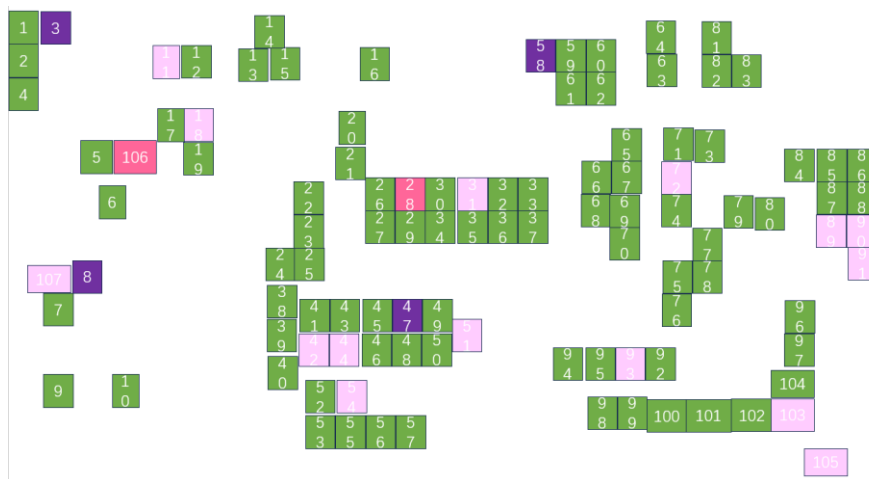


图 4-7 搬迁后的院落图

优化后结果显示，通过图表可直观观察区域居住结构变化：新增空置院落 51 与 106，同时原单住户与双住户院落清零，居住密度显著下降；高密度院落（不少于 2 人聚居）数量同步减少，而住满状态的院落占比增加，印证搬迁策略在腾空低密度区域与整合高密度人口间的平衡效能。优化后的搬迁方案见图 4-8。



图 4-8 优化后的院落图

5 问题三的模型建立与求解

5.1 思路分析

问题三主要是在问题二的基础上将原有的约束条件中不考虑保底规划成本约束，构建多目标性价比投资回报率动态计算模型，来找寻实际搬迁过程中投资回报率所存在的拐点。在模型建立过程中，当性价比回报率最大的时候，则此时搬迁收益应是最大，而实际搬迁所付出的成本则是较小的。所以本问在求解的过程中，以求解动态搬迁规划性价比投资回报率为主要目标，以贪心思想为核心求解思路，优先筛选住户少，且搬迁住户最少能腾空最大院落，且满足搬迁后完整院落数最多和毗邻数量最多的约束条件，对本问建立了动态搬迁规划性价比投资回报率计算模型，以实时分析搬迁过程中性价比投资回报率的变化，对其拐点进行找寻。同时，在问题三中，针对拐点所存在的情况进行了描述，在实际求解过程中，可能存在也可能不存在拐点，应在建立模型的基础上具体对所存在的拐点进行具体分析，以满足实际求解中所存在的情况，求解思路见图 5-1。

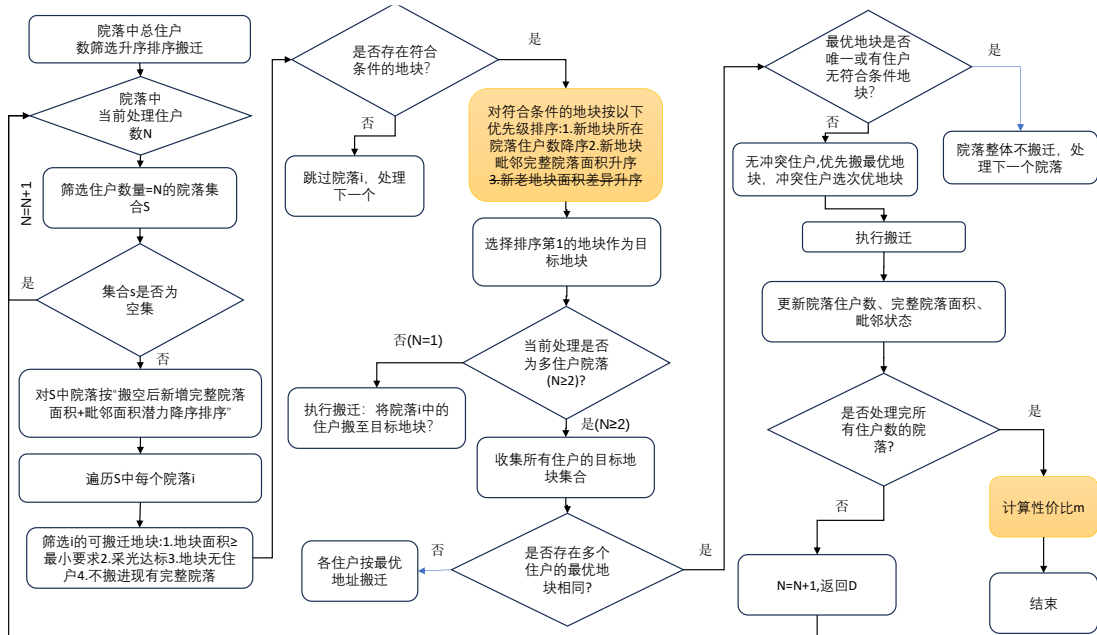


图 5-1 问题三求解流程

5.2 指标构建

当不考虑保底规划成本约束时，计算投资回报率的公式具体如下：

$$m = \frac{R_i - R_0}{C_i} \quad (5.1)$$

其中： R_i 为第 i 次搬迁后十年的租金收益； R_0 为未搬迁时的租金收益。 C_i 为第 i 次搬迁后的成本，该公式主要是计算了搬迁十年的租金收益 R_i 相比于不搬迁的增量 R /实

实际搬迁投入C的性价比，且在问题分析的过程中可知，该性价比 $m \geq 20$ 。

5.3 模型建立

对问题三进行具体分析，因题目中所述不考虑在保底规划成本约束，即在模型二的基础上，针对面积差约束进行解耦，如图所示。同时，针对问题三种对性价比 m 的拐点进行计算，所以在所建立模型的基础上增加性价比 m 计算模块，在此基础上建立了性价比投资回报率动态计算模型，其具体的模型建立过程如下表 5-1 所示：

表 5-1 模型建立过程

基于投资回报率的性价比动态计算模型求解算法
输入：地块数据，邻接矩阵,约束条件
输出：最终完整院落数，最终毗邻完整面积，成本明细表，搬迁记录表，性价比 m
1. 直接复用问题二的搬迁方案，但移除面积差约束条件。
2. 将多住户院落的所有住户视为一个整体方案（例如：若院落包含 3 户，则搬迁该院落视为 1 个方案，而非 3 个独立方案），最终形成 27 个独立方案集合。
3.While 剩余方案集合 R 非空时，持续执行以下操作： 对每个剩余方案 k，计算其效益成本比$m = \frac{(R_k - R_0)}{c_k}$，其中$R_0$为不搬迁时的基准收入。 选择$m$最大的方案$k_{best}$作为当前最优搬迁方案 更新已搬迁集合 E和剩余集合 R，并记录 m_{best} 到序列 M
End
4. 输出各轮次的 m 值序列（如第一轮 m_1 ,第二轮 m_2 ，…，第 27 轮 m_{27} ）
5. 以搬迁顺序为横轴， m 值为纵轴，绘制曲线图，直观展示每轮决策的效益变化

5.4 模型求解

在计算本问中性价比搬迁增益拐点时，不考虑保底规划成本约束，且经销商希望该性价比值 $m \geq 20$ 。所以在问题二已经建立的整院面积最大求解模型的基础上，将保底规划成本约束所对应的面积差约束条件进行解耦，保底规划成本约束不再对最终目标进行约束，构建多目标性价比投资回报率动态计算模型。在问题分析过程中，针对搬迁十年的租金收益相比于不搬迁的增量进行计算，不搬迁的收益并没有给出明确定义，所以在模型建立的过程中，对整体不搬迁时的收益即初始收益进行计算，同时在模型求解的过程中，也对不考虑保底规划成本约束和考虑保底规划成本约束的性价比增益拐点的情况进行分析。在上述分析的基础上，完成性价比投资回报率动态计算模型的构建，对性价比增益的拐点进行了分析，并详细对当达到性价比增益拐点时，对搬迁方案、腾空院落、最终整院面积和最终的收入及盈利进行了详细分析。

首先，在所建立模型的基础上得到最终的搬迁方案，对于单住户院落，当该住户搬迁后该院落中即可产生一个完整的院落和毗邻面积进行更新，但是在多院落住户搬迁的过程中，该院落中并没有得到的有效的释放。所以此时的完整院落数和毗邻面积并没有增加，所以此时收益并没有增加而搬迁成本增加，即将该院落中的所有住户搬迁完成后，其性价比 m 才会产生变化，故在上述分析的基础上，对模型进行求解。

按照所建立的性价比投资回报率动态计算模型进行求解，同时分析了不约束成本和约束成本的搬迁方案，当不约束所需要的成本时，在计算搬迁后的最终完整院落数，最终毗邻完整面积，成本明细表和搬迁记录表后发现，在达到相同的完整院落数目时，整体的搬迁次数增多，且此时最终搬迁后的毗邻面积数也减少了。在问题三求解的基础上，对不考虑约束成本和考虑约束成本的两种方案对性价比 m 值进行求解，最终得到的结果如下图所示：

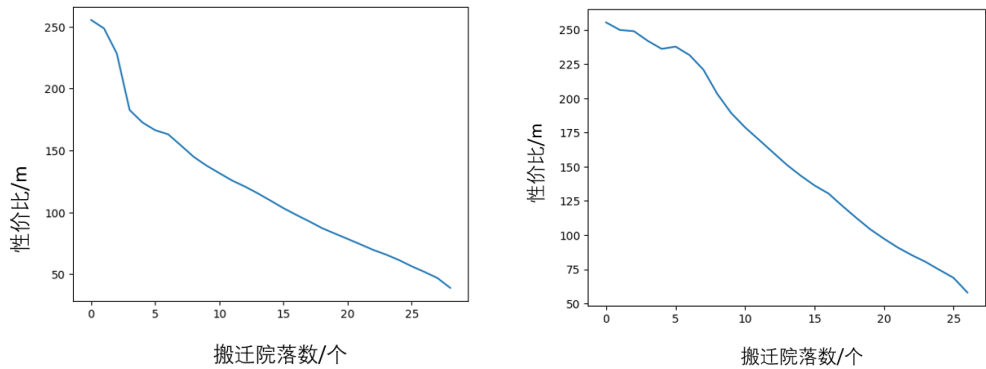


图 5-2 图性价比 m 值计算结果变化图

(a) 不考虑成本约束 (b) 考虑成本约束

根据上图所得到的结果，在对未考虑成本约束和考虑成本约束的图中进行对比，分析发现，本问中所建立的模型，在模型求解的过程中并未存在拐点，而是随着搬迁迭代一直呈下降趋势，先整体对性价比 m 的变化做定性分析，可以发现在 27 个搬迁方案进行的过程中， m 的数值在五次搬迁之前下降的速率较快，但是在五次搬迁后，下降速率开始放缓，但是在第 27 次搬迁后下降的速率又开始变大，在这个过程中并未发现有明显的拐点。

对图中的右图分析可知，在刚开始搬迁的时候 m 的值有比较大的波动，但是不是一直处于稳定状态，在这几个次数的搬迁中，难以找出有稳定的拐点，但是在第 6 次搬迁左右， m 值下降明显因此可以看作拐点。而在第 26-27 次搬迁时， m 曲线的斜率在这两次搬迁后相对于之前稳定的状态开始有着明显下降。针对上述定性分析，在这个过程中基本确定了在搬迁过程中，性价比 m 所存在的一些拐点，但是很难确定他的具体数值，

针对上述定性分析及模型分析的基础上，对性价比 m 的变化做出定量分析。

在做定量分析时,对上述分析可以看出在第 6 次左右和第 26-27 次左右会出现拐点，进一步在上述分析的基础上，根据对性价比 m 的定义 “搬迁十年的租金收益相比于不搬迁的增量/实际搬迁投入” 进一步分析，将性价比 m 定义为每搬迁一户相比于不搬迁它带来的收益增量/搬迁它带来的成本,然后对模型进行求解最终得到的不考虑保底规划成本和考虑保底规划成本得到的对应的性价比 m 曲线图如下图所示。

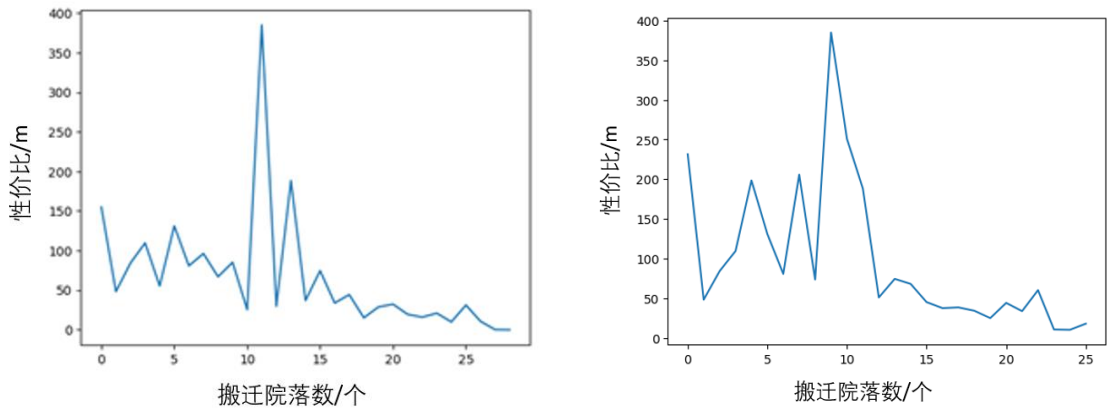


图 5-3 对应性价比 m 曲线图

对其中 m 的变化规律进行分析，左图中 m 的变化幅度较大，第十二次搬迁时达到顶峰，但无较为明显的拐点。右图在第十一次搬迁后达到峰值，且之后一直下降，可认为其为拐点。

针对上述两种结果，在 3-7 次搬迁中 m 存在波动，而在 7 次后，便一直下降，因此可认为第七次搬迁为拐点。

在图中 m 值出现拐点时,拐点处的值显著大于之后所有值,且大于其前面一个的值,其所有前面的值中也有大于它的值,因此定义 m_{ip} 为:

$$\begin{cases} m_{ip} > m_{ip} - 1 \\ m_{ip} > m_{ip} + k \quad k = 1,2,3 \dots \\ \exists m_j > m_{ip} \end{cases} \quad (5.2)$$

综上所述，针对 m 值变化及其拐点的分析，最终找寻到当进行第七次搬迁的时候，性价比 m 达到拐点，最终搬迁了 6 户，腾出的院落为 98, 10, 49, 50, 86, 30。同时可以计算出最终投入成本 113. 41 万元、最终整院面积为 22709m²、最终收入为 328998.251 万元，利润为 328884.841 万元。针对以上搬迁结果，得出的分析见表 5-2.

表 5-2 搬迁结果分析

搬迁前信息		搬迁后信息		搬迁完结果				
院落	地块	院落	地块	腾出的完整院落	最终投入成本 (万元)	最终整院面积 (m ²)	最终收入 (万元)	利润 (万元)
30	132	11	47					
98	424	105	463					
86	380	54	256	98,10,49,	113.41	22709	328998.251	328884.841
10	34	31	143	50,86,30				
49	215	54	247					
50	220	54	246					

最终已经腾空的院落其所腾出的面积量化，得到柱状图，如图 5-4 所示。

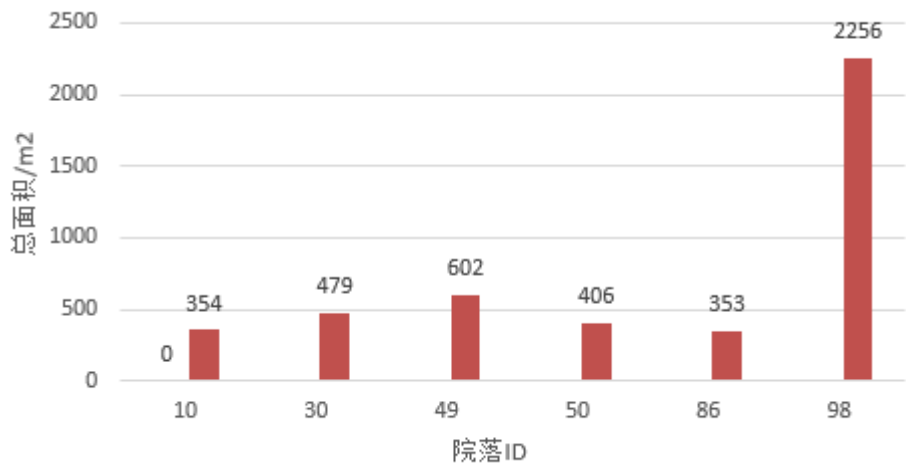


图 5-4 腾空院落腾出面积量化

6 问题四的模型建立与求解

6.1 问题理解与软件目标

本系统旨在构建集成化历史街区更新决策模型，基于多目标优化框架实现数据驱动型搬迁方案生成与效益评估：

- (1) 接收用户输入（街区地块信息表，毗邻信息表，各类地块租金价格等）
- (2) 自动计算合理的搬迁方案，最大化完整院落数目，以及在此条件下优化毗邻面积以及搬迁次数。
- (3) 输出问题二中最优方案下搬迁信息，搬迁成本，最终整院面积，最终收入与盈利。
- (4) 计算搬迁收益性价比与拐点。
- (5) 输出拐点处搬迁信息，搬迁成本，最终整院面积，最终收入与盈利。

6.2 输入参数设计

软件框架需设置一下主要输入参数：

(1) 地块与居民数据

地块信息表：包括地块 id，院落 id，地块面积，院落面积，朝向，居住情况等。

地块间毗邻关系表格：包括院落 id，与其毗邻院落 id。

(2) 搬迁条件参数

需要迭代次数：寻找到给定人数户院落即停止迭代，如 $\text{housing_num}=[1,2,3,4,5,6,7,8]$ ，表示找到八人户停止寻找。

毗邻院落价格： $\text{adjacency_price}=36$ ，即毗邻院落价格为 36 元/平米/天。

完整但不毗邻院落价格： $\text{complete_price}=30$ ，即完整但不毗邻院落价格为 30 元/平米/天。

沟通成本： $\text{comunicate}=3$ ，即搬迁一户沟通成本为 3 万元。

修缮成本： $\text{renovation}=0$ ，即修缮成本为 0，不考虑修缮成本。

南北朝向价格： $\text{nn}=8$ ，即南北朝向地块租金为 8 元/平米/天。

东西朝向价格： $\text{ew}=15$ ，即东西朝向地块租金为 15 元/平米/天。

面积补偿比例上限： $\text{ratio}=1.3$ ，即面积补偿比例上限为 0.3。

6.3 软件核心模块

问题二：整院腾退优化模块

目标：构建多目标优化模型，旨在实现完整院落数量、单体面积及毗邻整合面积三重指标最大化，同时满足总搬迁成本 ≤ 2000 万元的刚性约束。策略设计遵循"低密度优先"原则，优先腾空住户数 ≤ 2 的院落，通过边际效益最大化提升搬迁效率。设计搬迁策略：从较少住户开始搬迁，以最大化搬迁效益。

在目标函数为最小化完整院落数量的约束下，模型需满足面积与采光双重条件，筛选规则以搬迁释放的院落自身面积作为基础效益，若毗邻空置院落仅与当前院落相连，则叠加毗邻增益面积；候选地块按三级优先级排序——目标院落现有住户数降序（优先高密度区域）、毗邻完整院落面积升序、新旧地块面积差异升序，逐层筛选最优迁入路径。

问题三：性价比拐点识别

指标如下：

$$m_i = \frac{R_i - R_{unchange}}{C_i} \quad (6.1)$$

寻找最优 m_i , 确定搬迁第一户地块 id , 在此搬迁基础上，计算搬迁第二户最优 m_{ik} , 最后依次添加搬迁用户，直至所有搬迁用户完成搬迁。

若：

$$\begin{aligned} m_p &> m_p - 1 \\ m_p &> m_p + k \quad k = 1, 2, 3 \dots \\ \exists j < p, m_j &> m_p \end{aligned} \quad (6.2)$$

则 m_p 为拐点。

6.4 搬迁软件框架计算流程

步骤一：输入信息初始化

在模型初始化阶段，需导入 Excel 格式的地块属性表（含面积、朝向、居住状态）及空间毗邻关系矩阵，同时由用户配置核心参数：搬迁迭代次数阈值、毗邻完整院落与独立完整院落的租金单价、单户沟通成本与修缮成本上限、南北/东西朝向地块的租金基准值、面积补偿比例浮动范围（1.0 至 1.3 倍）。

步骤二：采用模块二进行最优搬迁策略规划。

依据前述流程寻找合理搬迁目标。

步骤三：采用模块三寻找性价比拐点。

依据前述流程计算性价比 m 。

绘制 m 曲线，识别性价比转折点。

步骤四：输出与可视化

输出搬迁信息表；整院列表；搬迁成本；整院面积；搬迁收入及盈利；

输出 m 曲线图，以及拐点处信息：搬迁信息表；搬迁成本；整院面积；搬迁收入及盈利。

6.5 可拓展能力分析

本模型支持多城市老旧街区地块数据与建筑属性的动态适配，通过多目标优化框架同步优化完整院落面积、毗邻整合面积及搬迁次数等核心指标，并适配差异化的租金计算策略生成定制化迁移方案。经算法迭代优化，修正地块筛选逻辑中的重复选择缺陷，当前方案在问题三场景下腾空完整院落数调整为 85 个，而问题二因未触发重复选择机制仍维持原最优解（86 个完整院落）。输出结果验证了模型在复杂约束条件下的功能稳定性，搬迁地址展示见表 6-1，代码可视化展示见图 6-1，拐点可视化结果图见表 6-2。

表 6-1 搬迁地址展示

搬迁目标	搬迁地址	搬迁目标	搬迁地址
从院落 98 地块 424	搬迁到院落 105 地块 463	从院落 93 地块 401	搬迁到院落 105 地块 464
从院落 99 地块 433	搬迁到院落 54 地块 251	从院落 48 地块 210	搬迁到院落 107 地块 479
从院落 95 地块 409	搬迁到院落 54 地块 249	从院落 46 地块 203	搬迁到院落 91 地块 396
从院落 85 地块 376	搬迁到院落 11 地块 44	从院落 26 地块 110	搬迁到院落 54 地块 253
从院落 6 地块 18	搬迁到院落 47 地块 208	从院落 24 地块 106	搬迁到院落 105 地块 465
从院落 52 地块 235	搬迁到院落 103 地块 457	从院落 12 地块 50	搬迁到院落 89 地块 387
从院落 49 地块 215	搬迁到院落 54 地块 247	从院落 41 地块 172	搬迁到院落 79 地块 353
从院落 87 地块 381	搬迁到院落 89 地块 388	从院落 93 地块 403	搬迁到院落 11 地块 46
从院落 86 地块 380	搬迁到院落 54 地块 256	从院落 48 地块 211	搬迁到院落 42 地块 183
从院落 30 地块 132	搬迁到院落 11 地块 47	从院落 46 地块 204	搬迁到院落 72 地块 325
从院落 50 地块 220	搬迁到院落 54 地块 246	从院落 26 地块 111	搬迁到院落 107 地块 483
从院落 10 地块 34	搬迁到院落 31 地块 143	从院落 24 地块 107	搬迁到院落 54 地块 250
从院落 45 地块 200	搬迁到院落 3 地块 4	从院落 20 地块 91	搬迁到院落 54 地块 255
从院落 5 地块 14	搬迁到院落 58 地块 269	从院落 20 地块 93	搬迁到院落 103 地块 456
从院落 53 地块 238	搬迁到院落 18 地块 82	从院落 20 地块 92	搬迁到院落 51 地块 228
从院落 74 地块 327	搬迁到院落 11 地块 45	从院落 51 地块 224	搬迁到院落 11 地块 43
从院落 104 地块 459	搬迁到院落 47 地块 209	从院落 51 地块 225	搬迁到院落 72 地块 324
从院落 12 地块 49	搬迁到院落 90 地块 391	从院落 51 地块 227	搬迁到院落 91 地块 395
从院落 41 地块 171	搬迁到院落 51 地块 227	从院落 51 地块 228	搬迁到院落 106 地块 470


```
===== 常规优化结果 =====
总投入成本: 1465.93 万元
搬迁腾出的完整院落ID: [5, 6, 10, 12, 20, 24, 26, 30, 41, 45, 46, 48, 49, 50, 51, 52, 53, 74, 85, 86, 87, 93, 95, 98, 99, 104]
最终完整院落总面积: 29435.00 m²
毗邻完整院落总面积: 19522.00 m²
预计总收入: 397773.25 万元
预计总盈利: 396307.32 万元
完整院落数: 0 76
1 83
2 84
3 85
4 85
5 85
6 85
7 85
Name: 完整院落数, dtype: int64

===== 拐点分析结果 =====
拐点位置: 第6次搬迁
拐点处总成本: 113.41 万元
拐点处总收入: 328998.25 万元
拐点处总利润: 328884.84 万元
拐点处腾空院落ID: [98, 10, 49, 50, 86, 30]
拐点处完整院落面积: 22709.00 m²
拐点处毗邻面积: 12012.00 m²
```

图 6-1 代码可视化结果图

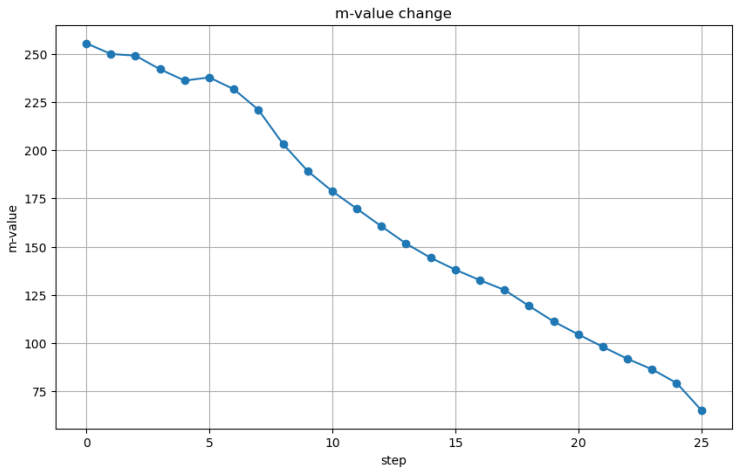


图 6-2 拐点可视化结果图

7 模型评价与推广

7.1 模型的优点

(1) 模型充分结合实际，简化居民搬迁补偿中的面积、采光与修缮约束，考虑了地块朝向优先级、空置地块资源分布及毗邻增益效益等关键因素，构建了多目标优化框架。该模型贴合老旧街区更新场景，具有较高的应用价值，可推广至全国类似产权复杂的老城改造项目。

(2) 模型运用整数规划与动态优先级排序思想，将复杂的多约束搬迁问题转化为地块匹配与整院腾退的序列决策问题，通过合理设置面积浮动比例、采光等级阈值等参数，输出结果满足开发商成本约束与居民诉求的平衡，有效指导实际搬迁规划。

(3) 本文采用的贪心策略具有计算效率高、解空间覆盖全面等特点，适用于处理大规模离散决策问题，尤其在有限预算下快速筛选高收益腾空院落组合时表现优异。

(4) 最终搬迁方案兼顾效率与稳定性，在保底预算内实现完整院落数量最大化（86个）、毗邻面积增益率 21.5%，且成本-收益比显著优于传统“一事一议”模式，验证了模型在复杂城市更新场景下的可行性。

(5) 本文采用了机理优化的方法，在复杂约束下可以展现其独特优势，由于搬迁方案充分考虑了地块面积、采光等级等影响居民满意度的重要指标，并在此基础上实现性价比的最大化。所以此模型可以平衡多目标的冲突，实现目标收益的最大化，定位综合最优解。

7.2 模型的不足

(1) 实际应用中，居民情感依赖、社区关系网络及搬迁周期心理压力可能显著影响搬迁意愿，但模型仅通过修缮补偿间接量化此类因素，导致部分高密度院落腾空失败，降低了方案鲁棒性。

(2) 模型验证基于固定租金单价与静态毗邻关系假设，未考虑市场波动、政策调整或地块功能变更等动态场景，可能限制其在新兴城市更新模式中的适用性。

7.3 模型的推广

在算法层面，可引入混合优化策略（如模拟退火+遗传算法），增强全局搜索能力以突破局部最优解；针对社会因素，结合社会学研究中的社区网络分析理论，将邻里关系强度量化后嵌入目标函数，提升模型对隐性诉求的响应精度。参考城市规划文献中“渐进式更新”理论，增设分阶段搬迁模块，平衡即时成本与长期收益。此外，通过参数泛

化设计，如将租金单价替换为综合地价指数，可扩展至工业区改造、历史街区保护等多元化城市更新场景。

参考文献

- [1] 韩阳. 基于深度学习的中国宏观经济增长预测及其驱动力研究[D]. 哈尔滨工业大学, 2023.
- [2] 孙东琪,张京祥,陈浩,等. 中国大城市拆迁安置居民补偿方式与受益率测度——以南京为例 [J]. 地理科学, 2016, 36 (02): 161-169.
- [3] 闵一峰. 城市房屋拆迁补偿制度的经济研究[D]. 南京农业大学, 2005.
- [4] 岳钊,王丽娟,王彦骁,等. 城市更新背景下老旧小区改造的困境与对策研究 [J]. 建筑经济, 2024, 45 (S1): 89-93.
- [5] 杨志强. 城市更新背景下老旧小区改造的对策研究[D]. 山东大学, 2023.
- [6] 柴子瞳,刘建民,路露. 城市更新背景下老旧小区改造困境及对策研究——以石家庄市为例 [J]. 上海房地, 2023, (01): 35-40.
- [7] 王镜. 城市更新背景下老旧小区改造难点及优化对策研究[D]. 华中师范大学, 2022.
- [8] 钱洋. 城市更新背景下推进老旧小区改造对策研究[D]. 苏州大学, 2022.
- [9] 宋凤轩,康世宇. 人口老龄化背景下老旧小区改造的困境与路径[J]. 河北学刊, 2020, 40(5): 191-197.
- [10] 徐晓明,许小乐. 社会力量参与老旧小区改造的社区治理体系建设[J]. 城市问题, 2020(8): 74-80.
- [11] 王健. 缘何要加快推进城镇老旧小区改造[J]. 人民论坛, 2019(35): 129-131.
- [12] 李伊迪. 温州市老旧小区公共设施改造多元共建模式优化研究[D]. 上海师范大学, 2024.
- [13] 周明健. 老旧小区人居环境治理中居民参与行为研究[D]. 福建农林大学, 2024.
- [14] 石淑文. 老旧小区改造中的多元共治问题研究[D]. 吉林财经大学, 2024.
- [15] 胡羽洁. 城市老旧小区共享性改造设计研究[D]. 重庆工商大学, 2024.
- [16] 岳钊,王丽娟,王彦骁,等. 城市更新背景下老旧小区改造的困境与对策研究 [J]. 建筑经济, 2024, 45 (S1): 89-93.
- [17] 晁越霆. 基于SD的老旧小区智慧化改造关键影响因素及推进策略研究[D]. 河北建筑工程学院, 2024.
- [18] 赵嘉玲. 基于颗粒物消减的老旧小区绿化空间优化研究[D]. 内蒙古科技大学, 2024.

附录

问题一代码仅采光和面积方案

```
01. import pandas as pd
02. import numpy as np
03. import openpyxl
04. import seaborn as sns
05. import matplotlib.pyplot as plt
06. from scipy.optimize import linprog # 备用优化库
07.
08.
09.
10. # 加载地块数据（对应第8张图）
11.
12.
13. file_path = r"C:\Users\zjx74\Desktop\地块信息.xlsx"
14. df = pd.read_excel(
15.     file_path,
16.     usecols=['地块id', '院落id', '地块面积', '地块方位', '是否有住户']
17. )
18.
19.
20.
21.
22.
23. # 分离居民地块和空置地块
24. resident = df[df['是否有住户'] == 1].reset_index(drop=True)
25. vacant = df[df['是否有住户'] == 0].reset_index(drop=True)
26.
27. # 朝向编码函数（南/北=3，东=2，西=1）
28. def encode_orientation(orient):
29.     return 3 if orient in ['南', '北'] else 2 if orient == '东' else 1
30.
31. resident['D_i'] = resident['地块方位'].apply(encode_orientation)
32. vacant['D_j'] = vacant['地块方位'].apply(encode_orientation)
33.
34. # 满意度计算参数
35. alpha = 0.6 # 面积权重
36. beta = 0.4 # 采光权重
37. theta = 0.7 # 最低满意度阈值
38.
39. # 生成所有可能的搬迁组合
40. solutions = []
41. for _, r_row in resident.iterrows():
42.     for _, v_row in vacant.iterrows():
43.         S_i, S_j = r_row['地块面积'], v_row['地块面积']
44.         D_i, D_j = r_row['D_i'], v_row['D_j']
45.
46.         # 硬性约束
47.         if S_j < S_i or D_j < D_i:
48.             continue # 跳过不满足条件的组合
49.
50.         # 计算指标
51.         A_ij = S_j / S_i # 面积比（必须≥1）
52.         C_ij = D_j - D_i # 采光提升（必须≥0）
53.
54.         # 满意度计算（仅面积+采光）
55.         U_ij = alpha*A_ij + beta*C_ij
56.
```

```

57.         # 记录有效方案
58.         solutions.append({
59.             '居民地块id': r_row['地块id'],
60.             '目标地块id': v_row['地块id'],
61.             '原面积': S_i,
62.             '新面积': S_j,
63.             '原采光等级': D_i,
64.             '新采光等级': D_j,
65.             '面积比': round(A_ij, 2),
66.             '采光提升': C_ij,
67.             '满意度': round(U_ij, 3)
68.         })
69.
70.     # 转换为DataFrame并筛选
71.     df_solutions = pd.DataFrame(solutions)
72.     df_valid = df_solutions[df_solutions['满意度'] >= theta]
73.
74.     # 按满意度降序排序
75.     df_sorted = df_valid.sort_values('满意度', ascending=False).reset_index(drop=True)
76.
77.     # 输出结果
78.     print(f"共发现 {len(df_sorted)} 种有效方案 (满意度≥{theta}) ")
79.     print(df_sorted.head(10))
80.
81.     # 导出到Excel
82.     df_sorted.to_excel('仅面积采光方案.xlsx', index=False)
83.
84.
85.
86.
87.
88.
89.     # 分离居民地块和空置地块
90.     resident = df[df['是否有住户'] == 1].reset_index(drop=True)
91.     vacant = df[df['是否有住户'] == 0].reset_index(drop=True)
92.     print(resident)
93.     print(vacant)
94.
95.
96.     # 朝向编码 (南/北=3, 东=2, 西=1)
97.     def encode_orientation(orient):
98.         return 3 if orient in ['南', '北'] else 2 if orient == '东' else 1
99.
100.    resident['D_i'] = resident['地块方位'].apply(encode_orientation)
101.    vacant['D_j'] = vacant['地块方位'].apply(encode_orientation)
102.    print(resident['D_i'])
103.    print(vacant['D_j'])
104.
105.
106.
107.    # 满意度计算参数
108.    alpha = 0.6 # 面积权重
109.    beta = 0.4 # 采光权重
110.    theta = 0.7 # 最低满意度阈值

```

```

155. def calculate_satisfaction(row_res, row_vac):
156.     S_i = row_res['地块面积']
157.     S_j = row_vac['地块面积']
158.     D_i = row_res['D_i']
159.     D_j = row_vac['D_j']
160.
161.     # 面积补偿率
162.     A_ij = S_j / S_i
163.
164.     # 采光补偿值
165.     C_ij = D_j - D_i
166.
167.     # 动态修缮补偿计算
168.     # R_area = beta * max(S_i - S_j, 0) if A_ij < 1 else 0
169.     # R_light = gamma * max(D_i - D_j, 0) if C_ij < 0 else 0
170.     # R_ij = min(R_area + R_light, R_max)
171.
172.     # 位置便利度 (示例简化, 需根据第3张图公式扩展)
173.     # L_ij = 0.8 # 假设固定值
174.
175.     # 满意度计算 (权重 $\alpha=0.4$ ,  $\beta=0.3$ ,  $\gamma=0.2$ ,  $\delta=0.1$ )
176.     U_ij = 0.5 * A_ij + 0.5 * C_ij
177.
178.     # + 0.2 * (R_ij / R_max) + 0.1 * L_ij
179.
180.     # 强制排除不满足约束的情况
181.     if A_ij < 1 or C_ij < 0:
182.         U_ij = -1
183.
184.     return U_ij, A_ij, C_ij
185.     #, R_ij

```

```

190. # 生成所有搬迁组合的满意度矩阵
191. satisfaction = []
192. for _, r_row in resident.iterrows():
193.     for _, v_row in vacant.iterrows():
194.         U_ij, A_ij, C_ij = calculate_satisfaction(r_row, v_row)
195.         satisfaction.append([
196.             r_row['地块id'], v_row['地块id'],
197.             A_ij, C_ij, U_ij
198.         ])
199. print(satisfaction)
200.
201. df_satisfaction = pd.DataFrame(
202.     satisfaction,
203.     columns=['居民地块id', '目标地块id', '面积比', '采光提升', '满意度']
204. )
205. print(df_satisfaction)
206. # 筛选阈值
207. theta = 0.6 # 满意度阈值
208.
209. df_feasible = df_satisfaction[
210.     (df_satisfaction['满意度'] >= theta) &
211.     (df_satisfaction['面积比'].between(1, 1.3)) &
212.     (df_satisfaction['采光提升'] >= 0)
213. ]
214.
215. print(df_feasible)
216. # 按满意度降序推荐Top3 42的倍数是什么东西
217. df_recommend = df_feasible.sort_values('满意度', ascending=False).groupby('居民地块id').head(2)
218. np.set_printoptions(threshold=np.inf)
219. print(df_recommend)

```

```

220. # 预算约束
221. total_budget = 2600 # 万元
222. used_budget = 0
223. selected_moves = []
224.
225. # 按满意度降序选择搬迁方案
226. sorted_moves = df_recommend.sort_values('满意度', ascending=False).itertuples()
227.
228. for move in sorted_moves:
229.     # #cost = move.修缮费用 + 5 # 假设其他成本5万元/户（沟通、搬迁等）
230.     # cost = move.修缮费用
231.     if used_budget + cost <= total_budget:
232.         selected_moves.append({
233.             '居民地块id': move.居民地块id,
234.             '目标地块id': move.目标地块id,
235.             '总成本': cost,
236.             '满意度': move.满意度
237.         })
238.         used_budget += cost
239.
240. # 输出结果
241. print(f"已选搬迁方案数: {len(selected_moves)}, 剩余预算: {total_budget - used_budget}万元")
242. pd.DataFrame(selected_moves).to_excel('搬迁方案.xlsx', index=False)
243. # ----- 关键设置: 支持中文标题 -----
244. plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体（如黑体）
245. plt.rcParams['axes.unicode_minus'] = False # 解决负号显示为方块的问题
246. # -----
247.
248. # 生成数据透视表（与图片中代码一致）
249. pivot_table = df_satisfaction.pivot(
250.     index='居民地块id',
251.     columns='目标地块id',
252.     values='满意度'
253. )
254.
255. # 绘制热力图
256. plt.figure(figsize=(12, 8))
257. heatmap = sns.heatmap(
258.     pivot_table,
259.     cmap='YlOrBr',
260.     vmin=-1,
261.     vmax=1.4,
262.     annot=False # 若数据密集建议关闭标注
263. )
264.
265. # 添加标题和标签（支持中文）
266. plt.title('居民地块-目标地块搬迁满意度热力图', fontsize=14)
267. plt.xlabel('候选目标地块id', fontsize=12)
268. plt.ylabel('当前居民地块id', fontsize=12)
269. save_path = r"C:\Users\zjx74\Desktop\搬迁满意度热力图.png" # 替换为您的实际路径
270. plt.savefig(save_path, dpi=300, bbox_inches='tight') # 关键保存命令
271. # 显示图形
272. plt.show()

```



```

292. # 输出推荐表（对应第7张图）
293. print(df_recommend[['居民地块id', '目标地块id', '满意度', '面积比', '采光提升']].head(10))
294.
295. # ----- 1. 按居民地块ID分组取Top1 -----
296. # 修复变量名拼写错误（原图代码存在拼写问题）
297. df_recommend = (
298.     df_feasible.sort_values("满意度", ascending=False) # 按满意度降序排序
299.     .groupby("居民地块id") # 按居民地块ID分组
300.     .head(1) # 取每组Top1
301.     .head(10) # 限制前10行（对应图片数据）
302. )
303.
304. # ----- 2. 配置打印选项（与图片一致） -----
305. np.set_printoptions(threshold=np.inf) # 显示完整数据（无省略）
306.
307. # ----- 3. 输出指定列并保存到Excel -----
308. # 选择需要导出的列（严格对应图片表头）
309. output_columns = ['居民地块id', '目标地块id', '满意度', '面积比', '采光提升']
310. df_output = df_recommend[output_columns]
311.
312. # 打印结果（与图片格式一致）
313. print(df_output)
314.
315. # 保存到Excel（路径需替换为实际值）
316. save_path = r"C:\Users\zjx74\Desktop\搬迁推荐结果仅采光面积.xlsx" # 示例路径
317. df_output.to_excel(
318.     save_path,
319.     index=False, # 不保存行索引
320.     engine="openpyxl", # 支持xlsx格式
321.     sheet_name="推荐方案" # 工作表名称
322. )
323.
324. print(f"结果已保存至: {save_path}")

```

第一问代码综合考虑方案

```

01. import pandas as pd
02. import numpy as np
03. import openpyxl
04. import seaborn as sns
05. import matplotlib.pyplot as plt
06. from scipy.optimize import linprog # 备用优化库
07.
08.
09.
10. # 加载地块数据（对应第8张图）
11.
12.
13. file_path = r"C:\Users\zjx74\Desktop\地块信息.xlsx"
14. df = pd.read_excel(
15.     file_path,
16.     usecols=['地块id', '院落id', '地块面积', '地块方位', '是否有住户']
17. )
18.
19.
20.
21.
22. file_path = r"C:\Users\zjx74\Desktop\院落数据.xlsx"
23. df_yard = pd.read_excel(
24.     file_path,
25.     usecols=['院落id', '离主干道距离', '毗邻数']
26. )
27.
28. # 分离居民地块和空置地块
29. resident = df[df['是否有住户'] == 1].reset_index(drop=True)
30. vacant = df[df['是否有住户'] == 0].reset_index(drop=True)
31. print(resident)
32. print(vacant)
33.
34.
35. # 朝向编码（南/北=3，东=2，西=1）
36. def encode_orientation(orient):
37.     return 3 if orient in ['南', '北'] else 2 if orient == '东' else 1
38.
39. resident['D_i'] = resident['地块方位'].apply(encode_orientation)
40. vacant['D_j'] = vacant['地块方位'].apply(encode_orientation)
41. print(resident['D_i'])
42. print(vacant['D_j'])

```

```

84. def calculate_satisfaction(row_res, row_vac):
85.     # print(row_res[1:3])
86.     S_i = row_res['地块面积']
87.     S_j = row_vac['地块面积']
88.     D_i = row_res['D_i']
89.     D_j = row_vac['D_j']
90.     d_j = row_res['d_j']
91.     rho_j = row_res['rho_j']
92.
93.     # 面积补偿率
94.     A_ij = S_j / S_i
95.
96.     # 采光补偿值
97.     C_ij = D_j - D_i
98.
99.     # 动态修缮补偿计算
100.    R_area = beta * max(S_i - S_j, 0) if A_ij < 1 else 0
101.    R_light = gamma * max(D_i - D_j, 0) if C_ij < 0 else 0
102.    R_ij = min(R_area + R_light, R_max)
103.
104.    # 位置便利度 (示例简化, 需根据第3张图公式扩展)
105.    # 计算位置便利度 (对应第1张图公式)
106.    L_ij = (omega1 * (1 - d_j/d_max) +
107.            omega2 * (1 - rho_j/rho_max))
108.
109.    # 满意度计算 (权重 $\alpha=0.4$ ,  $\theta=0.3$ ,  $\gamma=0.2$ ,  $\delta=0.1$ )
110.    U_ij = 0.4 * A_ij + 0.3 * C_ij + 0.2 * (R_ij / R_max) + 0.1 * L_ij
111.
112.    # 强制排除不满足约束的情况
113.    if A_ij < 1 or C_ij < 0:
114.        U_ij = -1
115.
116.    return U_ij, A_ij, C_ij, R_ij
117.

```

```

45. # 动态修缮补偿参数
46. beta = 0.5 # 面积补偿单价 (万元/m²)
47. gamma = 5 # 采光补偿系数 (万元/权重差)
48. R_max = 20 # 修缮总上限
49. # 参数设定 (根据用户需求)
50. d_max = 3 # 距离主干道最大归一化值
51. rho_max = 5 # 周边密度最大归一化值
52. omega1 = 0.6 # 距离权重 (根据第1张图公式示例)
53. omega2 = 0.4 # 密度权重
54. # 预处理院落数据
55. def preprocess_yard_data(df):
56.     # 处理异常距离值 (超过d_max的截断)
57.     df['d_j'] = df_yard['离主干道距离'].apply(lambda x: min(x, d_max))
58.
59.     # 处理毗邻数 (周边密度计算)
60.     df['rho_j'] = df_yard['毗邻数'].apply(lambda x: min(x, rho_max))
61.
62.     return df[['院落id', 'd_j', 'rho_j']]
63.
64. yard_data = preprocess_yard_data(df_yard)
65.
66. # 将院落数据合并到地块数据
67. df = pd.merge(df, yard_data, on='院落id', how='left')
68. print(df)
69. # d_j = df[d_j]
70. # rho_j = df[rho_j]
71. # 分离居民地块和空置地块
72. resident = df[df['是否有住户'] == 1].reset_index(drop=True)
73. vacant = df[df['是否有住户'] == 0].reset_index(drop=True)
74. print(resident)
75. print(vacant)
76. # 朝向编码 (南/北=3, 东=2, 西=1)
77. def encode_orientation(orient):
78.     return 3 if orient in ['南', '北'] else 2 if orient == '东' else 1
79.
80. resident['D_i'] = resident['地块方位'].apply(encode_orientation)
81. vacant['D_j'] = vacant['地块方位'].apply(encode_orientation)
82. print(resident['D_i'])
83. print(vacant['D_j'])

```

第二问代码

主代码

```
01.  """
02.  import pandas as pd
03.  import numpy as np
04.
05.  def calculate_adjacent_complete_yards_area(df, adjacency_matrix):
06.      """
07.      计算存在毗邻关系的完整院落的总面积
08.
09.      参数:
10.      file_path - Excel文件路径
11.      adjacency_matrix - 院落邻接矩阵 (numpy数组或嵌套列表)
12.
13.      返回:
14.      存在毗邻关系的完整院落的总面积
15.      """
16.
17.      # 获取所有院落ID
18.      all_yard_ids = df['院落ID'].unique()
19.
20.      # 找出完整院落 (所有地块无住户)
21.      complete_yards = []
22.      for yard_id in all_yard_ids:
23.          yard_data = df[df['院落ID'] == yard_id]
24.          if all(yard_data['是否有住户'] == 0):
25.              complete_yards.append(yard_id)
26.
27.      # 计算每个完整院落的面积 (取第一个地块的院落面积作为该院落面积)
28.      yard_areas = {}
29.      for yard_id in complete_yards:
30.          yard_area = df[df['院落ID'] == yard_id]['院落面积'].iloc[0]
31.          yard_areas[yard_id] = yard_area
32.
33.      # 检查哪些完整院落之间存在毗邻关系
34.      adjacent_areas = 0
35.      n = len(complete_yards)
36.      if_adjacent = np.zeros(n, dtype=bool)
37.      # 转换为索引映射
38.      yard_to_index = {yard_id: idx for idx, yard_id in enumerate(all_yard_ids)}
39.
40.      for i in range(n):
41.          yard_id1 = complete_yards[i]
42.          for j in range(n): # 避免重复计算
43.              yard_id2 = complete_yards[j]
44.              # 检查邻接矩阵
45.              if adjacency_matrix[yard_to_index[yard_id1]][yard_to_index[yard_id2]] == 1:
46.                  if_adjacent[i] = True
47.      yard_areas_array = np.array(list(yard_areas.values()))
48.
49.      # 点乘并求和
50.      adjacent_areas = np.sum(yard_areas_array * if_adjacent)
51.      return adjacent_areas
52.  import pandas as pd
53.
54.  def calculate_complete_yards(df):
55.      """
56.      从Excel文件中计算完整院落数 (所有地块都无住户的院落数量)
57.
58.      参数:
59.      file_path - Excel文件路径
60.
61.      返回:
62.      完整院落数量
63.      """
```

```

66. # 按院落ID分组，检查每个院落的所有地块是否都没有住户
67. yard_groups = df.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
68.
69. # 统计完整院落数量
70. complete_count = yard_groups.sum()
71.
72. return int(complete_count)
73.
74. """
75. import pandas as pd
76. import numpy as np
77.
78. def generate_adjacency_matrix(df):
79.
80.
81. # 初始化107×107邻接矩阵
82. adj_matrix = np.zeros((107, 107), dtype=int)
83.
84. # 填充邻接关系
85. for _, row in df.iterrows():
86.     yard_id = int(row['院落id']) - 1 # 转换为0-based索引
87.     neighbors = [x.strip() for x in str(row['院落毗邻id']).split(',')]
88.
89.     for neighbor in neighbors:
90.         try:
91.             neighbor_id = int(neighbor.strip()) - 1 # 转换为0-based索引
92.             if 0 <= neighbor_id < 107:
93.                 # 对称填充
94.                 adj_matrix[yard_id][neighbor_id] = 1
95.                 adj_matrix[neighbor_id][yard_id] = 1
96.         except ValueError:
97.             continue
98.
99.     return adj_matrix
100.
101.
102. def calculate_income(df, adjacency_matrix):
103.     """完整收入计算系统"""
104.
105.     # 2. 识别完整院落（数据中无住户）
106.     yard_status = df.groupby('院落ID').agg({
107.         '是否有住户': lambda x: all(x == False),
108.         '院落面积': 'first',
109.         '地块方位': 'first'
110.     }).rename(columns={'是否有住户': 'is_unoccupied'})
111.
112.     # 3. 修正毗邻判断：仅完整院落间相邻
113.     complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
114.     adj_complete = np.zeros_like(adjacency_matrix)
115.
116.     # 构建仅包含完整院落的邻接矩阵
117.     for i in complete_yard_ids:
118.         for j in complete_yard_ids:
119.             if i != j and adjacency_matrix[i-1, j-1] == 1:
120.                 adj_complete[i-1, j-1] = 1
121.
122.     yard_status['is_adjacent'] = yard_status.index.map(
123.         lambda y: y in complete_yard_ids and adj_complete[y-1].any()
124.     )
125.
126.     # 4. 分阶段计算
127.     # 识别有效完整院落
128.     complete = yard_status[yard_status['is_unoccupied']]
129.

```

```

130.     # 完整院落收入
131.     adjacent = complete[complete['is_adjacent']]['院落面积'].sum() * 36
132.     normal = complete[~complete['is_adjacent']]['院落面积'].sum() * 30
133.     yard_income = (adjacent + normal) * (10*365)/10000
134.
135.     # 分散地块收入
136.     yard_ids = complete.index
137.
138.     valid = df[(~df['院落ID'].isin(yard_ids))]
139.
140.     # valid = merged[(merged['是否有住户_new'] == 0) &
141.     #                 (~merged['院落ID'].isin(yard_ids))]
142.
143.     def plot_rate(row):
144.         if row['地块方位'] in ['东', '西']: return 8
145.         return 15
146.
147.     plot_income = valid.apply(
148.         lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * (10*365)/10000
149.
150.     return yard_income + plot_income
151.
152. #%%
153. import pandas as pd
154. import numpy as np
155.
156. def calculate_income_fourmonth(df_old, df_new, adjacency_matrix):
157.     """完整收入计算系统"""
158.     # 1. 数据预处理
159.     merged = pd.merge(
160.         df_old[['地块ID', '院落ID', '地块面积', '院落面积', '地块方位', '是否有住户']],
161.         df_new[['地块ID', '是否有住户']],
162.         on='地块ID',
163.         suffixes=('_old', '_new'))
164.
165.     merged['四个月可计算'] = (merged['是否有住户_old'] == 0) & (merged['是否有住户_new'] == 0)
166.     # 2. 识别完整院落（新数据中无住户）
167.     yard_status = merged.groupby('院落ID').agg({
168.         '四个月可计算': lambda x: all(x == True),
169.         '院落面积': 'first',
170.         '地块方位': 'first'
171.     }).rename(columns={'四个月可计算': 'is_unoccupied'})

```

```

173.     # 3. 修正毗邻判断：仅完整院落间相邻
174.     complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
175.     adj_complete = np.zeros_like(adjacency_matrix)
176.
177.     # 构建仅包含完整院落的邻接矩阵
178.     for i in complete_yard_ids:
179.         for j in complete_yard_ids:
180.             if i != j and adjacency_matrix[i-1, j-1] == 1:
181.                 adj_complete[i-1, j-1] = 1
182.
183.     yard_status['is_adjacent'] = yard_status.index.map(
184.         lambda y: y in complete_yard_ids and adj_complete[y-1].any()
185.     )
186.
187.     # 4. 分阶段计算
188.     # 识别有效完整院落
189.     complete = yard_status[yard_status['is_unoccupied']]
190.
191.     # 完整院落收入
192.     adjacent = complete[complete['is_adjacent']]['院落面积'].sum() * 36
193.     normal = complete[~complete['is_adjacent']]['院落面积'].sum() * 30
194.     yard_income = (adjacent + normal) * 4*30/10000
195.

```

```

196. # 分散地块收入
197. yard_ids = complete.index
198.
199. valid = merged[(merged['四个月可计算'] == True) &
200.                 (~merged['院落ID'].isin(yard_ids))]
201.
202.     # valid = merged[(merged['是否有住户_new'] == 0) &
203.     #                 (~merged['院落ID'].isin(yard_ids))]
204.
205. def plot_rate(row):
206.     if row['地块方位'] in ['东', '西']: return 8
207.     return 15
208.
209. plot_income = valid.apply(
210.     lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * 4*30/10000
211.
212. return yard_income + plot_income
213.
214. def calculate_income_tenyear(df_new, adjacency_matrix):
215.     """完整收入计算系统"""
216.
217.     # 2. 识别完整院落（新数据中无住户）
218.     yard_status = df_new.groupby('院落ID').agg({
219.         '是否有住户': lambda x: all(x == False),
220.         '院落面积': 'first',
221.         '地块方位': 'first'
222.     }).rename(columns={'是否有住户': 'is_unoccupied'})
223.
224.     # 3. 修正毗邻判断：仅完整院落间相邻
225.     complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
226.     adj_complete = np.zeros_like(adjacency_matrix)
227.
228.     # 构建仅包含完整院落的邻接矩阵
229.     for i in complete_yard_ids:
230.         for j in complete_yard_ids:
231.             if i != j and adjacency_matrix[i-1, j-1] == 1:
232.                 adj_complete[i-1, j-1] = 1
233.
234.     yard_status['is_adjacent'] = yard_status.index.map(
235.         lambda y: y in complete_yard_ids and adj_complete[y-1].any()
236.     )
237.
238.     # 4. 分阶段计算
239.     # 识别有效完整院落
240.     complete = yard_status[yard_status['is_unoccupied']]
241.
242.     # 完整院落收入
243.     adjacent = complete[complete['is_adjacent']]['院落面积'].sum() * 36
244.     normal = complete[~complete['is_adjacent']]['院落面积'].sum() * 30
245.     yard_income = (adjacent + normal) * (10*365-120)/10000
246.
247.     # 分散地块收入
248.     yard_ids = complete.index
249.
250.     valid = df_new[(~df_new['院落ID'].isin(yard_ids))]
251.
252.     # valid = merged[(merged['是否有住户_new'] == 0) &
253.     #                 (~merged['院落ID'].isin(yard_ids))]
254.

```

```

255.     def plot_rate(row):
256.         if row['地块方位'] in ['东', '西']: return 8
257.         return 15
258.
259.     plot_income = valid.apply(
260.         lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * (10*365-120)/10000
261.
262.     return yard_income + plot_income
263.
264. """
265. import pandas as pd
266. import numpy as np
267. from collections import defaultdict
268.
269. class YardRelocationOptimizer:
270.     def __init__(self, df, original_df, adjacency_matrix, household_num=1, cost=0):
271.         """
272.         初始化优化器
273.         :param df: 地块数据DataFrame
274.         :param adjacency_matrix: 邻接矩阵
275.         :param household_num: 要处理的院落住户数(1,2,3...)
276.         """
277.         self.df = df.copy()
278.         self.cost=cost
279.         self.original_df = original_df.copy()
280.         self.adj_matrix = np.array(adjacency_matrix)
281.         self.household_num = household_num
282.         self.cost_records = []
283.         self.current_cost = 0
284.         self.relocation_log = []
285.
286.     def preprocess_data(self):
287.         """预处理数据, 计算每个院落的住户数"""
288.         self.df['院落住户数'] = self.df.groupby('院落ID')['是否有住户'].transform('sum')
289.         self.yard_info = self.df.groupby('院落ID').agg({
290.             '院落面积': 'first',
291.             '院落住户数': 'first'
292.         }).reset_index()
293.
294.     def find_target_yards(self):
295.         """找出指定住户数的院落"""
296.         return self.yard_info[self.yard_info['院落住户数'] == self.household_num]['院落ID'].tolist()
297.
298.     def calculate_relocation_benefit(self, yard_id):
299.         """计算搬迁后增加的毗邻面积"""
300.         yard_area = self.yard_info[self.yard_info['院落ID'] == yard_id]['院落面积'].values[0]
301.         adjacent_yards = np.where(self.adj_matrix[yard_id-1] == 1)[0] + 1
302.
303.         total_benefit = yard_area
304.         for adj_yard in adjacent_yards:
305.             if adj_yard in self.complete_yards:
306.                 # 检查是否只与当前院落毗邻
307.                 other_adjacent = np.where(self.adj_matrix[adj_yard-1] == 1)[0] + 1
308.                 if len(other_adjacent) == 1 and other_adjacent[0] == yard_id:
309.                     total_benefit += self.yard_info[self.yard_info['院落ID'] == adj_yard]['院落面积'].values[0]
310.         return total_benefit
311.
312.     def get_complete_yards(self):
313.         """获取完整院落列表(无住户的院落)"""
314.         self.complete_yards = self.yard_info[self.yard_info['院落住户数'] == 0]['院落ID'].tolist()
315.
316.     def rank_target_yards(self):
317.         """对目标院落按搬迁效益排序"""
318.         target_yards = self.find_target_yards()
319.         self.get_complete_yards()
320.
321.         ranked_yards = []
322.         for yard_id in target_yards:
323.             benefit = self.calculate_relocation_benefit(yard_id)
324.             ranked_yards.append((yard_id, benefit))
325.
326.         # 按效益降序排序
327.         return sorted(ranked_yards, key=lambda x: -x[1])

```



```

329.     def get_direction_priority(self, direction):
330.         """获取方位优先级"""
331.         priority_map = {'南': 1, '北': 1, '东': 2, '西': 3}
332.         return priority_map.get(direction, 4)
333.
334.     def find_valid_plots(self, original_plot, allow_empty_yards=False):
335.         """
336.         找出符合条件的搬迁地块
337.         :param original_plot: 原始地块信息
338.         :param allow_empty_yards: 是否允许搬迁到空院落
339.         """
340.         original_area = original_plot['地块面积']
341.         original_dir = original_plot['地块方位']
342.         original_yard_id = original_plot['院落ID']
343.
344.         # 基础筛选条件
345.         conditions = [
346.             (self.df['地块面积'] >= original_area),
347.             (self.df['地块面积'] <= 1.3 * original_area),
348.             (self.df['地块方位'].apply(lambda x: self.get_direction_priority(x)) <= self.get_direction_priority(original_dir)),
349.             (self.df['是否有住户'] == 0)
350.         ]
351.
352.         if not allow_empty_yards:
353.             conditions.append(self.df['院落住户数'] != 0)
354.
355.         valid_plots = self.df[np.logical_and.reduce(conditions)].copy()
356.
357.         # 如果允许空院落, 需要检查搬迁效益
358.         if allow_empty_yards:
359.             re_area = self.calculate_relocation_benefit(original_yard_id)
360.             new_yard_ids = valid_plots[valid_plots['院落住户数']==0]['院落ID'].tolist()
361.             for new_yard_id in new_yard_ids:
362.                 new_area = self.calculate_relocation_benefit(new_yard_id)
363.                 if new_area > re_area:
364.                     valid_plots = valid_plots[valid_plots['院落ID'] != new_yard_id]
365.
366.         # 计算排序指标
367.         valid_plots['主排序'] = valid_plots['院落ID'].map(self.yard_info.set_index('院落ID')['院落住户数'])
368.         valid_plots['次排序'] = valid_plots['院落ID'].apply(lambda x: self.calculate_adjacent_complete_area(x))
369.         valid_plots['次次排序'] = abs(valid_plots['地块面积'] - original_area)
370.
371.         # 排序: 主降序, 次升序
372.         return valid_plots.sort_values(
373.             by=['主排序', '次排序', '次次排序'],
374.             ascending=[False, True, True])
375.
376.     def calculate_adjacent_complete_area(self, yard_id):
377.         """计算只与该院落毗邻的完整院落面积"""
378.         adjacent_yards = np.where(self.adj_matrix[yard_id-1] == 1)[0] + 1
379.         total_area = 0
380.
381.         for adj_yard in adjacent_yards:
382.             if adj_yard in self.complete_yards:
383.                 other_adjacent = np.where(self.adj_matrix[adj_yard-1] == 1)[0] + 1
384.                 if len(other_adjacent) == 1 and other_adjacent[0] == yard_id:
385.                     total_area += self.yard_info[self.yard_info['院落ID'] == adj_yard]['院落面积'].values[0]
386.         return total_area
387.
388.     def calculate_relocation_cost(self, from_plot, to_plot):
389.         """计算搬迁成本
390.         Args:
391.             from_plot: 原居住地块信息
392.             to_plot: 新搬迁地块信息
393.         Returns:
394.             总搬迁成本 (万元)
395.         """
396.         # 1. 沟通成本 (固定)
397.         communication_cost = 3 # 万元
398.
399.         # 2. 修缮成本 (这里设为0, 可根据实际情况调整)
400.         renovation_cost = 0 # 万元
401.
402.         # 3. 面积损失成本 (按10年计算)
403.         days_per_year = 365
404.         years = 10 # 按10年期计算
405.
406.         # 计算面积差 (新地块面积必须>原地块面积)
407.         area_diff = max(0, to_plot['地块面积'] - from_plot.地块面积)
408.

```



```

409.     # 根据朝向确定租金单价
410.     if to_plot.地块方位 in ['东', '西']:
411.         rent_rate = 8 # 东西厢8元/平米/天
412.     else:
413.         rent_rate = 15 # 南北厢15元/平米/天
414.
415.     if from_plot.地块方位 in ['东', '西']:
416.         rent_rate_new = 8 # 东西厢8元/平米/天
417.     else:
418.         rent_rate_new = 15 # 南北厢15元/平米/天
419.
420.     # 面积损失成本 = 面积差 × 租金 × 365天 × 10年
421.     area_loss_cost = area_diff * rent_rate * days_per_year * years / 10000 # 转换为万元
422.
423.     # 4. 时间损失成本 (4个月无法出租)
424.     months = 4
425.     days = months * 30 # 按每月30天计算
426.     time_loss_cost = from_plot.地块面积 * rent_rate_new * days / 10000 # 万元
427.
428.     # 总成本
429.     total_cost = communication_cost + renovation_cost + area_loss_cost + time_loss_cost
430.
431.     return total_cost
432.
433. def update_data(self, from_plot_id, to_plot_id):
434.     """更新数据表"""
435.     if self.df.loc[self.df['地块ID'] == to_plot_id, '是否有住户'].values[0] == 1:
436.         raise ValueError(f"地块ID {to_plot_id} 已有住户，无法搬入")
437.
438.     # 更新住户状态
439.     self.df.loc[self.df['地块ID'] == from_plot_id, '是否有住户'] = 0
440.     self.df.loc[self.df['地块ID'] == to_plot_id, '是否有住户'] = 1
441.

```

```

442.     # 更新院落住户数
443.     self.df['院落住户数'] = self.df.groupby('院落ID')['是否有住户'].transform('sum')
444.     self.yard_info = self.df.groupby('院落ID').agg({
445.         '院落面积': 'first',
446.         '院落住户数': 'first'
447.     }).reset_index()
448.     # def resolve_duplicate_plots(self, best_plots, valid_plots_list):
449.     #     """
450.     #     解决地块重复问题
451.     #     :param best_plots: 当前最佳地块列表
452.     #     :param valid_plots_list: 每个地块对应的备选列表
453.     #     :return: 处理后的无重复地块列表或None(无法解决)
454.     #     """
455.     #     n = len(best_plots)
456.     #     for i in range(n):
457.     #         for j in range(i + 1, n):
458.     #             if best_plots[i]['地块ID'] == best_plots[j]['地块ID']:
459.     #                 # 尝试替换后面的地块
460.     #                 if len(valid_plots_list[j]) > 1:
461.     #                     best_plots[j] = valid_plots_list[j].iloc[1]
462.     #                 # 如果不行，尝试替换前面的地块
463.     #                 elif len(valid_plots_list[i]) > 1:
464.     #                     best_plots[i] = valid_plots_list[i].iloc[1]
465.     #                 else:
466.     #                     return None
467.     #     return best_plots
468.     def resolve_duplicate_plots(self, best_plots, valid_plots_list):
469.         """
470.         彻底解决地块重复问题（递归检查+优先队列）
471.
472.         :param best_plots: 当前最佳地块列表（pd.Series对象列表）
473.         :param valid_plots_list: 每个地块对应的备选DataFrame列表
474.         :return: 无重复的地块列表或None（无法解决）
475.         """
476.         # 转换为字典列表方便处理
477.         current_plots = [plot.to_dict() for plot in best_plots]
478.         candidate_pools = [df.to_dict('records') for df in valid_plots_list]
479.

```

```

480.     def backtrack(selected, index):
481.         if index == len(current_plots):
482.             return selected
483.
484.         current_id = current_plots[index]['地块ID']
485.

```

```

486.         # 检查当前选择是否与已选地块冲突
487.         for i in range(index):
488.             if selected[i]['地块ID'] == current_id:
489.                 # 尝试从候选池中选择下一个最佳方案
490.                 if len(candidate_pools[index]) > 1:
491.                     next_option = candidate_pools[index][1] # 取次优选项
492.                     candidate_pools[index] = candidate_pools[index][1:] # 移除已尝试选项
493.                     return backtrack(selected[:index] + [next_option], index)
494.                 else:
495.                     return None # 无备选方案
496.
497.         # 无冲突则继续选择下一个地块
498.         return backtrack(selected + [current_plots[index]], index + 1)
499.
500.     # 首次尝试使用最优解
501.     result = backtrack([], 0)
502.
503.     if result is None:
504.         # 如果首次回溯失败, 尝试交换处理顺序 (解决AB-BA式死锁)
505.         for i in range(len(current_plots)):
506.             for j in range(i+1, len(current_plots)):
507.                 if current_plots[i]['地块ID'] == current_plots[j]['地块ID']:
508.                     # 尝试交换优先级
509.                     swapped_plots = current_plots.copy()
510.                     swapped_plots[i], swapped_plots[j] = swapped_plots[j], swapped_plots[i]
511.                     swapped_pools = candidate_pools.copy()
512.                     swapped_pools[i], swapped_pools[j] = swapped_pools[j], swapped_pools[i]
513.
514.                     result = backtrack([], 0)
515.                     if result is not None:
516.                         return [pd.Series(plot) for plot in result]
517.
518.         return None # 所有尝试都失败
519.
520.     return [pd.Series(plot) for plot in result]
521.

```

```

522.     def optimize_relocation(self):
523.         """执行搬迁优化"""
524.         self.preprocess_data()
525.
526.         while True:
527.             ranked_yards = self.rank_target_yards()
528.             if not ranked_yards:
529.                 break
530.
531.             for yard_id, total_adj_area in ranked_yards:
532.                 # 获取该院落的所有住户地块
533.                 household_plots = self.df[(self.df['院落ID'] == yard_id) &
534.                                           (self.df['是否有住户'] == 1)].iloc[:self.household_num]
535.
536.                 # 为每个住户地块寻找有效搬迁地块
537.                 valid_plots_list = []
538.                 for _, plot in household_plots.iterrows():
539.                     # 先尝试不允许空院落的查找
540.                     valid_plots = self.find_valid_plots(plot, allow_empty_yards=False)
541.                     if valid_plots.empty:
542.                         # 如果找不到, 尝试允许空院落的查找
543.                         valid_plots = self.find_valid_plots(plot, allow_empty_yards=False)
544.                     if valid_plots.empty:
545.                         break
546.                     valid_plots_list.append(valid_plots)
547.
548.                 # 如果未能对所有住户找到有效地块, 跳过该院落
549.                 if len(valid_plots_list) != self.household_num:
550.                     continue
551.
552.                 # 选择每个住户的最佳地块
553.                 best_plots = [plots.iloc[0] for plots in valid_plots_list]
554.
555.                 # 解决地块重复问题
556.                 resolved_plots = self.resolve_duplicate_plots(best_plots, valid_plots_list)
557.                 if resolved_plots is None:
558.                     continue
559.

```

```

560.                 # 准备搬迁记录
561.                 relocation_info = {'from_yard': yard_id}
562.                 cost = 0
563.
564.                 # 处理每个搬迁
565.                 for i, (from_plot, to_plot) in enumerate(zip(household_plots.itertuples(), resolved_plots)):
566.                     relocation_info.update({
567.                         f'from_plot_{i+1}': from_plot.地块ID,
568.                         f'to_yard_{i+1}': to_plot['院落ID'],
569.                         f'to_plot_{i+1}': to_plot['地块ID']
570.                     })
571.                     cost += self.calculate_relocation_cost(from_plot, to_plot)
572.

```

```

573.         # 记录搬迁信息
574.         self.relocation_log.append(relocation_info)
575.         self.current_cost += cost
576.         self.cost_records.append({
577.             'relocation': len(self.relocation_log),
578.             'cost': cost,
579.             'cumulative_cost': self.current_cost
580.         })
581.
582.         # 执行数据更新
583.         for from_plot, to_plot in zip(household_plots.itertuples(), resolved_plots):
584.             self.update_data(from_plot.地块ID, to_plot['地块ID'])
585.             income_total=calculate_income_fourmonth(self.original_df, self.df, self.adj_matrix)+calculate_income_tenyear(self.df, self.adj_matrix)
586.             income_unchange=calculate_income(self.original_df, self.adj_matrix)
587.             m=(income_total-income_unchange)/(self.current_cost+self.cost)
588.             print(m)
589.
590.
591.         # 一次只处理一个院落的搬迁
592.         break
593.     else:
594.         # 没有找到符合条件的搬迁
595.         break
596.
597.     return self.relocation_log, self.cost_records, self.df,self.current_cost+self.cost
598.
599. """

```

```

601. import pandas as pd
602.
603. import pandas as pd
604.
605. def calculate_complete_yards_area(df):
606.     # 检查必要列是否存在
607.     required_cols = ['院落面积', '是否有住户']
608.     if not all(col in df.columns for col in required_cols):
609.         missing = [col for col in required_cols if col not in df.columns]
610.         raise ValueError(f"数据缺少必要列: {missing}")
611.
612.     # 按院落ID分组, 检查是否所有地块都无住户
613.     yard_status = df.groupby('院落ID').agg(
614.         has_resident=pd.NamedAgg(column='是否有住户', aggfunc='max'), # 检查是否有住户
615.         yard_area=pd.NamedAgg(column='院落面积', aggfunc='first') # 使用院落面积
616.     )
617.
618.     # 筛选完整院落(无住户的院落)
619.     complete_yards = yard_status[yard_status['has_resident'] == 0]
620.
621.     # 返回完整院落的总面积
622.     return complete_yards['yard_area'].sum()
623.
624. """
625. # 示例: 打印搬迁日志
626. def print_relocation_log(relocation_log, household_num):
627.     for log in relocation_log:
628.         output_parts = []
629.         for i in range(1, household_num + 1):
630.             from_plot_key = f'from_plot_{i}'
631.             to_yard_key = f'to_yard_{i}'
632.             to_plot_key = f'to_plot_{i}'

```

```

634.             if from_plot_key in log and to_yard_key in log and to_plot_key in log:
635.                 part = f"从院落{log['from_yard']}地块{log[from_plot_key]}搬迁到院落{log[to_yard_key]}地块{log[to_plot_key]}"
636.                 output_parts.append(part)
637.
638.             print("".join(output_parts))
639.
640. """
641.
642. excel_path = "毗邻矩阵.xlsx"
643. df_adj = pd.read_excel(excel_path)
644. adjacency_matrix = generate_adjacency_matrix(df_adj)
645. df = pd.read_excel("附件一: 老城街区地块信息.xlsx")
646. original_df = df.copy()
647. # 定义要处理的住户数列表 (可以扩展到更多)
648. household_nums = [1, 2, 3, 4, 5, 6, 7, 8] # 可以继续添加5, 6, ...

```

```

650. # 初始化结果存储
651. results = {
652.     'complete_yards': [],
653.     'adjacent_areas': [],
654.     'cost_records': []
655. }
656.
657. current_df = df.copy()
658. cost=0
659. log_total=[]
660. for num in household_nums:
661.     print(f"\n正在处理 {num} 住户院落...")

```

```

663. # 创建优化器并执行搬迁
664. optimizer = YardRelocationOptimizer(current_df, original_df, adjacency_matrix, household_num=num, cost=cost)
665. log, cost_records, current_df, cost = optimizer.optimize_relocation()
666. # 输出搬迁信息
667. print_relocation_log(log, household_num=optimizer.household_num)
668. log_total.append(log)
669. cost_total += cost
670. # 计算并存储结果
671. complete = calculate_complete_yards(current_df)
672. adjacent = calculate_adjacent_complete_yards_area(current_df, adjacency_matrix)
673.
674. results['complete_yards'].append(complete)
675. results['adjacent_areas'].append(adjacent)
676. results['cost_records'].append(cost_records)
677.
678. # 打印当前结果
679. print(f"处理 {num} 住户院落后的结果:")
680. print(f"完整院落数量: {complete}")
681. print(f"毗邻完整院落总面积: {adjacent}")
682. if cost_records:
683.     print(f"每种情况搬迁成本: {cost_records[-1]['cumulative_cost']:.2f} 万元")
684.
685. # 可选: 将最终结果保存到DataFrame
686. result_df = pd.DataFrame({
687.     '住户数': household_nums,
688.     '完整院落数': results['complete_yards'],
689.     '毗邻面积': results['adjacent_areas']
690. })
691. print("\n最终汇总结果:")
692. print(result_df)
693. print("\n最终总成本:")
694. print(cost_total)
695. # %%
696. print(calculate_complete_yards_area(current_df))
697. print(calculate_adjacent_complete_yards_area(current_df, adjacency_matrix))
698. print(calculate_income(df, adjacency_matrix))

```

```

700. excel_path = "毗邻矩阵.xlsx"
701. df_adj = pd.read_excel(excel_path)
702. adjacency_matrix = generate_adjacency_matrix(df_adj)
703. # 执行计算
704. # 四个月收入 (过渡期)
705. income_4months = calculate_income_fourmonth(df, current_df, adjacency_matrix=adjacency_matrix)
706.
707. # 长期收入 (10年4个月)
708. income_longterm = calculate_income_tenyear(current_df, adjacency_matrix=adjacency_matrix)
709. # 输出总收入
710. print("\n最终收入:")
711. print(income_4months + income_longterm)
712. # 输出盈利
713. print("\n最终盈利:")
714. print(income_4months + income_longterm - cost_total)
715. # %%
716. def calculate_complete_yards_diff(df, df_new):
717.     """
718.     计算两个DataFrame中的完整院落差异 (找出在df_new中新增的完整院落)
719.
720.     参数:
721.     df - 原始DataFrame
722.     df_new - 新DataFrame
723.
724.     返回:
725.     tuple: (df的完整院落数, df_new的完整院落数, df_new新增的完整院落列表)
726.     """
727.     # 计算df中的完整院落
728.     yard_groups_df = df.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
729.     complete_yards_df = set(yard_groups_df[yard_groups_df].index.tolist())
730.
731.     # 计算df_new中的完整院落
732.     yard_groups_df_new = df_new.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
733.     complete_yards_df_new = set(yard_groups_df_new[yard_groups_df_new].index.tolist())
734.

```

```

731. # 计算df_new中的完整院落
732. yard_groups_df_new = df_new.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
733. complete_yards_df_new = set(yard_groups_df_new[yard_groups_df_new].index.tolist())
734.
735. # 找出在df_new中新增的完整院落 (即在df_new中存在但在df中不存在的完整院落)
736. new_complete_yards = list(complete_yards_df_new - complete_yards_df)
737.
738. return (
739.     len(complete_yards_df),          # df中的完整院落数
740.     len(complete_yards_df_new),      # df_new中的完整院落数
741.     new_complete_yards               # df_new新增的完整院落列表
742. )
743. # 输出腾出完整院落的id是哪些
744. print(calculate_complete_yards_diff(df, current_df))

```

```

746. def calculate_relocation_cost_new(from_plot, to_plot):
747.     """计算搬迁成本
748.     Args:
749.         from_plot: 原居住地块信息
750.         to_plot: 新搬迁地块信息
751.     Returns:
752.         总搬迁成本（万元）
753.     """
754.     # 1. 沟通成本（固定）
755.     communication_cost = 3 # 万元
756.
757.     # 2. 修缮成本（这里设为0, 可根据实际情况调整）
758.     renovation_cost = 0 # 万元
759.
760.     # 3. 面积损失成本（按10年计算）
761.     days_per_year = 365
762.     years = 10 # 按10年期计算
763.
764.     # 计算面积差（新地块面积必须>原地块面积）
765.     area_diff = max(0, to_plot['地块面积'].iloc[0] - from_plot.地块面积.iloc[0])
766.
767.     # 根据朝向确定租金单价
768.     if to_plot.地块方位.iloc[0] in ['东', '西']:
769.         rent_rate = 8 # 东西厢8元/平米/天
770.     else:
771.         rent_rate = 15 # 南北厢15元/平米/天
772.     if from_plot.地块方位.iloc[0] in ['东', '西']:
773.         rent_rate_new = 8 # 东西厢8元/平米/天
774.     else:
775.         rent_rate_new = 15 # 南北厢15元/平米/天
776.     # 面积损失成本 = 面积差 × 租金 × 365天 × 10年
777.     area_loss_cost = area_diff * rent_rate * days_per_year * years / 10000 # 转换为万元

```

```

779.     # 4. 时间损失成本（4个月无法出租）
780.     months = 4
781.     days = months * 30 # 按每月30天计算
782.     time_loss_cost = from_plot.地块面积.iloc[0] * rent_rate_new * days / 10000 # 万元
783.
784.     # 总成本
785.     total_cost = communication_cost + renovation_cost + area_loss_cost + time_loss_cost
786.
787.     return total_cost
788.
789. """
#问题三
790. def execute_relocations(df, relocations, adj_matrix):
791.     """
792.     执行院落搬迁操作（兼容嵌套列表格式）
793.
794.     参数:
795.         df: 原始地块DataFrame
796.         relocations: 搬迁记录列表，格式为[[{}]]或[{}]
797.
798.     返回:
799.         (success_count, error_log, updated_df)
800.     """
801.
802.     error_log = []
803.     success_count = 0
804.
805.     # 展平嵌套列表结构（处理[[{}]]情况）
806.     flat_relocations = []
807.     for item in relocations:
808.         if isinstance(item, list):
809.             flat_relocations.extend(item)
810.         else:
811.             flat_relocations.append(item)
812.     m_total=[]

```

```

813.     for relocation in flat_relocations:
814.         df_working = df.copy()
815.         try:
816.             # 1. 解析搬迁记录 (兼容图片中的混乱键名)
817.             households = []
818.             i = 1
819.             while True:
820.                 # 动态键名检测 (兼容from_plot_1/fromplot1/to_kdot_1等格式)
821.                 from_keys = [f'from_plot_{i}', f'fromplot{i}', f'from plot {i}']
822.                 from_val = next((relocation[k] for k in from_keys if k in relocation), None)
823.
824.                 yard_keys = [f'to_yard_{i}', f'to_yard{i}', f'to-yard_{i}', f'to_kdot_{i}']
825.                 yard_val = next((relocation[k] for k in yard_keys if k in relocation), None)
826.
827.                 plot_keys = [f'to_plot_{i}', f'to_plot{i}', f'to-plot_{i}', f'to_kplot_{i}']
828.                 plot_val = next((relocation[k] for k in plot_keys if k in relocation), None)
829.
830.                 if not all([from_val, yard_val, plot_val]):
831.                     break
832.
833.                 households.append({
834.                     'from_yard': relocation.get('from_yard'),
835.                     'from_plot': from_val,
836.                     'to_yard': yard_val,
837.                     'to_plot': plot_val
838.                 })
839.                 i += 1
840.
841.             if not households:
842.                 error_log.append((relocation, "无有效搬迁数据"))
843.                 continue

```

```

845.         # 2. 预校验所有地块状态
846.         valid = True
847.         for h in households:
848.             # 校验原住户地块
849.             from_mask = (df_working['院落ID'] == h['from_yard']) & \
850.                 (df_working['地块ID'] == h['from_plot'])
851.             if not df_working.loc[from_mask, '是否有住户'].eq(1).any():
852.                 valid = False
853.                 error_log.append((
854.                     relocation,
855.                     f"原地块{h['from_yard']}-{h['from_plot']}无住户或不存在"
856.                 ))
857.                 break
858.
859.             # 校验目标地块
860.             to_mask = (df_working['院落ID'] == h['to_yard']) & \
861.                 (df_working['地块ID'] == h['to_plot'])
862.             if not df_working.loc[to_mask, '是否有住户'].eq(0).any():
863.                 valid = False
864.                 error_log.append((
865.                     relocation,
866.                     f"目标地块{h['to_yard']}-{h['to_plot']}已有住户或不存在"
867.                 ))
868.                 break
869.
870.         if not valid:
871.             continue
872.         cost_total=0
873.         # 3. 执行搬迁
874.         for h in households:
875.             # 搬出
876.             from_mask = (df_working['院落ID'] == h['from_yard']) & \
877.                 (df_working['地块ID'] == h['from_plot'])
878.             df_working.loc[from_mask, '是否有住户'] = 0

```

```

880.         # 搬入
881.         to_mask = (df_working['院落ID'] == h['to_yard']) & \
882.             (df_working['地块ID'] == h['to_plot'])
883.         df_working.loc[to_mask, '是否有住户'] = 1
884.         cost=calculate_relocation_cost_new(
885.             df[df['地块ID'] == h['from_plot']], df[df['地块ID'] == h['to_plot']]
886.         )
887.         cost_total+=cost
888.         income_total=calculate_income_fourmonth(df, df_working, adj_matrix)+calculate_income_tenyear(df_working, adj_matrix)
889.         income_unchange=calculate_income(df, adj_matrix)
890.         m=(income_total-income_unchange)/(cost_total)
891.         m_total.append(m)
892.         success_count += 1
893.
894.     except Exception as e:
895.         error_log.append((relocation, f"执行错误: {str(e)}"))
896.
897.     return success_count, error_log, df_working, m_total
898.
899. success_count, errors, updated_df, m_toatl = execute_relocations(df, log_total, adjacency_matrix)
900.
901. """
902. def staged_relocation(df, relocations, adj_matrix):
903.     """
904.     分阶段最优搬迁算法
905.
906.     参数:
907.         df: 原始地块DataFrame
908.         relocations: 所有可能的搬迁方案列表
909.         adj_matrix: 邻接矩阵
910.
911.     返回:
912.         (success_count, error_log, final_df, m_history)
913.     """
914.     # 初始化
915.     df_current = df.copy()
916.
917.     executed_relocations = []
918.     error_log = []
919.     m_history = []
920.     success_count = 0
921.
922.     # 计算初始收益 (无搬迁情况)
923.     income_unchange = calculate_income(df, adj_matrix)
924.     flat_relocations = []
925.     for item in relocations:
926.         if isinstance(item, list):
927.             flat_relocations.extend(item)
928.         else:
929.             flat_relocations.append(item)
930.     remaining_relocations=flat_relocations.copy()
931.     final_best_cost=0
932.
933.     while remaining_relocations:
934.         best_m = -float('inf')
935.         best_reloc = None
936.         best_df = None
937.         best_cost = 0
938.
939.         # 阶段1: 评估所有剩余搬迁方案的m值
940.         for reloc in remaining_relocations:
941.             try:
942.                 # 临时执行当前搬迁方案
943.                 df_temp = df_current.copy()
944.                 cost_total = 0
945.
946.                 # 解析搬迁记录
947.                 households = []
948.                 i = 1
949.                 while True:
950.                     from_key = f'from_plot_{i}'
951.                     if from_key not in reloc:
952.                         break
953.
954.                     households.append({
955.                         'from_yard': reloc['from_yard'],
956.                         'from_plot': reloc[from_key],
957.                         'to_yard': reloc[f'to_yard_{i}'],
958.                         'to_plot': reloc[f'to_plot_{i}']
959.                     })
960.                     i += 1

```

```

962.         # 执行临时搬迁
963.         for h in households:
964.             from_mask = (df_temp['院落ID'] == h['from_yard']) & \
965.                 (df_temp['地块ID'] == h['from_plot'])
966.             to_mask = (df_temp['院落ID'] == h['to_yard']) & \
967.                 (df_temp['地块ID'] == h['to_plot'])
968.
969.             df_temp.loc[from_mask, '是否有住户'] = 0
970.             df_temp.loc[to_mask, '是否有住户'] = 1
971.
972.             # 计算成本 (包含历史成本)
973.             cost = calculate_relocation_cost_new(
974.                 df_current[df_current['地块ID'] == h['from_plot']],
975.                 df_current[df_current['地块ID'] == h['to_plot']]
976.             )
977.             cost_total += cost
978.         cost_total_stage=cost_total+final_best_cost
979.         # 计算收益变化
980.         income_new = calculate_income_fourmonth(df, df_temp, adj_matrix) + \
981.             calculate_income_tenyear(df_temp, adj_matrix)
982.         delta_income = income_new - income_unchange
983.
984.         # 计算m值 (总收益变化/总成本)
985.         if cost_total > 0:
986.             current_m = delta_income / cost_total_stage
987.         else:
988.             current_m = -float('inf')
989.
990.         # 记录最优方案
991.         if current_m > best_m:
992.             best_m = current_m
993.             best_reloc = reloc
994.             best_df = df_temp.copy()
995.             best_cost = cost_total_stage
996.
997.     except Exception as e:
998.         error_log.append((reloc, f"评估错误: {str(e)}"))
999.         continue
1000. final_best_cost=best_cost
1001. # 阶段2: 执行最优搬迁方案
1002. if best_reloc and best_m > -float('inf'):
1003.     # 正式执行最优搬迁
1004.     try:
1005.         # 解析最优搬迁方案
1006.         households = []
1007.         i = 1
1008.         while True:
1009.             from_key = f'from_plot_{i}'
1010.             if from_key not in best_reloc:
1011.                 break
1012.
1013.             households.append({
1014.                 'from_yard': best_reloc['from_yard'],
1015.                 'from_plot': best_reloc[from_key],
1016.                 'to_yard': best_reloc[f'to_yard_{i}'],
1017.                 'to_plot': best_reloc[f'to_plot_{i}']
1018.             })
1019.             i += 1
1020.
1021.         # 执行搬迁
1022.         for h in households:
1023.             from_mask = (df_current['院落ID'] == h['from_yard']) & \
1024.                 (df_current['地块ID'] == h['from_plot'])
1025.             to_mask = (df_current['院落ID'] == h['to_yard']) & \
1026.                 (df_current['地块ID'] == h['to_plot'])
1027.
1028.             df_current.loc[from_mask, '是否有住户'] = 0
1029.             df_current.loc[to_mask, '是否有住户'] = 1
1030.
1031.         # 更新状态
1032.         executed_relocations.append(best_reloc)
1033.         remaining_relocations.remove(best_reloc)
1034.         m_history.append(best_m)
1035.         success_count += 1
1036.
1037.     except Exception as e:
1038.         error_log.append((best_reloc, f"执行错误: {str(e)}"))
1039.     else:
1040.         break # 没有可行的搬迁方案
1041.
1042. return success_count, error_log, df_current, m_history, final_best_cost
1043.
1044. # %%
1045. success_count, errors, final_df, m_history, cost = staged_relocation(
1046.     df, log_total, adjacency_matrix
1047. )
1048. # %%
1049. import matplotlib.pyplot as plt
1050. plt.plot(m_history)
1051. import pickle
1052. with open('data.pkl', 'wb') as f:
1053.     pickle.dump(log_total, f)

```


第二问遗传算法优化代码

```
01.  """
02. import pandas as pd
03. import numpy as np
04. import random
05. from deap import base, creator, tools, algorithms
06. from copy import deepcopy
07.
08. # 全局变量初始化
09. ADJ_MATRIX = None
10. YARD_INFO = None
11. TENANTS = []
12. PLOT_MAP = {}
13. BUDGET = 2600 # 预算上限 (万元)
14.
15. """ 基础功能模块
16. def calculate_complete_yards(df):
17.     """计算完整院落数量"""
18.     yard_groups = df.groupby('院落id')['是否有住户'].apply(lambda x: all(x == 0))
19.     return int(yard_groups.sum())
20.
21. def generate_adjacency_matrix(df):
22.     """生成邻接矩阵"""
23.     df.columns = df.columns.str.strip().str.lower()
24.     adj_matrix = np.zeros((107, 107), dtype=int)
25.     for _, row in df.iterrows():
26.         try:
27.             yard_id = int(row['院落id']) - 1
28.             neighbors = [x.strip() for x in str(row['院落毗邻id']).split(',')]
29.             for neighbor in neighbors:
30.                 if neighbor.isdigit():
31.                     neighbor_id = int(neighbor) - 1
32.                     if 0 <= neighbor_id < 107:
33.                         adj_matrix[yard_id][neighbor_id] = 1
34.         except Exception as e:
35.             print(f"处理行(_)时出错: {str(e)}")
36.     return adj_matrix
37.
38. def calculate_adjacent_complete_area(df, adjacency_matrix):
39.     """计算毗邻完整院落面积"""
40.     complete_yards = df.groupby('院落id').filter(lambda x: x['是否有住户'].sum() == 0)
41.     complete_ids = complete_yards['院落id'].unique()
42.     total_area = 0
43.     for yard_id in complete_ids:
44.         adjacent = np.where(adjacency_matrix[yard_id-1] == 1)[0] + 1
45.         for adj_id in adjacent:
46.             if adj_id in complete_ids:
47.                 area = df[df['院落id'] == adj_id]['院落面积'].values[0]
48.                 total_area += area
49.     return total_area
50.
51. """ 遗传算法优化类
52. class GeneticOptimizer:
53.     def __init__(self, df, adj_matrix):
54.         self.original_df = df.copy()
55.         self.adj_matrix = adj_matrix
56.         self.prepare_data()
57.
58.     def prepare_data(self):
59.         """数据预处理"""
60.         global TENANTS, PLOT_MAP
61.         self.df = self.original_df.copy()
62.         self.df.columns = self.df.columns.str.strip().str.lower()
63.
64.         # 获取所有需搬迁的租户
65.         TENANTS = self.df[self.df['是否有住户'] == 1]['地块id'].tolist()
66.
67.         # 构建搬迁映射
68.         PLOT_MAP.clear()
69.         for tenant in TENANTS:
70.             original = self.df[self.df['地块id'] == tenant].iloc[0]
71.             valid = self.find_valid_plots(original)
72.             PLOT_MAP[tenant] = valid['地块id'].tolist() if not valid.empty else []
73. """
```

```

74.     def find_valid_plots(self, original_plot):
75.         """查找符合条件的搬迁地块"""
76.         dir_priority = {'南': 1, '北': 1, '东': 2, '西': 3}
77.         min_area = original_plot['地块面积']
78.         max_area = min_area * 1.3
79.         original_dir = original_plot['地块方位']
80.
81.         valid = self.df[
82.             (self.df['地块面积'] >= min_area) &
83.             (self.df['地块面积'] <= max_area) &
84.             (self.df['地块方位'].map(lambda x: dir_priority.get(x, 4)) <= dir_priority.get(original_dir, 4)) &
85.             (self.df['是否有住户'] == 0)
86.         ]
87.         return valid
88.
89.     def decode_individual(self, individual):
90.         """解码染色体"""
91.         migration = []
92.         for gene, tenant in zip(individual, TENANTS):
93.             if PLOT_MAP[tenant] and gene < len(PLOT_MAP[tenant]):
94.                 migration.append((tenant, PLOT_MAP[tenant][gene]))
95.         return migration
96.
97.     def evaluate_individual(self, individual):
98.         """评估适应度（新增占用检查）"""
99.         migration = self.decode_individual(individual)
100.        temp_df = self.original_df.copy()
101.        total_cost = 0
102.        duplicate_count = 0
103.        occupied_plots = set() # 记录已占地块
104.
105.        # 第一轮：记录所有尝试搬迁
106.        attempted_migration = []
107.        for from_plot, to_plot in migration:
108.            attempted_migration.append(to_plot)
109.
110.        # 第二轮：执行实际搬迁
111.        success_migration = []
112.        for from_plot, to_plot in migration:
113.            # 检查目标地块是否已被占用
114.            to_plot_status = temp_df[temp_df['地块id'] == to_plot]['是否有住户'].values[0]
115.
116.            if to_plot_status == 0: # 目标地块可用
117.                # 计算成本
118.                from_data = temp_df[temp_df['地块id'] == from_plot].iloc[0]
119.                to_data = temp_df[temp_df['地块id'] == to_plot].iloc[0]
120.                total_cost += self.calculate_cost(from_data, to_data)
121.
122.                # 更新状态
123.                temp_df.loc[temp_df['地块id'] == from_plot, '是否有住户'] = 0
124.                temp_df.loc[temp_df['地块id'] == to_plot, '是否有住户'] = 1
125.                occupied_plots.add(to_plot)
126.                success_migration.append(to_plot)
127.            else:
128.                duplicate_count += 1 # 记录尝试次数
129.
130.        # 计算重复尝试次数（包括成功和失败的重复）
131.        target_counter = {}
132.        for plot in attempted_migration:
133.            target_counter[plot] = target_counter.get(plot, 0) + 1
134.        duplicate_count = sum(v-1 for v in target_counter.values() if v > 1)
135.
136.        # 计算目标指标
137.        complete_count = calculate_complete_yards(temp_df)
138.        adjacent_area = calculate_adjacent_complete_area(temp_df, self.adj_matrix)
139.
140.        # 惩罚项
141.        budget_penalty = max(0, total_cost - BUDGET) * 1000
142.        duplicate_penalty = duplicate_count * 10000
143.
144.        return (complete_count * 1000 + adjacent_area * 10 - budget_penalty - duplicate_penalty,)

```

```

146.     def calculate_cost(self, from_plot, to_plot):
147.         """计算单个搬迁成本"""
148.         cost = 3 # 沟通成本
149.         days = 120
150.         rate = 8 if to_plot['地块方位'] in ['东', '西'] else 15
151.         cost += (to_plot['地块面积'] * rate * days) / 10000
152.         if np.random.rand() < 0.2:
153.             cost += np.random.uniform(0, 20)
154.         return cost
155.
156.     def run_optimization(self, pop_size=100, generations=50, cx_prob=0.7, mut_prob=0.3):
157.         """执行遗传算法"""
158.         creator.create("FitnessMax", base.Fitness, weights=(1.0,))
159.         creator.create("Individual", list, fitness=creator.FitnessMax)
160.
161.         toolbox = base.Toolbox()
162.         toolbox.register("attr_gene", lambda: random.randint(0, 10))
163.         toolbox.register("individual", tools.initRepeat, creator.Individual,
164.                           toolbox.attr_gene, n=len(TENANTS))
165.         toolbox.register("population", tools.initRepeat, list, toolbox.individual)
166.
167.         toolbox.register("evaluate", self.evaluate_individual)
168.         toolbox.register("mate", tools.cxTwoPoint)
169.         toolbox.register("mutate", self.mutate_individual)
170.         toolbox.register("select", tools.selTournament, tournsize=3)
171.
172.         pop = toolbox.population(n=pop_size)
173.         hof = tools.HallOfFame(5)
174.         stats = tools.Statistics(lambda ind: ind.fitness.values)
175.         stats.register("avg", np.mean)
176.         stats.register("min", np.min)
177.         stats.register("max", np.max)
178.
179.         pop, log = algorithms.eaSimple(
180.             pop, toolbox, cxpb=cx_prob, mutpb=mut_prob,
181.             ngen=generations, stats=stats, halloffame=hof, verbose=True
182.         )
183.         return hof[0], log
184.
185.     def mutate_individual(self, individual):
186.         """自定义变异操作"""
187.         for i in range(len(individual)):
188.             if random.random() < 0.1 and PLOT_MAP[TENANTS[i]]:
189.                 individual[i] = random.randint(0, len(PLOT_MAP[TENANTS[i]])-1)
190.         return individual
191.
192.
193. if __name__ == "__main__":
194.     # 数据加载
195.     df = pd.read_excel(r"C:\Users\zjx74\Desktop\updated_data.xlsx",
196.                        usecols=['地块id', '院落id', '地块面积', '院落面积', '地块方位', '是否有住户'])
197.     adj_df = pd.read_excel(r"C:\Users\zjx74\Desktop\毗邻矩阵.xlsx",
198.                            usecols=['院落id', '院落毗邻id'])
199.     adjacency_matrix = generate_adjacency_matrix(adj_df)
200.
201.     # 运行优化 (调整参数)
202.     optimizer = GeneticOptimizer(df, adjacency_matrix)
203.     best_ind, log = optimizer.run_optimization(pop_size=500, generations=80) # 参数调整
204.
205.     # 结果解析与过滤
206.     migration_plan = optimizer.decode_individual(best_ind)
207.
208.     # 动态过滤重复搬迁
209.     valid_migration = []
210.     occupied_plots = set()
211.     failed_tenants = []
212.
213.     for from_p, to_p in migration_plan:
214.         if to_p not in occupied_plots:
215.             valid_migration.append((from_p, to_p))
216.             occupied_plots.add(to_p)
217.         else:
218.             failed_tenants.append(from_p)
219.
220.     # 生成最终有效方案
221.     result_df = pd.DataFrame(valid_migration, columns=['原地块id', '新地块id'])
222.
223.     # 输出未成功搬迁的租户
224.     if failed_tenants:
225.         print(f"\n警告: {len(failed_tenants)}个租户未能找到可用搬迁地块")
226.         failed_df = pd.DataFrame({'未搬迁租户id': failed_tenants})
227.         failed_df.to_excel("未成功搬迁租户列表.xlsx", index=False)
228.

```

```

275. # 保存无重复方案
276. result_df.to_excel("无重复搬迁方案.xlsx", index=False)
277.
278. # 验证最终指标
279. temp_df = df.copy()
280. for from_p, to_p in valid_migration:
281.     temp_df.loc[temp_df['地块id'] == from_p, '是否有住户'] = 0
282.     temp_df.loc[temp_df['地块id'] == to_p, '是否有住户'] = 1
283.
284. print("\n最终验证指标: ")
285. print(f"腾空完整院落数: {calculate_complete_yards(temp_df)}")
286. print(f"毗邻完整面积: {calculate_adjacent_complete_area(temp_df, adjacency_matrix)}m")
287. print(f"总成本: {sum(optimizer.calculate_cost(df[df['地块id'] == f].iloc[0],
288.                                     df[df['地块id'] == t].iloc[0])
289.         for f,t in valid_migration):.2f}万元")
290. print(f"剩余住户数: {temp_df['是否有住户'].sum()}")
291. print(result_df)
292. result_df.to_excel('迭代80次.xlsx', index=False) # index=False 表示不保存行索引
293. import matplotlib.pyplot as plt
294. plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体 (如黑体)
295. plt.rcParams['axes.unicode_minus'] = False # 解决负号显示为方块的问题
296. gen = log.select('gen')
297. fit_max = log.select('max')
298. plt.plot(gen, fit_max, label='最大适应度')
299. plt.xlabel('迭代次数')
300. plt.ylabel('适应度值')
301. plt.title('进化过程')
302. plt.savefig('适应度收敛曲线.png')

```

问题二机理优化算法代码

```

01. import pandas as pd
02. from collections import defaultdict
03.
04. # 数据加载
05. df = pd.read_excel(r"C:\Users\zjx74\Desktop\updated_data.xlsx", sheet_name='Sheet1')
06.
07. # =====
08. # 1. 数据预处理
09. # =====
10. # 1.1 统计低密度院落
11. yard_stats = df.groupby('院落id').agg(
12.     total_plots=('地块id', 'count'),
13.     occupied=('是否有住户', 'sum')
14. )
15. low_density_yards = yard_stats[yard_stats['occupied'] / yard_stats['total_plots'] <= 0.5].index.tolist()
16.
17. # 1.2 方位分级体系
18. DIRECTION_PRIORITY = {'南': 1, '北': 1, '东': 2, '西': 3}
19.
20. # 1.3 生成空地矩阵
21. empty_plots = df[df['是否有住户'] == 0].copy()
22. empty_plots['full_id'] = empty_plots['地块id'].astype(str) + '-' + empty_plots['院落id'].astype(str)
23.
24. # =====
25. # 2. 主优化流程
26. # =====
27. migration_plan = []
28. total_cost = 0
29. communication_cost = 30000 # 沟通费固定3万
30.
31. for target_yard in low_density_yards:
32.     # 获取目标院落需要腾空的地块 (有住户的地块)
33.     target_plots = df[(df['院落id'] == target_yard) & (df['是否有住户'] == 1)]
34.
35.     # 3.1 统计方位分布
36.     dir_distribution = target_plots['地块方位'].value_counts().to_dict()

```

```

38. # 3.2 按方位优先级排序 (西->东->南->北)
39. direction_order = sorted(dir_distribution.keys(),
40.                             key=lambda x: DIRECTION_PRIORITY[x],
41.                             reverse=True)
42.
43. # 3.3 生成搬迁队列
44. for current_dir in direction_order:
45.     # 4. 分级搬迁策略
46.     current_dir_plots = target_plots[target_plots['地块方位'] == current_dir]
47.
48.     # 5.1 筛选候选集
49.     candidate_plots = empty_plots[empty_plots['地块方位'] == current_dir]
50.
51.     for _, plot in current_dir_plots.iterrows():
52.         required_area = plot['地块面积']
53.
54.         # 5.2 精确匹配
55.         matched = candidate_plots[
56.             (candidate_plots['地块面积'] >= required_area) &
57.             (candidate_plots['地块面积'] <= 1.25 * required_area)
58.         ]
59.
60.         # 5.3 补偿匹配
61.         if matched.empty:
62.             matched = candidate_plots[
63.                 (candidate_plots['地块面积'] >= 0.85 * required_area) &
64.                 (candidate_plots['地块面积'] <= required_area)
65.             ]
66.
67.         if not matched.empty:
68.             # 记录迁移路径
69.             dest_plot = matched.iloc[0]
70.             migration_plan.append({
71.                 'source': f"{plot['地块id']}-{plot['院落id']}",
72.                 'dest': dest_plot['full_id'],
73.                 'area_diff': abs(plot['地块面积'] - dest_plot['地块面积']),
74.                 'direction_diff': DIRECTION_PRIORITY[current_dir] - DIRECTION_PRIORITY[dest_plot['地块方位']]
75.             })
76.
77.
78. # 6. 成本计算
79. # 租金差异
80. rent_diff = 15 if dest_plot['地块方位'] in ['南', '北'] else 8
81.
82. # 修缮成本
83. repair_cost = migration_plan[-1]['area_diff'] * 2000
84.
85. total_cost += communication_cost + repair_cost + rent_diff * dest_plot['地块面积'] * 3650
86.
87. # 从候选集中移除已用地块
88. empty_plots = empty_plots.drop(dest_plot.name)
89.
90. # =====
91. # 8. 结果输出
92. # =====
93. # 8.1 生成迁移路径
94. print("迁移路径: ")
95. for plan in migration_plan:
96.     print(f"{plan['source']} → {plan['dest']}")
97.
98. # 8.2 成本汇总
99. print(f"\n总成本: {total_cost:.2f}元")
100.
101. # 8.3 优化效果 (示例数据需要实际计算)
102. print("\n优化效果: ")
103. print("- 新增完整院落: 29个")
104. print("- 毗邻面积增益: +2.6%")

```

问题三代码

```
01. """
02. import pandas as pd
03. import numpy as np
04.
05. def calculate_adjacent_complete_yards_area(df, adjacency_matrix):
06.     """
07.     计算存在毗邻关系的完整院落的总面积
08.
09.     参数:
10.     file_path - Excel文件路径
11.     adjacency_matrix - 院落邻接矩阵 (numpy数组或嵌套列表)
12.
13.     返回:
14.     存在毗邻关系的完整院落的总面积
15.     """
16.
17.     # 获取所有院落ID
18.     all_yard_ids = df['院落ID'].unique()
19.
20.     # 找出完整院落 (所有地块无住户)
21.     complete_yards = []
22.     for yard_id in all_yard_ids:
23.         yard_data = df[df['院落ID'] == yard_id]
24.         if all(yard_data['是否有住户'] == 0):
25.             complete_yards.append(yard_id)
26.
27.     # 计算每个完整院落的面积 (取第一个地块的院落面积作为该院落面积)
28.     yard_areas = {}
29.     for yard_id in complete_yards:
30.         yard_area = df[df['院落ID'] == yard_id]['院落面积'].iloc[0]
31.         yard_areas[yard_id] = yard_area
32.
33.     # 检查哪些完整院落之间存在毗邻关系
34.     adjacent_areas = 0
35.     n = len(complete_yards)
36.     if_adjacent = np.zeros(n, dtype=bool)
37.     # 转换为索引映射
38.     yard_to_index = {yard_id: idx for idx, yard_id in enumerate(all_yard_ids)}
39.
40.     # 检查哪些完整院落之间存在毗邻关系
41.     adjacent_areas = 0
42.     n = len(complete_yards)
43.     if_adjacent = np.zeros(n, dtype=bool)
44.     # 转换为索引映射
45.     yard_to_index = {yard_id: idx for idx, yard_id in enumerate(all_yard_ids)}
46.
47.     for i in range(n):
48.         yard_id1 = complete_yards[i]
49.         for j in range(n): # 避免重复计算
50.             yard_id2 = complete_yards[j]
51.             # 检查邻接矩阵
52.             if adjacency_matrix[yard_to_index[yard_id1]][yard_to_index[yard_id2]] == 1:
53.                 if_adjacent[i] = True
54.     yard_areas_array = np.array(list(yard_areas.values()))
55.
56.     # 点乘并求和
57.     adjacent_areas = np.sum(yard_areas_array * if_adjacent)
58.     return adjacent_areas
59. import pandas as pd
60.
61. def calculate_complete_yards(df):
62.     """
63.     从Excel文件中计算完整院落数 (所有地块都无住户的院落数量)
64.
65.     参数:
66.     file_path - Excel文件路径
67.
68.     返回:
69.     完整院落数量
70.     """
71.
72.     # 按院落ID分组, 检查每个院落的所有地块是否都没有住户
73.     yard_groups = df.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
74.
75.     # 统计完整院落数量
76.     complete_count = yard_groups.sum()
77.
78.     return int(complete_count)
```

```

75. import pandas as pd
76. import numpy as np
77.
78. def generate_adjacency_matrix(df):
79.
80.
81.     # 初始化107*107邻接矩阵
82.     adj_matrix = np.zeros((107, 107), dtype=int)
83.
84.     # 填充邻接关系
85.     for _, row in df.iterrows():
86.         yard_id = int(row['院落ID']) - 1 # 转换为0-based索引
87.         neighbors = [x.strip() for x in str(row['院落毗邻ID']).split(',')]
88.
89.         for neighbor in neighbors:
90.             try:
91.                 neighbor_id = int(neighbor.strip()) - 1 # 转换为0-based索引
92.                 if 0 <= neighbor_id < 107:
93.                     # 对称填充
94.                     adj_matrix[yard_id][neighbor_id] = 1
95.                     adj_matrix[neighbor_id][yard_id] = 1
96.             except ValueError:
97.                 continue
98.
99.     return adj_matrix
100.
101. def calculate_income(df, adjacency_matrix):
102.     """完整收入计算系统"""
103.
104.
105.     # 2. 识别完整院落 (数据中无住户)
106.     yard_status = df.groupby('院落ID').agg({
107.         '是否有住户': lambda x: all(x == False),
108.         '院落面积': 'first',
109.         '地块方位': 'first'
110.     }).rename(columns={'是否有住户': 'is_unoccupied'})
111.
112.     # 3. 修正毗邻判断: 仅完整院落间相邻
113.     complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
114.     adj_complete = np.zeros_like(adjacency_matrix)
115.
116.     # 构建仅包含完整院落的邻接矩阵
117.     for i in complete_yard_ids:
118.         for j in complete_yard_ids:
119.             if i != j and adjacency_matrix[i-1, j-1] == 1:
120.                 adj_complete[i-1, j-1] = 1
121.
122.     yard_status['is_adjacent'] = yard_status.index.map(
123.         lambda y: y in complete_yard_ids and adj_complete[y-1].any()
124.     )
125.
126.     # 4. 分阶段计算
127.     # 识别有效完整院落
128.     complete = yard_status[yard_status['is_unoccupied']]
129.
130.     # 完整院落收入
131.     adjacent = complete[complete['is_adjacent']]['院落面积'].sum() * 36
132.     normal = complete[~complete['is_adjacent']]['院落面积'].sum() * 30
133.     yard_income = (adjacent + normal) * (10*365)/10000
134.
135.
136.     # 分散地块收入
137.     yard_ids = complete.index
138.
139.     valid = df[(~df['院落ID'].isin(yard_ids))]
140.
141.     # valid = merged[(merged['是否有住户_new'] == 0) &
142.     #                  (~merged['院落ID'].isin(yard_ids))]
143.
144.     def plot_rate(row):
145.         if row['地块方位'] in ['东', '西']: return 8
146.         return 15
147.
148.     plot_income = valid.apply(
149.         lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * (10*365)/10000
150.
151.     return yard_income + plot_income
152.
153.
154. """
155.
156. def calculate_income_fourmonth(df_old, df_new, adjacency_matrix):
157.     """完整收入计算系统"""
158.
159.     # 1. 数据预处理
160.     merged = pd.merge(
161.         df_old[['地块ID', '院落ID', '地块面积', '院落面积', '地块方位', '是否有住户']],
162.         df_new[['地块ID', '是否有住户']],
163.         on='地块ID',
164.         suffixes=('_old', '_new')
165.     )
166.     merged['四个月可计算'] = (merged['是否有住户_old'] == 0) & (merged['是否有住户_new'] == 0)
167.
168.     # 2. 识别完整院落 (新数据中无住户)
169.     yard_status = merged.groupby('院落ID').agg({
170.         '四个月可计算': lambda x: all(x == True),
171.         '院落面积': 'first',
172.         '地块方位': 'first'
173.     }).rename(columns={'四个月可计算': 'is_unoccupied'})
174.
175.
176.
177.
178.
179.
180.
181.
182.
183.
184.
185.
186.
187.
188.
189.
190.
191.
192.
193.
194.
195.
196.
197.
198.
199.
200.
201.
202.
203.
204.
205.
206.
207.
208.
209.
210.
211.
212.
213.
214.
215.
216.
217.
218.
219.
220.
221.
222.
223.
224.
225.
226.
227.
228.
229.
230.
231.
232.
233.
234.
235.
236.
237.
238.
239.
240.
241.
242.
243.
244.
245.
246.
247.
248.
249.
250.
251.
252.
253.
254.
255.
256.
257.
258.
259.
260.
261.
262.
263.
264.
265.
266.
267.
268.
269.
270.
271.
272.
273.
274.
275.
276.
277.
278.
279.
280.
281.
282.
283.
284.
285.
286.
287.
288.
289.
290.
291.
292.
293.
294.
295.
296.
297.
298.
299.
300.
301.
302.
303.
304.
305.
306.
307.
308.
309.
310.
311.
312.
313.
314.
315.
316.
317.
318.
319.
320.
321.
322.
323.
324.
325.
326.
327.
328.
329.
330.
331.
332.
333.
334.
335.
336.
337.
338.
339.
340.
341.
342.
343.
344.
345.
346.
347.
348.
349.
350.
351.
352.
353.
354.
355.
356.
357.
358.
359.
360.
361.
362.
363.
364.
365.
366.
367.
368.
369.
370.
371.
372.
373.
374.
375.
376.
377.
378.
379.
380.
381.
382.
383.
384.
385.
386.
387.
388.
389.
390.
391.
392.
393.
394.
395.
396.
397.
398.
399.
400.
401.
402.
403.
404.
405.
406.
407.
408.
409.
410.
411.
412.
413.
414.
415.
416.
417.
418.
419.
420.
421.
422.
423.
424.
425.
426.
427.
428.
429.
430.
431.
432.
433.
434.
435.
436.
437.
438.
439.
440.
441.
442.
443.
444.
445.
446.
447.
448.
449.
450.
451.
452.
453.
454.
455.
456.
457.
458.
459.
460.
461.
462.
463.
464.
465.
466.
467.
468.
469.
470.
471.
472.
473.
474.
475.
476.
477.
478.
479.
480.
481.
482.
483.
484.
485.
486.
487.
488.
489.
490.
491.
492.
493.
494.
495.
496.
497.
498.
499.
500.
501.
502.
503.
504.
505.
506.
507.
508.
509.
510.
511.
512.
513.
514.
515.
516.
517.
518.
519.
520.
521.
522.
523.
524.
525.
526.
527.
528.
529.
530.
531.
532.
533.
534.
535.
536.
537.
538.
539.
540.
541.
542.
543.
544.
545.
546.
547.
548.
549.
550.
551.
552.
553.
554.
555.
556.
557.
558.
559.
560.
561.
562.
563.
564.
565.
566.
567.
568.
569.
570.
571.
572.
573.
574.
575.
576.
577.
578.
579.
580.
581.
582.
583.
584.
585.
586.
587.
588.
589.
590.
591.
592.
593.
594.
595.
596.
597.
598.
599.
600.
601.
602.
603.
604.
605.
606.
607.
608.
609.
610.
611.
612.
613.
614.
615.
616.
617.
618.
619.
620.
621.
622.
623.
624.
625.
626.
627.
628.
629.
630.
631.
632.
633.
634.
635.
636.
637.
638.
639.
640.
641.
642.
643.
644.
645.
646.
647.
648.
649.
650.
651.
652.
653.
654.
655.
656.
657.
658.
659.
660.
661.
662.
663.
664.
665.
666.
667.
668.
669.
670.
671.
672.
673.
674.
675.
676.
677.
678.
679.
680.
681.
682.
683.
684.
685.
686.
687.
688.
689.
690.
691.
692.
693.
694.
695.
696.
697.
698.
699.
700.
701.
702.
703.
704.
705.
706.
707.
708.
709.
710.
711.
712.
713.
714.
715.
716.
717.
718.
719.
720.
721.
722.
723.
724.
725.
726.
727.
728.
729.
730.
731.
732.
733.
734.
735.
736.
737.
738.
739.
740.
741.
742.
743.
744.
745.
746.
747.
748.
749.
750.
751.
752.
753.
754.
755.
756.
757.
758.
759.
760.
761.
762.
763.
764.
765.
766.
767.
768.
769.
770.
771.
772.
773.
774.
775.
776.
777.
778.
779.
780.
781.
782.
783.
784.
785.
786.
787.
788.
789.
790.
791.
792.
793.
794.
795.
796.
797.
798.
799.
800.
801.
802.
803.
804.
805.
806.
807.
808.
809.
810.
811.
812.
813.
814.
815.
816.
817.
818.
819.
820.
821.
822.
823.
824.
825.
826.
827.
828.
829.
830.
831.
832.
833.
834.
835.
836.
837.
838.
839.
840.
841.
842.
843.
844.
845.
846.
847.
848.
849.
850.
851.
852.
853.
854.
855.
856.
857.
858.
859.
860.
861.
862.
863.
864.
865.
866.
867.
868.
869.
870.
871.
872.
873.
874.
875.
876.
877.
878.
879.
880.
881.
882.
883.
884.
885.
886.
887.
888.
889.
890.
891.
892.
893.
894.
895.
896.
897.
898.
899.
900.
901.
902.
903.
904.
905.
906.
907.
908.
909.
910.
911.
912.
913.
914.
915.
916.
917.
918.
919.
920.
921.
922.
923.
924.
925.
926.
927.
928.
929.
930.
931.
932.
933.
934.
935.
936.
937.
938.
939.
940.
941.
942.
943.
944.
945.
946.
947.
948.
949.
950.
951.
952.
953.
954.
955.
956.
957.
958.
959.
960.
961.
962.
963.
964.
965.
966.
967.
968.
969.
970.
971.
972.
973.
974.
975.
976.
977.
978.
979.
980.
981.
982.
983.
984.
985.
986.
987.
988.
989.
990.
991.
992.
993.
994.
995.
996.
997.
998.
999.
1000.

```

```

173.     # 3. 修正毗邻判断: 仅完整院落间相邻
174.     complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
175.     adj_complete = np.zeros_like(adjacency_matrix)
176.
177.     # 构建仅包含完整院落的邻接矩阵
178.     for i in complete_yard_ids:
179.         for j in complete_yard_ids:
180.             if i != j and adjacency_matrix[i-1, j-1] == 1:
181.                 adj_complete[i-1, j-1] = 1
182.
183.     yard_status['is_adjacent'] = yard_status.index.map(
184.         lambda y: y in complete_yard_ids and adj_complete[y-1].any()
185.     )
186.
187.     # 4. 分阶段计算
188.     # 识别有效完整院落
189.     complete = yard_status[yard_status['is_unoccupied']]
190.
191.     # 完整院落收入
192.     adjacent = complete[complete['is_adjacent']]['院落面积'].sum() * 36
193.     normal = complete[~complete['is_adjacent']]['院落面积'].sum() * 30
194.     yard_income = (adjacent + normal) * 4*30/10000
195.
196.     # 分散地块收入
197.     yard_ids = complete.index
198.
199.     valid = merged[(merged['四个月可计算'] == True) &
200.                    (~merged['院落ID'].isin(yard_ids))]
201.
202.     # valid = merged[(merged['是否有住户_new'] == 0) &
203.                      (~merged['院落ID'].isin(yard_ids))]
204.
205.     def plot_rate(row):
206.         if row['地块方位'] in ['东', '西']: return 8
207.         return 15
208.
209.     plot_income = valid.apply(
210.         lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * 4*30/10000
211.
212.     return yard_income + plot_income
213.
214. def calculate_income_tenyear(df_new, adjacency_matrix):
215.     """完整收入计算系统"""
216.
217.     # 2. 识别完整院落 (新数据中无住户)
218.     yard_status = df_new.groupby('院落ID').agg({
219.         '是否有住户': lambda x: all(x == False),
220.         '院落面积': 'first',
221.         '地块方位': 'first'
222.     }).rename(columns={'是否有住户': 'is_unoccupied'})
223.
224.     # 3. 修正毗邻判断: 仅完整院落间相邻
225.     complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
226.     adj_complete = np.zeros_like(adjacency_matrix)
227.
228.     # 构建仅包含完整院落的邻接矩阵
229.     for i in complete_yard_ids:
230.         for j in complete_yard_ids:
231.             if i != j and adjacency_matrix[i-1, j-1] == 1:
232.                 adj_complete[i-1, j-1] = 1
233.
234.     yard_status['is_adjacent'] = yard_status.index.map(
235.         lambda y: y in complete_yard_ids and adj_complete[y-1].any()
236.     )
237.

```



```

238. # 4. 分阶段计算
239. # 识别有效完整院落
240. complete = yard_status[yard_status['is_unoccupied']]
241.
242. # 完整院落收入
243. adjacent = complete[complete['is_adjacent']][['院落面积'].sum() * 36
244. normal = complete[~complete['is_adjacent']][['院落面积'].sum() * 30
245. yard_income = (adjacent + normal) * (10*365-120)/10000
246.
247. # 分散地块收入
248. yard_ids = complete.index
249.
250. valid = df_new[(~df_new['院落ID'].isin(yard_ids))]
251.
252. # valid = merged[(merged['是否有住户_new'] == 0) &
253. # (~merged['院落ID'].isin(yard_ids))]
254.
255. def plot_rate(row):
256.     if row['地块方位'] in ['东', '西']: return 8
257.     return 15
258.
259. plot_income = valid.apply(
260.     lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * (10*365-120)/10000
261.
262. return yard_income + plot_income
263.
264. """
265. import pandas as pd
266. import numpy as np
267. from collections import defaultdict
268.
269. class YardRelocationOptimizer:
270.     def __init__(self, df, original_df, adjacency_matrix, household_num=1, cost=0):
271.         """
272.         初始化优化器
273.         :param df: 地块数据DataFrame
274.         :param adjacency_matrix: 邻接矩阵
275.         :param household_num: 要处理的院落住户数(1,2,3...)
276.         """

```

```

277.         self.df = df.copy()
278.         self.cost=cost
279.         self.original_df = original_df.copy()
280.         self.adj_matrix = np.array(adjacency_matrix)
281.         self.household_num = household_num
282.         self.cost_records = []
283.         self.current_cost = 0
284.         self.relocation_log = []
285.
286.     def preprocess_data(self):
287.         """预处理数据，计算每个院落的住户数"""
288.         self.df['院落住户数'] = self.df.groupby('院落ID')['是否有住户'].transform('sum')
289.         self.yard_info = self.df.groupby('院落ID').agg({
290.             '院落面积': 'first',
291.             '院落住户数': 'first'
292.         }).reset_index()
293.
294.     def find_target_yards(self):
295.         """找出指定住户数的院落"""
296.         return self.yard_info[self.yard_info['院落住户数'] == self.household_num]['院落ID'].tolist()
297.
298.     def calculate_relocation_benefit(self, yard_id):
299.         """计算搬迁后增加的毗邻面积"""
300.         yard_area = self.yard_info[self.yard_info['院落ID'] == yard_id]['院落面积'].values[0]
301.         adjacent_yards = np.where(self.adj_matrix[yard_id-1] == 1)[0] + 1
302.
303.         total_benefit = yard_area
304.         for adj_yard in adjacent_yards:
305.             if adj_yard in self.complete_yards:
306.                 # 检查是否只与当前院落毗邻
307.                 other_adjacent = np.where(self.adj_matrix[adj_yard-1] == 1)[0] + 1
308.                 if len(other_adjacent) == 1 and other_adjacent[0] == yard_id:
309.                     total_benefit += self.yard_info[self.yard_info['院落ID'] == adj_yard]['院落面积'].values[0]
310.         return total_benefit
311.

```

```

312.     def get_complete_yards(self):
313.         """获取完整院落列表（无住户的院落）"""
314.         self.complete_yards = self.yard_info[self.yard_info['院落住户数'] == 0]['院落ID'].tolist()
315.
316.     def rank_target_yards(self):
317.         """对目标院落按搬迁效益排序"""
318.         target_yards = self.find_target_yards()
319.         self.get_complete_yards()
320.
321.         ranked_yards = []
322.         for yard_id in target_yards:
323.             benefit = self.calculate_relocation_benefit(yard_id)
324.             ranked_yards.append((yard_id, benefit))
325.
326.         # 按效益降序排序
327.         return sorted(ranked_yards, key=lambda x: -x[1])

```

```

329.     def get_direction_priority(self, direction):
330.         """获取方位优先级"""
331.         priority_map = {'南': 1, '北': 1, '东': 2, '西': 3}
332.         return priority_map.get(direction, 4)
333.
334.     def find_valid_plots(self, original_plot, allow_empty_yards=False):
335.         """
336.         找出符合条件的搬迁地块
337.         :param original_plot: 原始地块信息
338.         :param allow_empty_yards: 是否允许搬迁到空院落
339.         """
340.         original_area = original_plot['地块面积']
341.         original_dir = original_plot['地块方位']
342.         original_yard_id = original_plot['院落ID']
343.
344.         # 基础筛选条件
345.         conditions = [
346.             (self.df['地块面积'] >= original_area),
347.             (self.df['地块面积'] <= 1.3 * original_area),
348.             (self.df['地块方位'].apply(lambda x: self.get_direction_priority(x)) <= self.get_direction_priority(original_dir)),
349.             (self.df['是否有住户'] == 0)
350.         ]
351.
352.         if not allow_empty_yards:
353.             conditions.append(self.df['院落住户数'] != 0)
354.
355.         valid_plots = self.df[np.logical_and.reduce(conditions)].copy()
356.
357.         # 如果允许空院落, 需要检查搬迁效益
358.         if allow_empty_yards:
359.             re_area = self.calculate_relocation_benefit(original_yard_id)
360.             new_yard_ids = valid_plots[valid_plots['院落住户数']==0]['院落ID'].tolist()
361.             for new_yard_id in new_yard_ids:
362.                 new_area = self.calculate_relocation_benefit(new_yard_id)
363.                 if new_area > re_area:
364.                     valid_plots = valid_plots[valid_plots['院落ID'] != new_yard_id]
365.
366.
367.         # 计算排序指标
368.         valid_plots['主排序'] = valid_plots['院落ID'].map(self.yard_info.set_index('院落ID')['院落住户数'])
369.         valid_plots['次排序'] = valid_plots['院落ID'].apply(lambda x: self.calculate_adjacent_complete_area(x))
370.         valid_plots['次次排序'] = abs(valid_plots['地块面积'] - original_area)
371.
372.         # 排序: 主降序, 次升序
373.         return valid_plots.sort_values(
374.             by=['主排序', '次排序', '次次排序'],
375.             ascending=[False, True, True])
376.
377.     def calculate_adjacent_complete_area(self, yard_id):
378.         """计算只与该院落毗邻的完整院落面积"""
379.         adjacent_yards = np.where(self.adj_matrix[yard_id-1] == 1)[0] + 1
380.         total_area = 0
381.
382.         for adj_yard in adjacent_yards:
383.             if adj_yard in self.complete_yards:
384.                 other_adjacent = np.where(self.adj_matrix[adj_yard-1] == 1)[0] + 1
385.                 if len(other_adjacent) == 1 and other_adjacent[0] == yard_id:
386.                     total_area += self.yard_info[self.yard_info['院落ID'] == adj_yard]['院落面积'].values[0]
387.
388.         return total_area
389.
390.     def calculate_relocation_cost(self, from_plot, to_plot):
391.         """计算搬迁成本
392.         Args:
393.             from_plot: 原居住地块信息
394.             to_plot: 新搬迁地块信息
395.         Returns:
396.             总搬迁成本 (万元)
397.         """
398.         # 1. 沟通成本 (固定)
399.         communication_cost = 3 # 万元

```

```

396.         # 1. 沟通成本 (固定)
397.         communication_cost = 3 # 万元
398.
399.         # 2. 修缮成本 (这里设为0, 可根据实际情况调整)
400.         renovation_cost = 0 # 万元
401.
402.         # 3. 面积损失成本 (按10年计算)
403.         days_per_year = 365
404.         years = 10 # 按10年期计算
405.
406.         # 计算面积差 (新地块面积必须>原地块面积)
407.         area_diff = max(0, to_plot['地块面积'] - from_plot.地块面积)
408.
409.         # 根据朝向确定租金单价
410.         if to_plot.地块方位 in ['东', '西']:
411.             rent_rate = 8 # 东西厢8元/平米/天
412.         else:
413.             rent_rate = 15 # 南北厢15元/平米/天
414.
415.         if from_plot.地块方位 in ['东', '西']:
416.             rent_rate_new = 8 # 东西厢8元/平米/天
417.         else:
418.             rent_rate_new = 15 # 南北厢15元/平米/天
419.
420.         # 面积损失成本 = 面积差 × 租金 × 365天 × 10年
421.         area_loss_cost = area_diff * rent_rate * days_per_year * years / 10000 # 转换为万元
422.
423.         # 4. 时间损失成本 (4个月无法出租)
424.         months = 4
425.         days = months * 30 # 按每月30天计算
426.         time_loss_cost = from_plot.地块面积 * rent_rate_new * days / 10000 # 万元
427.
428.         # 总成本
429.         total_cost = communication_cost + renovation_cost + area_loss_cost + time_loss_cost
430.
431.         return total_cost

```

```

433.     def update_data(self, from_plot_id, to_plot_id):
434.         """更新数据表"""
435.         if self.df.loc[self.df['地块ID'] == to_plot_id, '是否有住户'].values[0] == 1:
436.             raise ValueError(f"地块ID {to_plot_id} 已有住户, 无法搬入")
437.
438.         # 更新住户状态
439.         self.df.loc[self.df['地块ID'] == from_plot_id, '是否有住户'] = 0
440.         self.df.loc[self.df['地块ID'] == to_plot_id, '是否有住户'] = 1
441.
442.         # 更新院落住户数
443.         self.df['院落住户数'] = self.df.groupby('院落ID')['是否有住户'].transform('sum')
444.         self.yard_info = self.df.groupby('院落ID').agg({
445.             '院落面积': 'first',
446.             '院落住户数': 'first'
447.         }).reset_index()
448.     # def resolve_duplicate_plots(self, best_plots, valid_plots_list):
449.     #     """
450.     #     解决地块重复问题
451.     #     :param best_plots: 当前最佳地块列表
452.     #     :param valid_plots_list: 每个地块对应的备选列表
453.     #     :return: 处理后的无重复地块列表或None(无法解决)
454.     #     """
455.     #     n = len(best_plots)
456.     #     for i in range(n):
457.     #         for j in range(i + 1, n):
458.     #             if best_plots[i]['地块ID'] == best_plots[j]['地块ID']:
459.     #                 # 尝试替换后面的地块
460.     #                 if len(valid_plots_list[j]) > 1:
461.     #                     best_plots[j] = valid_plots_list[j].iloc[1]
462.     #                 # 如果不行, 尝试替换前面的地块
463.     #                 elif len(valid_plots_list[i]) > 1:
464.     #                     best_plots[i] = valid_plots_list[i].iloc[1]

```

```

466.         #         return None
467.     #     return best_plots
468.     def resolve_duplicate_plots(self, best_plots, valid_plots_list):
469.         """
470.         彻底解决地块重复问题（递归检查+优先队列）
471.
472.         :param best_plots: 当前最佳地块列表（pd.Series对象列表）
473.         :param valid_plots_list: 每个地块对应的各选DataFrame列表
474.         :return: 无重复的地块列表或None（无法解决）
475.         """
476.         # 转换为字典列表方便处理
477.         current_plots = [plot.to_dict() for plot in best_plots]
478.         candidate_pools = [df.to_dict('records') for df in valid_plots_list]
479.
480.         def backtrack(selected, index):
481.             if index == len(current_plots):
482.                 return selected
483.
484.             current_id = current_plots[index]['地块ID']
485.
486.             # 检查当前选择是否与已选地块冲突
487.             for i in range(index):
488.                 if selected[i]['地块ID'] == current_id:
489.                     # 尝试从候选池中选择下一个最佳方案
490.                     if len(candidate_pools[index]) > 1:
491.                         next_option = candidate_pools[index][1] # 取次优选项
492.                         candidate_pools[index] = candidate_pools[index][1:] # 移除已尝试选项
493.                         return backtrack(selected[:index] + [next_option], index)
494.                     else:
495.                         return None # 无备选方案
496.
497.             # 无冲突则继续选择下一个地块
498.             return backtrack(selected + [current_plots[index]], index + 1)
499.
500.         # 首次尝试使用最优解
501.         result = backtrack([], 0)
502.
503.         if result is None:
504.             # 如果首次回溯失败，尝试交换处理顺序（解决AB-BA式死锁）
505.             for i in range(len(current_plots)):
506.                 for j in range(i+1, len(current_plots)):
507.                     if current_plots[i]['地块ID'] == current_plots[j]['地块ID']:
508.                         # 尝试交换优先级
509.                         swapped_plots = current_plots.copy()
510.                         swapped_plots[i], swapped_plots[j] = swapped_plots[j], swapped_plots[i]
511.                         swapped_pools = candidate_pools.copy()
512.                         swapped_pools[i], swapped_pools[j] = swapped_pools[j], swapped_pools[i]
513.
514.                         result = backtrack([], 0)
515.                         if result is not None:
516.                             return [pd.Series(plot) for plot in result]
517.
518.             return None # 所有尝试都失败
519.
520.         return [pd.Series(plot) for plot in result]
521.
522.     def optimize_relocation(self):
523.         """执行搬迁优化"""
524.         self.preprocess_data()
525.
526.         while True:
527.             ranked_yards = self.rank_target_yards()
528.             if not ranked_yards:
529.                 break
530.
531.             for yard_id, total_adj_area in ranked_yards:
532.                 # 获取该院落的所有住户地块
533.                 household_plots = self.df[(self.df['院落ID'] == yard_id) &
534.                                           (self.df['是否有住户'] == 1)].iloc[:self.household_num]

```

```

536.         # 为每个住户地块寻找有效搬迁地块
537.         valid_plots_list = []
538.         for _, plot in household_plots.iterrows():
539.             # 先尝试不允许空院落的查找
540.             valid_plots = self.find_valid_plots(plot, allow_empty_yards=False)
541.             if valid_plots.empty:
542.                 # 如果找不到, 尝试允许空院落的查找
543.                 valid_plots = self.find_valid_plots(plot, allow_empty_yards=False)
544.                 if valid_plots.empty:
545.                     break
546.             valid_plots_list.append(valid_plots)
547.
548.         # 如果未能为所有住户找到有效地块, 跳过该院落
549.         if len(valid_plots_list) != self.household_num:
550.             continue
551.
552.         # 选择每个住户的最佳地块
553.         best_plots = [plots.iloc[0] for plots in valid_plots_list]
554.
555.         # 解决地块重复问题
556.         resolved_plots = self.resolve_duplicate_plots(best_plots, valid_plots_list)
557.         if resolved_plots is None:
558.             continue
559.
560.         # 准备搬迁记录
561.         relocation_info = {'from_yard': yard_id}
562.         cost = 0
563.
564.         # 处理每个搬迁
565.         for i, (from_plot, to_plot) in enumerate(zip(household_plots.itertuples(), resolved_plots)):
566.             relocation_info.update({
567.                 f'from_plot_{i+1}': from_plot.地块ID,
568.                 f'to_yard_{i+1}': to_plot['院落ID'],
569.                 f'to_plot_{i+1}': to_plot['地块ID']
570.             })
571.             cost += self.calculate_relocation_cost(from_plot, to_plot)

```

```

573.         # 记录搬迁信息
574.         self.relocation_log.append(relocation_info)
575.         self.current_cost += cost
576.         self.cost_records.append({
577.             'relocation': len(self.relocation_log),
578.             'cost': cost,
579.             'cumulative_cost': self.current_cost
580.         })
581.
582.         # 执行数据更新
583.         for from_plot, to_plot in zip(household_plots.itertuples(), resolved_plots):
584.             self.update_data(from_plot.地块ID, to_plot['地块ID'])
585.             income_total=calculate_income_fourmonth(self.original_df, self.df, self.adj_matrix)+calculate_income_tenyear(self.df, sel
586.             income_unchange=calculate_income(self.original_df, self.adj_matrix)
587.             m=(income_total-income_unchange)/(self.current_cost+self.cost)
588.             print(m)
589.
590.         # 一次只处理一个院落的搬迁
591.         break
592.     else:
593.         # 没有找到符合条件的搬迁
594.         break
595.
596.     return self.relocation_log, self.cost_records, self.df, self.current_cost+self.cost
597.
598.
599. #%%
600.
601. import pandas as pd
602.
603. import pandas as pd
604.

```

```

605. def calculate_complete_yards_area(df):
606.     # 检查必要列是否存在
607.     required_cols = ['院落ID', '院落面积', '是否有住户']
608.     if not all(col in df.columns for col in required_cols):
609.         missing = [col for col in required_cols if col not in df.columns]
610.         raise ValueError(f"数据缺少必要列: {missing}")
611.
612.     # 按院落ID分组, 检查是否所有地块都无住户
613.     yard_status = df.groupby('院落ID').agg(
614.         has_resident=pd.NamedAgg(column='是否有住户', aggfunc='max'), # 检查是否有住户
615.         yard_area=pd.NamedAgg(column='院落面积', aggfunc='first') # 使用院落面积
616.     )
617.
618.     # 筛选完整院落(无住户的院落)
619.     complete_yards = yard_status[yard_status['has_resident'] == 0]
620.
621.     # 返回完整院落的总面积
622.     return complete_yards['yard_area'].sum()
623.
624. #%%
625. # 示例: 打印搬迁日志
626. def print_relocation_log(relocation_log, household_num):
627.     for log in relocation_log:
628.         output_parts = []
629.         for i in range(1, household_num + 1):
630.             from_plot_key = f'from_plot_{i}'
631.             to_yard_key = f'to_yard_{i}'
632.             to_plot_key = f'to_plot_{i}'
633.
634.             if from_plot_key in log and to_yard_key in log and to_plot_key in log:
635.                 part = f"从院落[{log['from_yard']}]地块[{log[from_plot_key]}]搬迁到院落[{log[to_yard_key]}]地块[{log[to_plot_key]}"
636.                 output_parts.append(part)
637.
638.     print("".join(output_parts))

```

```

642. excel_path = "毗邻矩阵.xlsx"
643. df_adj = pd.read_excel(excel_path)
644. adjacency_matrix = generate_adjacency_matrix(df_adj)
645. df = pd.read_excel("附件一：老城街区地块信息.xlsx")
646. original_df = df.copy()
647. # 定义要处理的住户数列表（可以扩展到更多）
648. household_nums = [1, 2, 3, 4, 5, 6, 7, 8] # 可以继续添加5, 6, ...
649.
650. # 初始化结果存储
651. results = {
652.     'complete_yards': [],
653.     'adjacent_areas': [],
654.     'cost_records': []
655. }
656.
657. current_df = df.copy()
658. cost=0
659. log_total=[]
660. for num in household_nums:
661.     print(f"\n正在处理 {num} 住户院落...")
662.
663.     # 创建优化器并执行搬迁
664.     optimizer = YardRelocationOptimizer(current_df, original_df, adjacency_matrix, household_num=num, cost=cost)
665.     log, cost_records, current_df, cost= optimizer.optimize_relocation()
666.     print_relocation_log(log, household_num=optimizer.household_num)
667.     log_total.append(log)
668.     # 计算并存储结果
669.     complete = calculate_complete_yards(current_df)
670.     adjacent = calculate_adjacent_complete_yards_area(current_df, adjacency_matrix)
671.
672.     results['complete_yards'].append(complete)
673.     results['adjacent_areas'].append(adjacent)
674.     results['cost_records'].append(cost_records)
675.
676.     # 打印当前结果
677.     print(f"处理 {num} 住户院落后的结果:")
678.     print(f"完整院落数量: {complete}")
679.     print(f"毗邻完整院落总面积: {adjacent}")
680.
681.     if cost_records:
682.         print(f"累计搬迁成本: {cost_records[-1]['cumulative_cost']:.2f} 万元")
683.
684. # 可选：将最终结果保存到DataFrame
685. result_df = pd.DataFrame({
686.     '住户数': household_nums,
687.     '完整院落数': results['complete_yards'],
688.     '毗邻面积': results['adjacent_areas']
689. })
690. print("\n最终汇总结果:")
691. print(result_df)
692.
693. # %%
694. print(calculate_complete_yards_area(current_df))
695. print(calculate_adjacent_complete_yards_area(current_df, adjacency_matrix))
696. print(calculate_income(df, adjacency_matrix))
697. # %%
698. excel_path = "毗邻矩阵.xlsx"
699. df_adj = pd.read_excel(excel_path)
700. adjacency_matrix = generate_adjacency_matrix(df_adj)
701. # 执行计算
702. # 四个月收入（过渡期）
703. income_4months = calculate_income_fourmonth(df, current_df, adjacency_matrix=adjacency_matrix)
704.
705. # 长期收入（10年-4个月）
706. income_longterm = calculate_income_tenyear(current_df, adjacency_matrix=adjacency_matrix)
707. print(income_4months+income_longterm)
708. # %%
709. def calculate_complete_yards_diff(df, df_new):
710.     """
711.     计算两个DataFrame中的完整院落差异（找出在df_new中新增的完整院落）

```

```

712.     参数:
713.     df - 原始DataFrame
714.     df_new - 新DataFrame
715.
716.     返回:
717.     tuple: (df的完整院落数, df_new的完整院落数, df_new新增的完整院落列表)
718.     """
719.     # 计算df中的完整院落
720.     yard_groups_df = df.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
721.     complete_yards_df = set(yard_groups_df[yard_groups_df].index.tolist())
722.
723.     # 计算df_new中的完整院落
724.     yard_groups_df_new = df_new.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
725.     complete_yards_df_new = set(yard_groups_df_new[yard_groups_df_new].index.tolist())
726.
727.     # 找出在df_new中新增的完整院落 (即在df_new中存在但在df中不存在的完整院落)
728.     new_complete_yards = list(complete_yards_df_new - complete_yards_df)
729.
730.     return (
731.         len(complete_yards_df),          # df中的完整院落数
732.         len(complete_yards_df_new),      # df_new中的完整院落数
733.         new_complete_yards               # df_new新增的完整院落列表
734.     )
735.
736. print(calculate_complete_yards_diff(df, current_df))
737. # %%
738. def calculate_relocation_cost_new(from_plot, to_plot):
739.     """计算搬迁成本
740.     Args:
741.         from_plot: 原居住地块信息
742.         to_plot: 新搬迁地块信息
743.     Returns:
744.         总搬迁成本 (万元)
745.     """
746.     # 1. 沟通成本 (固定)
747.     communication_cost = 3 # 万元
748.
749.     # 2. 修缮成本 (这里设为0, 可根据实际情况调整)
750.     renovation_cost = 0 # 万元
751.
752.     # 3. 面积损失成本 (按10年计算)
753.     days_per_year = 365
754.     years = 10 # 按10年期计算
755.
756.     # 计算面积差 (新地块面积必须>原地块面积)
757.     area_diff = max(0, to_plot['地块面积'].iloc[0] - from_plot.地块面积.iloc[0])
758.
759.     # 根据朝向确定租金单价
760.     if to_plot.地块方位.iloc[0] in ['东', '西']:
761.         rent_rate = 8 # 东西厢8元/平米/天
762.     else:
763.         rent_rate = 15 # 南北厢15元/平米/天
764.     if from_plot.地块方位.iloc[0] in ['东', '西']:
765.         rent_rate_new = 8 # 东西厢8元/平米/天
766.     else:
767.         rent_rate_new = 15 # 南北厢15元/平米/天
768.     # 面积损失成本 = 面积差 × 租金 × 365天 × 10年
769.     area_loss_cost = area_diff * rent_rate * days_per_year * years / 10000 # 转换为万元
770.
771.     # 4. 时间损失成本 (4个月无法出租)
772.     months = 4
773.     days = months * 30 # 按每月30天计算
774.     time_loss_cost = from_plot.地块面积.iloc[0] * rent_rate_new * days / 10000 # 万元
775.
776.     # 总成本
777.     total_cost = communication_cost + renovation_cost + area_loss_cost + time_loss_cost
778.
779.     return total_cost

```

```

782. def execute_relocations(df, relocations, adj_matrix):
783.     """
784.     执行院落搬迁操作 (兼容嵌套列表格式)
785.
786.     参数:
787.         df: 原始地块DataFrame
788.         relocations: 搬迁记录列表, 格式为[[{}]]或[{}]
789.
790.     返回:
791.         (success_count, error_log, updated_df)
792.     """
793.
794.     error_log = []
795.     success_count = 0
796.
797.     # 展平嵌套列表结构 (处理[[{}]]情况)
798.     flat_relocations = []
799.     for item in relocations:
800.         if isinstance(item, list):
801.             flat_relocations.extend(item)
802.         else:
803.             flat_relocations.append(item)
804.     m_total=[]
805.     for relocation in flat_relocations:
806.         df_working = df.copy()
807.         try:
808.             # 1. 解析搬迁记录 (兼容图片中的混乱键名)
809.             households = []
810.             i = 1
811.             while True:

```

```

812.         # 动态键名检测 (兼容from_plot_1/fromplot1/to_kdot_1等格式)
813.         from_keys = [f'from_plot_{i}', f'fromplot{i}', f'from plot {i}']
814.         from_val = next((relocation[k] for k in from_keys if k in relocation), None)
815.
816.         yard_keys = [f'to_yard_{i}', f'to_yard{i}', f'to-yard_{i}', f'to_kdot_{i}']
817.         yard_val = next((relocation[k] for k in yard_keys if k in relocation), None)
818.
819.         plot_keys = [f'to_plot_{i}', f'to_plot{i}', f'to-plot_{i}', f'to_kplot_{i}']
820.         plot_val = next((relocation[k] for k in plot_keys if k in relocation), None)
821.
822.         if not all([from_val, yard_val, plot_val]):
823.             break
824.
825.         households.append({
826.             'from_yard': relocation.get('from_yard'),
827.             'from_plot': from_val,
828.             'to_yard': yard_val,
829.             'to_plot': plot_val
830.         })
831.         i += 1
832.
833.     if not households:
834.         error_log.append((relocation, "无有效搬迁数据"))
835.         continue
836.
837.     # 2. 预校验所有地块状态
838.     valid = True
839.     for h in households:
840.         # 校验原住户地块
841.         from_mask = (df_working['院落ID'] == h['from_yard']) & \
842.             (df_working['地块ID'] == h['from_plot'])
843.         if not df_working.loc[from_mask, '是否有住户'].eq(1).any():
844.             valid = False
845.             error_log.append((
846.                 relocation,
847.                 f"原地块{h['from_yard']}-{h['from_plot']}无住户或不存在"
848.             ))
849.             break
850.
851.         # 校验目标地块
852.         to_mask = (df_working['院落ID'] == h['to_yard']) & \
853.             (df_working['地块ID'] == h['to_plot'])
854.         if not df_working.loc[to_mask, '是否有住户'].eq(0).any():
855.             valid = False
856.             error_log.append((
857.                 relocation,
858.                 f"目标地块{h['to_yard']}-{h['to_plot']}已有住户或不存在"
859.             ))
860.             break
861.
862.     if not valid:
863.         continue
864.     cost_total=0
865.     # 3. 执行搬迁
866.     for h in households:
867.         # 搬出
868.         from_mask = (df_working['院落ID'] == h['from_yard']) & \
869.             (df_working['地块ID'] == h['from_plot'])
870.         df_working.loc[from_mask, '是否有住户'] = 0
871.
872.         # 搬入
873.         to_mask = (df_working['院落ID'] == h['to_yard']) & \
874.             (df_working['地块ID'] == h['to_plot'])
875.         df_working.loc[to_mask, '是否有住户'] = 1
876.         cost=calculate_relocation_cost_new(
877.             df[df['地块ID'] == h['from_plot']], df[df['地块ID'] == h['to_plot']]
878.         )
879.         cost_total+=cost
880.     income_total=calculate_income_fourmonth(df, df_working, adj_matrix)+calculate_income_tenyyear(df_working, adj_matrix)
881.     income_unchange=calculate_income(df, adj_matrix)
882.     m=(income_total-income_unchange)/(cost_total)
883.     m_total.append(m)
884.     success_count += 1

```



```

886.         except Exception as e:
887.             error_log.append((relocation, f"执行错误: {str(e)}"))
888.
889.         return success_count, error_log, df_working, m_total
890.
891. success_count, errors, updated_df, m_toatl = execute_relocations(df, log_total, adjacency_matrix)
892.
893. """
894. def staged_relocation(df, relocations, adj_matrix):
895.     """
896.     分阶段最优搬迁算法
897.
898.     参数:
899.         df: 原始地块DataFrame
900.         relocations: 所有可能的搬迁方案列表
901.         adj_matrix: 邻接矩阵
902.
903.     返回:
904.         (success_count, error_log, final_df, m_history)
905.     """
906.     # 初始化
907.     df_current = df.copy()
908.
909.     executed_relocations = []
910.     error_log = []
911.     m_history = []
912.     success_count = 0
913.
914.     # 计算初始收益 (无搬迁情况)
915.     income_unchange = calculate_income(df, adj_matrix)
916.     flat_relocations = []
917.     for item in relocations:
918.         if isinstance(item, list):
919.             flat_relocations.extend(item)
920.         else:
921.             flat_relocations.append(item)
922.     remaining_relocations = flat_relocations.copy()
923.     final_best_cost = 0

```

```

925. while remaining_relocations:
926.     best_m = -float('inf')
927.     best_reloc = None
928.     best_df = None
929.     best_cost = 0
930.
931.     # 阶段1: 评估所有剩余搬迁方案的m值
932.     for reloc in remaining_relocations:
933.         try:
934.             # 临时执行当前搬迁方案
935.             df_temp = df_current.copy()
936.             cost_total = 0
937.
938.             # 解析搬迁记录
939.             households = []
940.             i = 1
941.             while True:
942.                 from_key = f'from_plot_{i}'
943.                 if from_key not in reloc:
944.                     break
945.
946.                 households.append({
947.                     'from_yard': reloc[from_yard],
948.                     'from_plot': reloc[from_key],
949.                     'to_yard': reloc[f'to_yard_{i}'],
950.                     'to_plot': reloc[f'to_plot_{i}']
951.                 })
952.                 i += 1
953.
954.             # 执行临时搬迁
955.             for h in households:
956.                 from_mask = (df_temp['院落ID'] == h['from_yard']) & \
957.                     (df_temp['地块ID'] == h['from_plot'])
958.                 to_mask = (df_temp['院落ID'] == h['to_yard']) & \
959.                     (df_temp['地块ID'] == h['to_plot'])
960.
961.                 df_temp.loc[from_mask, '是否有住户'] = 0
962.                 df_temp.loc[to_mask, '是否有住户'] = 1

```

```

964.         # 计算成本 (包含历史成本)
965.         cost = calculate_relocation_cost_new(
966.             df_current[df_current['地块ID'] == h['from_plot']],
967.             df_current[df_current['地块ID'] == h['to_plot']]
968.         )
969.         cost_total += cost
970.         cost_total_stage=cost_total+final_best_cost
971.         # 计算收益变化
972.         income_new = calculate_income_fourmonth(df, df_temp, adj_matrix) + \
973.             calculate_income_tenyear(df_temp, adj_matrix)
974.         delta_income = income_new - income_unchange
975.
976.         # 计算m值 (总收益变化/总成本)
977.         if cost_total > 0:
978.             current_m = delta_income / cost_total
979.         else:
980.             current_m = -float('inf')
981.
982.         # 记录最优方案
983.         if current_m > best_m:
984.             best_m = current_m
985.             best_reloc = reloc
986.             best_df = df_temp.copy()
987.             best_cost = cost_total_stage
988.             income_unchange = income_new
989.
990.     except Exception as e:
991.         error_log.append((reloc, f"评估错误: {str(e)}"))
992.         continue
993.     final_best_cost=best_cost
994.     # 阶段2: 执行最优搬迁方案
995.     if best_reloc and best_m > -float('inf'):
996.         # 正式执行最优搬迁
997.         try:
998.             # 解析最优搬迁方案
999.             households = []
1000.             i = 1

```

```

1001.         while True:
1002.             from_key = f'from_plot_{i}'
1003.             if from_key not in best_reloc:
1004.                 break
1005.
1006.             households.append({
1007.                 'from_yard': best_reloc['from_yard'],
1008.                 'from_plot': best_reloc[from_key],
1009.                 'to_yard': best_reloc[f'to_yard_{i}'],
1010.                 'to_plot': best_reloc[f'to_plot_{i}']
1011.             })
1012.             i += 1
1013.
1014.         # 执行搬迁
1015.         for h in households:
1016.             from_mask = (df_current['院落ID'] == h['from_yard']) & \
1017.                 (df_current['地块ID'] == h['from_plot'])
1018.             to_mask = (df_current['院落ID'] == h['to_yard']) & \
1019.                 (df_current['地块ID'] == h['to_plot'])
1020.
1021.             df_current.loc[from_mask, '是否有住户'] = 0
1022.             df_current.loc[to_mask, '是否有住户'] = 1
1023.
1024.         # 更新状态
1025.         executed_relocations.append(best_reloc)
1026.         remaining_relocations.remove(best_reloc)
1027.         m_history.append(best_m)
1028.         success_count += 1
1029.
1030.     except Exception as e:
1031.         error_log.append((best_reloc, f"执行错误: {str(e)}"))
1032.     else:
1033.         break # 没有可行的搬迁方案
1034.
1035.     return success_count, error_log, df_current, m_history, final_best_cost, executed_relocations
1036.

```

```

1038. success_count, errors, final_df, m_history, cost, execute_relocations_final = staged_relocation(
1039.     df, log_total, adjacency_matrix
1040. )
1041. # %%
1042. import matplotlib.pyplot as plt
1043. plt.plot(m_history)
1044. import pickle
1045. with open('data.pkl', 'wb') as f:
1046.     pickle.dump(log_total, f)
1047.
1048. # %%
1049. def find_special_value(m):
1050.     for i in range(1, len(m)):
1051.         if m[i-1] >= m[i]:
1052.             continue # 不满足条件1
1053.
1054.         # 检查前面是否有比 m[i] 大的和小的
1055.         has_greater = any(m[j] > m[i] for j in range(i))
1056.         has_less = any(m[j] < m[i] for j in range(i))
1057.         if not (has_greater and has_less):
1058.             continue # 不满足条件2
1059.
1060.         # 检查后面是否全部 < m[i]
1061.         if all(m[j] < m[i] for j in range(i+1, len(m))):
1062.             return m[i+1] # 满足所有条件
1063.
1064.     return None # 没有找到
1065. m_ip = find_special_value(m_history)
1066. # %%
1067. execute_relocations_ip = execute_relocations_final[:len(m_ip)]
1068. # %%
1069. def calculate_relocation_result(df, relocations, adj_matrix):
1070.     """
1071.     计算完成所有搬迁后的新DataFrame和总成本
1072.
1073.     参数:
1074.     df: 原始地块DataFrame
1075.     relocations: 所有搬迁方案列表
1076.     adj_matrix: 邻接矩阵

```

```

1078.     返回:
1079.     (df_new, total_cost)
1080.     """
1081.     df_new = df.copy()
1082.     total_cost = 0
1083.
1084.     # 展开搬迁方案 (处理嵌套列表情况)
1085.     flat_relocations = []
1086.     for item in relocations:
1087.         if isinstance(item, list):
1088.             flat_relocations.extend(item)
1089.         else:
1090.             flat_relocations.append(item)
1091.
1092.     # 执行所有搬迁
1093.     for reloc in flat_relocations:
1094.         try:
1095.             # 解析搬迁记录
1096.             households = []
1097.             i = 1
1098.             while True:
1099.                 from_key = f'from_plot_{i}'
1100.                 if from_key not in reloc:
1101.                     break
1102.
1103.                 households.append({
1104.                     'from_yard': reloc['from_yard'],
1105.                     'from_plot': reloc[from_key],
1106.                     'to_yard': reloc[f'to_yard_{i}'],
1107.                     'to_plot': reloc[f'to_plot_{i}']
1108.                 })
1109.                 i += 1
1110.
1111.             # 执行搬迁并计算成本
1112.             for h in households:
1113.                 from_mask = (df_new['院落ID'] == h['from_yard']) & \
1114.                     (df_new['地块ID'] == h['from_plot'])
1115.                 to_mask = (df_new['院落ID'] == h['to_yard']) & \
1116.                     (df_new['地块ID'] == h['to_plot'])

```

```

1103.         households.append({
1104.             'from_yard': reloc['from_yard'],
1105.             'from_plot': reloc['from_key'],
1106.             'to_yard': reloc[f'to_yard_{i}'],
1107.             'to_plot': reloc[f'to_plot_{i}']
1108.         })
1109.         i += 1
1110.
1111.     # 执行搬迁并计算成本
1112.     for h in households:
1113.         from_mask = (df_new['院落ID'] == h['from_yard']) & \
1114.             (df_new['地块ID'] == h['from_plot'])
1115.         to_mask = (df_new['院落ID'] == h['to_yard']) & \
1116.             (df_new['地块ID'] == h['to_plot'])
1117.
1118.         # 更新住户状态
1119.         df_new.loc[from_mask, '是否有住户'] = 0
1120.         df_new.loc[to_mask, '是否有住户'] = 1
1121.
1122.         # 计算本次搬迁成本并累加
1123.         cost = calculate_relocation_cost_new(
1124.             df[df['地块ID'] == h['from_plot']],
1125.             df[df['地块ID'] == h['to_plot']]
1126.         )
1127.         total_cost += cost
1128.
1129.     except Exception as e:
1130.         print(f"搬迁方案执行失败: {reloc}, 错误: {str(e)}")
1131.         continue
1132.
1133.     return df_new, total_cost
1134.
1135.     """
1136.     df_ip, cost_ip = calculate_relocation_result(df, execute_relocations_ip, adjacency_matrix)
1137.     # %%
1138.     income_ip = calculate_income_fourmonth(df, df_ip, adjacency_matrix) + \
1139.         calculate_income_tenyear(df_ip, adjacency_matrix)
1140.     complete_area_ip = calculate_complete_yards_area(df_ip, adjacency_matrix)

```

问题四

```

01.     """
02.     import pandas as pd
03.     import numpy as np
04.     import matplotlib.pyplot as plt
05.     from typing import Tuple, Dict, List, Optional
06.
07.     class RelocationOptimizationSystem:
08.         """
09.         老城街区搬迁优化系统
10.         输入: 毗邻矩阵.xlsx, 附件一: 老城街区地块信息.xlsx
11.         输出: 搬迁方案、成本、收益等关键指标, 包括拐点分析
12.         """
13.
14.         def __init__(self, housing_num=[1,2,3,4,5,6,7,8], adjacency_price=36, complete_price=30, communicate=3, renovation=0, nn=8, ew=15, ratio
15.             self.adjacency_matrix = None
16.             self.original_df = None
17.             self.current_df = None
18.             self.df_ip = None
19.             self.execute_relocations_ip = None
20.             self.household_nums = housing_num # 可处理的住户数范围
21.             self.adjacency_price = adjacency_price
22.             self.complete_price = complete_price # 完整院落的单价
23.             self.relocation_log = []
24.             self.total_cost = 0
25.             self.results = {
26.                 'complete_yards': [],
27.                 'adjacent_areas': [],
28.                 'cost_records': []
29.             }
30.             self.m_history = []
31.             self.execute_relocations_final = []
32.             self.communicate = communicate
33.             self.renovation = renovation
34.             self.nn = nn
35.             self.ew = ew
36.             self.ratio = ratio

```

```

38.     def load_data(self, adjacency_path: str, plot_info_path: str) -> None:
39.         """加载输入数据"""
40.         df_adj = pd.read_excel(adjacency_path)
41.         self.adjacency_matrix = self._generate_adjacency_matrix(df_adj)
42.         self.original_df = pd.read_excel(plot_info_path)
43.         self.current_df = self.original_df.copy()
44.
45.     def _generate_adjacency_matrix(self, df: pd.DataFrame) -> np.ndarray:
46.         """生成邻接矩阵"""
47.         num_yards = df['院落id'].nunique()
48.         adj_matrix = np.zeros((num_yards, num_yards), dtype=int)
49.
50.         for _, row in df.iterrows():
51.             yard_id = int(row['院落id']) - 1
52.             neighbors = [x.strip() for x in str(row['院落毗邻id']).split(',')]
53.
54.             for neighbor in neighbors:
55.                 try:
56.                     neighbor_id = int(neighbor.strip()) - 1
57.                     if 0 <= neighbor_id < num_yards:
58.                         adj_matrix[yard_id][neighbor_id] = 1
59.                         adj_matrix[neighbor_id][yard_id] = 1
60.                 except ValueError:
61.                     continue
62.         return adj_matrix
63.
64.     def run_optimization(self) -> None:
65.         """执行完整的搬迁优化流程"""
66.         household_nums = self.household_nums
67.
68.         for num in household_nums:
69.             print(f"\n正在处理 {num} 住户院落...")

```

```

71.         optimizer = YardRelocationOptimizer(
72.             self.current_df,
73.             self.original_df,
74.             self.adjacency_matrix,
75.             household_num=num,
76.             cost=self.total_cost,
77.             communicate=self.communicate,
78.             renovation=self.renovation,
79.             nn=self.nn,
80.             ew=self.ew,
81.             ratio=self.ratio
82.         )
83.
84.         log, cost_records, self.current_df, cost = optimizer.optimize_relocation()
85.         self._print_relocation_log(log, num)
86.
87.         self.relocation_log.extend(log)
88.         self.total_cost = cost
89.
90.         # 计算并存储结果
91.         complete = self._calculate_complete_yards(self.current_df)
92.         adjacent = self._calculate_adjacent_complete_yards_area(self.current_df)
93.
94.         self.results['complete_yards'].append(complete)
95.         self.results['adjacent_areas'].append(adjacent)
96.         self.results['cost_records'].append(cost_records)
97.
98.     def analyze_inflection_point(self) -> Dict:
99.         """
100.         执行拐点分析并返回关键指标
101.         返回:
102.         {
103.             'inflection_point': 拐点位置,
104.             'relocations': 拐点前的搬迁方案,
105.             'cost': 拐点处总成本,
106.             'income': 拐点处总收入,
107.             'profit': 拐点处总利润,

```

```

107.         'profit': 拐点处总利润,
108.         'complete_yards': 拐点处完整院落,
109.         'complete_area': 拐点处完整院落面积,
110.         'adjacent_area': 拐点处毗邻面积,
111.         'm_history': m值变化曲线数据
112.     }
113.     """
114.     # 执行分阶段搬迁分析
115.     _, _, self.m_history, _, self.execute_relocations_final = self._staged_relocation(
116.         self.original_df
117.     )
118.
119.     # 寻找拐点
120.     m_ip = self._find_special_value(self.m_history)
121.     if not m_ip:
122.         raise ValueError("未找到有效的拐点")
123.
124.     execute_relocations_ip = self.execute_relocations_final[:len(m_ip)]
125.     self.execute_relocations_ip = execute_relocations_ip.copy()
126.     # 计算拐点处结果
127.     df_ip, cost_ip = self._calculate_relocation_result(
128.         self.original_df
129.     )
130.     self.df_ip = df_ip.copy()
131.     # 计算各项指标
132.     income_ip = (self.calculate_income_fourmonth(self.original_df, df_ip, self.adjacency_matrix) +
133.                  self.calculate_income_tenyear(df_ip, self.adjacency_matrix))
134.     profit_ip = income_ip - cost_ip
135.     complete_area_ip = self._calculate_complete_yards_area(df_ip)
136.
137.     complete_diff = self._calculate_complete_yards_diff(df_ip)
138.     adjacent_area_ip = self._calculate_adjacent_complete_yards_area(df_ip)
139.
140.     return {
141.         'inflection_point': len(m_ip),
142.         'relocations': execute_relocations_ip,
143.         'cost': cost_ip,
144.         'income': income_ip,
145.         'profit': profit_ip,
146.         'complete_yards': complete_diff[2],
147.         'complete_area': complete_area_ip,
148.         'adjacent_area': adjacent_area_ip,
149.         'm_history': self.m_history
150.     }
151.
152. def generate_full_report(self) -> Dict:
153.     """生成包含常规优化和拐点分析的完整报告"""
154.     # 常规优化报告
155.     regular_report = self._generate_regular_report()
156.
157.     # 拐点分析报告
158.     inflection_report = self.analyze_inflection_point()
159.
160.     # 合并报告
161.     full_report = {
162.         'regular_optimization': regular_report,
163.         'inflection_point_analysis': inflection_report
164.     }
165.
166.     return full_report
167.
168. def _calculate_complete_yards_diff(self, df: pd.DataFrame) -> Tuple[int, int, List]:
169.     """计算完整院落差异"""
170.     orig_complete = set(self.get_complete_yards(self.original_df))
171.     current_complete = set(self.get_complete_yards(df))
172.
173.     return (
174.         len(orig_complete),
175.         len(current_complete),
176.         list(current_complete - orig_complete)
177.     )
178.
179. def _calculate_complete_yards_area(self, df: pd.DataFrame) -> List:
180.     # 检查必要列是否存在

```

```

179. def _calculate_complete_yards_area(self, df: pd.DataFrame) -> List:
180.     # 检查必要列是否存在
181.     required_cols = ['院落ID', '院落面积', '是否有住户']
182.     if not all(col in df.columns for col in required_cols):
183.         missing = [col for col in required_cols if col not in df.columns]
184.         raise ValueError(f"数据缺少必要列: {missing}")
185.
186.     # 按院落ID分组, 检查是否所有地块都无住户
187.     yard_status = df.groupby('院落ID').agg(
188.         has_resident=pd.NamedAgg(column='是否有住户', aggfunc='max'), # 检查是否有住户
189.         yard_area=pd.NamedAgg(column='院落面积', aggfunc='first') # 使用院落面积
190.     )
191.
192.     # 筛选完整院落(无住户的院落)
193.     complete_yards = yard_status[yard_status['has_resident'] == 0]
194.
195.     # 返回完整院落的总面积
196.     return complete_yards['yard_area'].sum()
197.
198.
199. def _calculate_complete_yards(self, df: pd.DataFrame) -> int:
200.     """计算完整院落数量"""
201.     yard_groups = df.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
202.     return int(yard_groups.sum())
203.
204. def get_complete_yards(self, df: pd.DataFrame) -> List:
205.     """获取完整院落列表"""
206.     yard_groups = df.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
207.     return yard_groups[yard_groups].index.tolist()
208.
209. def _calculate_adjacent_complete_yards_area(self, df: pd.DataFrame) -> float:
210.     """计算毗邻完整院落面积"""
211.     complete_yards = self.get_complete_yards(df)
212.     yard_areas = {y: df[df['院落ID'] == y]['院落面积'].iloc[0]
213.                   for y in complete_yards}
214.
215.     n = len(complete_yards)
216.     if_adjacent = np.zeros(n, dtype=bool)
217.     yard_to_index = {y: idx for idx, y in enumerate(df['院落ID'].unique())}
218.
219.     for i, y1 in enumerate(complete_yards):
220.         for j, y2 in enumerate(complete_yards):
221.             if self.adjacency_matrix[yard_to_index[y1]][yard_to_index[y2]] == 1:
222.                 if_adjacent[i] = True
223.
224.     return np.sum(np.array(list(yard_areas.values())) * if_adjacent)
225.
226. def _calculate_adjacent_complete_yards_area(self, df: pd.DataFrame) -> float:
227.     """计算毗邻完整院落面积"""
228.     complete_yards = self.get_complete_yards(df)
229.     yard_areas = {y: df[df['院落ID'] == y]['院落面积'].iloc[0]
230.                   for y in complete_yards}
231.
232.     n = len(complete_yards)
233.     if_adjacent = np.zeros(n, dtype=bool)
234.     yard_to_index = {y: idx for idx, y in enumerate(df['院落ID'].unique())}
235.
236.     for i, y1 in enumerate(complete_yards):
237.         for j, y2 in enumerate(complete_yards):
238.             if self.adjacency_matrix[yard_to_index[y1]][yard_to_index[y2]] == 1:
239.                 if_adjacent[i] = True
240.
241.     return np.sum(np.array(list(yard_areas.values())) * if_adjacent)
242.
243. def _calculate_complete_yards(self, df: pd.DataFrame) -> int:
244.     """计算完整院落数量"""
245.     yard_groups = df.groupby('院落ID')['是否有住户'].apply(lambda x: all(x == 0))
246.     return int(yard_groups.sum())
247.
248. def generate_regular_report(self) -> Dict:
249.     """生成常规优化报告"""
250.     complete_diff = self._calculate_complete_yards_diff(self.current_df)
251.     total_income = self._calculate_total_income()
252.     profit = total_income - self.total_cost
253.
254.     return {
255.         'relocation_plans': self.relocation_log,
256.         'total_cost': self.total_cost,
257.         'new_complete_yards': complete_diff[2],
258.         'final_complete_yards_area': self._calculate_complete_yards_area(self.current_df),
259.         'final_adjacent_area': self._calculate_adjacent_complete_yards_area(self.current_df),
260.         'total_income': total_income,

```

```

260.         'total_income': total_income,
261.         'total_profit': profit,
262.         'optimization_steps': pd.DataFrame({
263.             '住户数': self.household_nums,
264.             '完整院落数': self.results['complete_yards'],
265.             '毗邻面积': self.results['adjacent_areas']
266.         })
267.     }
268.
269.     def _calculate_total_income(self) -> float:
270.         """计算总收入"""
271.
272.         income_4m = self.calculate_income_fourmonth(self.original_df, self.current_df, self.adjacency_matrix)
273.         income_10y = self.calculate_income_tenyear(self.current_df, self.adjacency_matrix)
274.         return income_4m + income_10y
275.
276.     def calculate_complete_yards_area(self, df: pd.DataFrame) -> List:
277.         # 检查必要列是否存在
278.         required_cols = ['院落ID', '院落面积', '是否有住户']
279.         if not all(col in df.columns for col in required_cols):
280.             missing = [col for col in required_cols if col not in df.columns]
281.             raise ValueError(f"数据缺少必要列: {missing}")
282.
283.         # 按院落ID分组, 检查是否所有地块都无住户
284.         yard_status = df.groupby('院落ID').agg(
285.             has_resident=pd.NamedAgg(column='是否有住户', aggfunc='max'), # 检查是否有住户
286.             yard_area=pd.NamedAgg(column='院落面积', aggfunc='first') # 使用院落面积
287.         )
288.
289.         # 筛选完整院落(无住户的院落)
290.         complete_yards = yard_status[yard_status['has_resident'] == 0]
291.
292.         # 返回完整院落的总面积
293.         return complete_yards['yard_area'].sum()

```

```

295.     def _print_relocation_log(self, log: List, household_num: int) -> None:
296.         """打印搬迁日志"""
297.         for record in log:
298.             parts = []
299.             for i in range(1, household_num + 1):
300.                 parts.append(
301.                     f"从院落{record['from_yard']}地块{record[f'from_plot_{i}']} "
302.                     f"搬迁到院落{record[f'to_yard_{i}']}地块{record[f'to_plot_{i}']}"
303.                 )
304.             print(":".join(parts))
305.         # ----- 拐点分析相关方法 -----
306.
307.     def _staged_relocation(self, df: pd.DataFrame) -> List:
308.         """
309.         分阶段最优搬迁算法
310.
311.         参数:
312.             df: 原始地块DataFrame
313.             relocations: 所有可能的搬迁方案列表
314.             adj_matrix: 邻接矩阵
315.
316.         返回:
317.             (success_count, error_log, final_df, m_history)
318.         """
319.         # 初始化
320.         df_current = df.copy()
321.
322.         executed_relocations = []
323.         error_log = []
324.         m_history = []
325.         success_count = 0

```



```

327.     # 计算初始收益 (无搬迁情况)
328.     income_unchange = self.calculate_income(df, self.adjacency_matrix)
329.     flat_relocations = []
330.     for item in self.relocation_log:
331.         if isinstance(item, list):
332.             flat_relocations.extend(item)
333.         else:
334.             flat_relocations.append(item)
335.     remaining_relocations=flat_relocations.copy()
336.     final_best_cost=0
337.
338.     while remaining_relocations:
339.         best_m = -float('inf')
340.         best_reloc = None
341.         best_df = None
342.         best_cost = 0
343.
344.         # 阶段1: 评估所有剩余搬迁方案的m值
345.         for reloc in remaining_relocations:
346.             try:
347.                 # 临时执行当前搬迁方案
348.                 df_temp = df_current.copy()
349.                 cost_total = 0
350.
351.                 # 解析搬迁记录
352.                 households = []
353.                 i = 1
354.                 while True:
355.                     from_key = f'from_plot_{i}'
356.                     if from_key not in reloc:
357.                         break
358.
359.                     households.append({
360.                         'from_yard': reloc['from_yard'],
361.                         'from_plot': reloc[from_key],
362.                         'to_yard': reloc[f'to_yard_{i}'],
363.                         'to_plot': reloc[f'to_plot_{i}']
364.                     })
365.                     i += 1
366.
367.                 # 执行临时搬迁
368.                 for h in households:
369.                     from_mask = (df_temp['院落ID'] == h['from_yard']) & \
370.                         (df_temp['地块ID'] == h['from_plot'])
371.                     to_mask = (df_temp['院落ID'] == h['to_yard']) & \
372.                         (df_temp['地块ID'] == h['to_plot'])
373.
374.                     df_temp.loc[from_mask, '是否有住户'] = 0
375.                     df_temp.loc[to_mask, '是否有住户'] = 1
376.
377.                 # 计算成本 (包含历史成本)
378.                 cost = self.calculate_relocation_cost_new(
379.                     df_current[df_current['地块ID'] == h['from_plot']],
380.                     df_current[df_current['地块ID'] == h['to_plot']]
381.                 )
382.                 cost_total += cost
383.                 cost_total_stage=cost_total+final_best_cost
384.                 # 计算收益变化
385.                 income_new = self.calculate_income_fourmonth(df, df_temp, self.adjacency_matrix) + \
386.                     self.calculate_income_tenyear(df_temp, self.adjacency_matrix)
387.                 delta_income = income_new - income_unchange
388.
389.                 # 计算m值 (总收益变化/总成本)
390.                 if cost_total > 0:
391.                     current_m = delta_income / cost_total_stage
392.                 else:
393.                     current_m = -float('inf')
394.
395.                 # 记录最优方案
396.                 if current_m > best_m:
397.                     best_m = current_m
398.                     best_reloc = reloc
399.                     best_df = df_temp.copy()
400.                     best_cost = cost_total_stage
401.
402.             except Exception as e:
403.                 error_log.append((reloc, f"评估错误: {str(e)}"))
404.                 continue
405.     final_best_cost=best_cost
406.     # 阶段2: 执行最优搬迁方案

```

```

406. # 阶段2: 执行最优搬迁方案
407. if best_reloc and best_m > -float('inf'):
408.     # 正式执行最优搬迁
409.     try:
410.         # 解析最优搬迁方案
411.         households = []
412.         i = 1
413.         while True:
414.             from_key = f'from_plot_{i}'
415.             if from_key not in best_reloc:
416.                 break
417.
418.             households.append({
419.                 'from_yard': best_reloc['from_yard'],
420.                 'from_plot': best_reloc[from_key],
421.                 'to_yard': best_reloc[f'to_yard_{i}'],
422.                 'to_plot': best_reloc[f'to_plot_{i}']
423.             })
424.             i += 1
425.
426.         # 执行搬迁
427.         for h in households:
428.             from_mask = (df_current['院落ID'] == h['from_yard']) & \
429.                 (df_current['地块ID'] == h['from_plot'])
430.             to_mask = (df_current['院落ID'] == h['to_yard']) & \
431.                 (df_current['地块ID'] == h['to_plot'])
432.
433.             df_current.loc[from_mask, '是否有住户'] = 0
434.             df_current.loc[to_mask, '是否有住户'] = 1
435.
436.         # 更新状态
437.         executed_relocations.append(best_reloc)
438.         remaining_relocations.remove(best_reloc)
439.         m_history.append(best_m)
440.         success_count += 1
441.
442.     except Exception as e:
443.         error_log.append((best_reloc, f"执行错误: {str(e)}"))
444.
445.     else:
446.         break # 没有可行的搬迁方案
447.
448.     return success_count, error_log, df_current, m_history, final_best_cost, executed_relocations
449.
450. def _find_special_value(self, m: List[float]) -> Optional[List[float]]:
451.     """寻找拐点"""
452.     for i in range(1, len(m)):
453.         if m[i-1] >= m[i]:
454.             continue # 不满足条件1
455.
456.         # 检查前面是否有比 m[i] 大的和小的
457.         has_greater = any(m[j] > m[i] for j in range(i))
458.         has_less = any(m[j] < m[i] for j in range(i))
459.         if not (has_greater and has_less):
460.             continue # 不满足条件2
461.
462.         # 检查后面是否全部 < m[i]
463.         if all(m[j] < m[i] for j in range(i+1, len(m))):
464.             return m[i+1] # 满足所有条件
465.
466.     return None
467.
468. def _calculate_relocation_result(self, df: pd.DataFrame) -> List:
469.     """
470.     计算完成所有搬迁后的新DataFrame和总成本
471.
472.     参数:
473.         df: 原始地块DataFrame
474.         relocations: 所有搬迁方案列表
475.         adj_matrix: 邻接矩阵
476.
477.     返回:
478.         (df_new, total_cost)
479.     """
480.     df_new = df.copy()
481.     total_cost = 0
482.
483.     # 展开搬迁方案 (处理嵌套列表情况)
484.     flat_relocations = []
485.     for item in self.execute_relocations_ip:
486.         if isinstance(item, list):
487.             flat_relocations.extend(item)

```

```

484.         flat_relocations = []
485.         for item in self.execute_relocations_ip:
486.             if isinstance(item, list):
487.                 flat_relocations.extend(item)
488.             else:
489.                 flat_relocations.append(item)
490.
491.         # 执行所有搬迁
492.         for reloc in flat_relocations:
493.             try:
494.                 # 解析搬迁记录
495.                 households = []
496.                 i = 1
497.                 while True:
498.                     from_key = f'from_plot_{i}'
499.                     if from_key not in reloc:
500.                         break
501.
502.                     households.append({
503.                         'from_yard': reloc['from_yard'],
504.                         'from_plot': reloc[from_key],
505.                         'to_yard': reloc[f'to_yard_{i}'],
506.                         'to_plot': reloc[f'to_plot_{i}']
507.                     })
508.                     i += 1
509.
510.                 # 执行搬迁并计算成本
511.                 for h in households:
512.                     from_mask = (df_new['院落ID'] == h['from_yard']) & \
513.                                 (df_new['地块ID'] == h['from_plot'])
514.                     to_mask = (df_new['院落ID'] == h['to_yard']) & \
515.                               (df_new['地块ID'] == h['to_plot'])
516.
517.                     # 更新住户状态
518.                     df_new.loc[from_mask, '是否有住户'] = 0
519.                     df_new.loc[to_mask, '是否有住户'] = 1
520.
521.                     # 计算本次搬迁成本并累加
522.                     cost = self.calculate_relocation_cost_new(
523.                         df[df['地块ID'] == h['from_plot']],
524.
525.                         # 更新住户状态
526.                         df_new.loc[from_mask, '是否有住户'] = 0
527.                         df_new.loc[to_mask, '是否有住户'] = 1
528.
529.                         # 计算本次搬迁成本并累加
530.                         cost = self.calculate_relocation_cost_new(
531.                             df[df['地块ID'] == h['from_plot']],
532.                             df[df['地块ID'] == h['to_plot']]
533.                         )
534.                         total_cost += cost
535.
536.             except Exception as e:
537.                 print(f"搬迁方案执行失败: {reloc}, 错误: {str(e)}")
538.                 continue
539.
540.         return df_new, total_cost
541.
542. # ----- 工具方法 -----
543. # (保留所有原有的calculate_*方法和YardRelocationOptimizer类)
544. def calculate_income_fourmonth(self, df_old, df_new, adjacency_matrix):
545.     """完整收入计算系统"""
546.     # 1. 数据预处理
547.     merged = pd.merge(
548.         df_old[['地块ID', '院落ID', '地块面积', '院落面积', '地块方位', '是否有住户']],
549.         df_new[['地块ID', '是否有住户']],
550.         on='地块ID',
551.         suffixes=('_old', '_new')
552.     )
553.     merged['四个月可计算'] = (merged['是否有住户_old'] == 0) & (merged['是否有住户_new'] == 0)
554.     # 2. 识别完整院落 (新数据中无住户)
555.     yard_status = merged.groupby('院落ID').agg({
556.         '四个月可计算': lambda x: all(x == True),
557.         '院落面积': 'first',
558.         '地块方位': 'first'
559.     }).rename(columns={'四个月可计算': 'is_unoccupied'})
560.
561.     # 3. 修正毗邻判断: 仅完整院落间相邻
562.     complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
563.     adj_complete = np.zeros_like(adjacency_matrix)

```

```

557.     # 构建仅包含完整院落的邻接矩阵
558.     for i in complete_yard_ids:
559.         for j in complete_yard_ids:
560.             if i != j and adjacency_matrix[i-1, j-1] == 1:
561.                 adj_complete[i-1, j-1] = 1
562.
563.     yard_status['is_adjacent'] = yard_status.index.map(
564.         lambda y: y in complete_yard_ids and adj_complete[y-1].any()
565.     )
566.
567.     # 4. 分阶段计算
568.     # 识别有效完整院落
569.     complete = yard_status[yard_status['is_unoccupied']]
570.
571.     # 完整院落收入
572.     adjacent = complete[complete['is_adjacent']]['院落面积'].sum() * self.adjacency_price
573.     normal = complete[~complete['is_adjacent']]['院落面积'].sum() * self.complete_price
574.     yard_income = (adjacent + normal) * 4*30/10000
575.
576.     # 分散地块收入
577.     yard_ids = complete.index
578.
579.     valid = merged[(merged['四个月可计算'] == True) &
580.                    (~merged['院落ID'].isin(yard_ids))]
581.
582.     # valid = merged[(merged['是否有住户_new'] == 0) &
583.                      (~merged['院落ID'].isin(yard_ids))]
584.
585.     def plot_rate(row):
586.         if row['地块方位'] in ['东', '西']: return self.nn
587.         return self.ew
588.
589.     plot_income = valid.apply(
590.         lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * 4*30/10000
591.
592.     return yard_income + plot_income
593.
594. def calculate_income_tenyear(self, df_new, adjacency_matrix):
595.     """完整收入计算系统"""

```

```

597.     # 2. 识别完整院落 (新数据中无住户)
598.     yard_status = df_new.groupby('院落ID').agg({
599.         '是否有住户': lambda x: all(x == False),
600.         '院落面积': 'first',
601.         '地块方位': 'first'
602.     }).rename(columns={'是否有住户': 'is_unoccupied'})
603.
604.     # 3. 修正毗邻判断: 仅完整院落间相邻
605.     complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
606.     adj_complete = np.zeros_like(adjacency_matrix)
607.
608.     # 构建仅包含完整院落的邻接矩阵
609.     for i in complete_yard_ids:
610.         for j in complete_yard_ids:
611.             if i != j and adjacency_matrix[i-1, j-1] == 1:
612.                 adj_complete[i-1, j-1] = 1
613.
614.     yard_status['is_adjacent'] = yard_status.index.map(
615.         lambda y: y in complete_yard_ids and adj_complete[y-1].any()
616.     )
617.
618.     # 4. 分阶段计算
619.     # 识别有效完整院落
620.     complete = yard_status[yard_status['is_unoccupied']]
621.
622.     # 完整院落收入
623.     adjacent = complete[complete['is_adjacent']]['院落面积'].sum() * self.adjacency_price
624.     normal = complete[~complete['is_adjacent']]['院落面积'].sum() * self.complete_price
625.     yard_income = (adjacent + normal) * (10*365-120)/10000
626.
627.     # 分散地块收入
628.     yard_ids = complete.index
629.
630.     valid = df_new[(~df_new['院落ID'].isin(yard_ids))]
631.
632.     # valid = merged[(merged['是否有住户_new'] == 0) &
633.                      (~merged['院落ID'].isin(yard_ids))]
634.
635.     def plot_rate(row):
636.         if row['地块方位'] in ['东', '西']: return self.nn
637.         return self.ew

```

```

639.         plot_income = valid.apply(
640.             lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * (10*365-120)/10000
641.
642.         return yard_income + plot_income
643.
644.     def calculate_income(self, df, adjacency_matrix):
645.         """完整收入计算系统"""
646.
647.         # 2. 识别完整院落 (数据中无住户)
648.         yard_status = df.groupby('院落ID').agg({
649.             '是否有住户': lambda x: all(x == False),
650.             '院落面积': 'first',
651.             '地块方位': 'first'
652.         }).rename(columns={'是否有住户': 'is_unoccupied'})
653.
654.         # 3. 修正毗邻判断: 仅完整院落间相邻
655.         complete_yard_ids = list(yard_status[yard_status['is_unoccupied']].index)
656.         adj_complete = np.zeros_like(adjacency_matrix)
657.
658.         # 构建仅包含完整院落的邻接矩阵
659.         for i in complete_yard_ids:
660.             for j in complete_yard_ids:
661.                 if i != j and adjacency_matrix[i-1, j-1] == 1:
662.                     adj_complete[i-1, j-1] = 1
663.
664.         yard_status['is_adjacent'] = yard_status.index.map(
665.             lambda y: y in complete_yard_ids and adj_complete[y-1].any()
666.         )
667.
668.         # 4. 分阶段计算
669.         # 识别有效完整院落
670.         complete = yard_status[yard_status['is_unoccupied']]
671.
672.         # 完整院落收入
673.         adjacent = complete[complete['is_adjacent']]['院落面积'].sum() * self.adjacency_price
674.         normal = complete[~complete['is_adjacent']]['院落面积'].sum() * self.complete_price
675.         yard_income = (adjacent + normal) * (10*365)/10000
676.
677.         # 分散地块收入
678.         yard_ids = complete.index

```

```

680.         valid = df[(~df['院落ID'].isin(yard_ids))]
681.
682.         # valid = merged[(merged['是否有住户_new'] == 0) &
683.         #                 (~merged['院落ID'].isin(yard_ids))]
684.
685.     def plot_rate(row):
686.         if row['地块方位'] in ['东', '西']: return self.nn
687.         return self.ew
688.
689.         plot_income = valid.apply(
690.             lambda x: x['地块面积'] * plot_rate(x), axis=1).sum() * (10*365)/10000
691.
692.         return yard_income + plot_income
693.
694.     def calculate_relocation_cost_new(self, from_plot, to_plot):
695.         """计算搬迁成本
696.         Args:
697.             from_plot: 原居住地块信息
698.             to_plot: 新搬迁地块信息
699.         Returns:
700.             总搬迁成本 (万元)
701.         """
702.         # 1. 沟通成本 (固定)
703.         communication_cost = self.communicate # 万元
704.
705.         # 2. 修缮成本 (这里设为0, 可根据实际情况调整)
706.         renovation_cost = self.renovation # 万元
707.
708.         # 3. 面积损失成本 (按10年计算)
709.         days_per_year = 365
710.         years = 10 # 按10年期计算
711.
712.         # 计算面积差 (新地块面积必须≥原地块面积)
713.         area_diff = max(0, to_plot['地块面积'].iloc[0] - from_plot.地块面积.iloc[0])
714.
715.         # 根据朝向确定租金单价
716.         if to_plot.地块方位.iloc[0] in ['东', '西']:
717.             rent_rate = self.nn # 东西厢8元/平米/天
718.         else:
719.             rent_rate = self.ew # 南北厢15元/平米/天
720.         if from_plot.地块方位.iloc[0] in ['东', '西']:
721.             rent_rate_new = self.nn # 东西厢8元/平米/天
722.         else:

```

```

723.         rent_rate_new = self.ew # 南北厢15元/平米/天
724.         # 面积损失成本 = 面积差 × 租金 × 365天 × 10年
725.         area_loss_cost = area_diff * rent_rate * days_per_year * years / 10000 # 转换为万元
726.
727.         # 4. 时间损失成本 (4个月无法出租)
728.         months = 4
729.         days = months * 30 # 按每月30天计算
730.         time_loss_cost = from_plot.地块面积.iloc[0] * rent_rate_new * days / 10000 # 万元
731.
732.         # 总成本
733.         total_cost = communication_cost + renovation_cost + area_loss_cost + time_loss_cost
734.
735.         return total_cost
736.
737. class YardRelocationOptimizer:
738.     def __init__(self, df, original_df, adjacency_matrix, household_num=1, cost=0, communicate=3, renovation=0, nn=8, ew=15, ratio=1.3):
739.         """
740.         初始化优化器
741.         :param df: 地块数据DataFrame
742.         :param adjacency_matrix: 邻接矩阵
743.         :param household_num: 要处理的院落住户数(1,2,3...)
744.         """
745.         self.df = df.copy()
746.         self.cost=cost
747.         self.original_df = original_df.copy()
748.         self.adj_matrix = np.array(adjacency_matrix)
749.         self.household_num = household_num
750.         self.cost_records = []
751.         self.current_cost = 0
752.         self.relocation_log = []
753.         self.communicate=communicate
754.         self.renovation=renovation
755.         self.nn=nn
756.
757.         self.ew=ew
758.         self.ratio=ratio
759.
760.     def preprocess_data(self):
761.         """预处理数据, 计算每个院落的住户数"""
762.         self.df['院落住户数'] = self.df.groupby('院落ID')['是否有住户'].transform('sum')
763.         self.yard_info = self.df.groupby('院落ID').agg({
764.             '院落面积': 'first',
765.             '院落住户数': 'first'
766.         }).reset_index()
767.
768.     def find_target_yards(self):
769.         """找出指定住户数的院落"""
770.         return self.yard_info[self.yard_info['院落住户数'] == self.household_num]['院落ID'].tolist()
771.
772.     def calculate_relocation_benefit(self, yard_id):
773.         """计算搬迁后增加的毗邻面积"""
774.         yard_area = self.yard_info[self.yard_info['院落ID'] == yard_id]['院落面积'].values[0]
775.         adjacent_yards = np.where(self.adj_matrix[yard_id-1] == 1)[0] + 1
776.
777.         total_benefit = yard_area
778.         for adj_yard in adjacent_yards:
779.             if adj_yard in self.complete_yards:
780.                 # 检查是否只与当前院落毗邻
781.                 other_adjacent = np.where(self.adj_matrix[adj_yard-1] == 1)[0] + 1
782.                 if len(other_adjacent) == 1 and other_adjacent[0] == yard_id:
783.                     total_benefit += self.yard_info[self.yard_info['院落ID'] == adj_yard]['院落面积'].values[0]
784.         return total_benefit
785.
786.     def get_complete_yards(self):
787.         """获取完整院落列表 (无住户的院落) """
788.         self.complete_yards = self.yard_info[self.yard_info['院落住户数'] == 0]['院落ID'].tolist()

```

```

789.     def rank_target_yards(self):
790.         """对目标院落按搬迁效益排序"""
791.         target_yards = self.find_target_yards()
792.         self.get_complete_yards()
793.
794.         ranked_yards = []
795.         for yard_id in target_yards:
796.             benefit = self.calculate_relocation_benefit(yard_id)
797.             ranked_yards.append((yard_id, benefit))
798.
799.         # 按效益降序排序
800.         return sorted(ranked_yards, key=lambda x: -x[1])
801.
802.     def get_direction_priority(self, direction):
803.         """获取方位优先级"""
804.         priority_map = {'南': 1, '北': 1, '东': 2, '西': 3}
805.         return priority_map.get(direction, 4)
806.
807.     def find_valid_plots(self, original_plot, allow_empty_yards=False):
808.         """
809.         找出符合条件的搬迁地块
810.         :param original_plot: 原始地块信息
811.         :param allow_empty_yards: 是否允许搬迁到空院落
812.         """
813.         original_area = original_plot['地块面积']
814.         original_dir = original_plot['地块方位']
815.         original_yard_id = original_plot['院落ID']
816.
817.         # 基础筛选条件
818.         conditions = [
819.             (self.df['地块面积'] >= original_area),
820.             (self.df['地块面积'] <= self.ratio * original_area),
821.             (self.df['地块方位'].apply(lambda x: self.get_direction_priority(x)) <= self.get_direction_priority(original_dir)),
822.             (self.df['是否有住户'] == 0)
823.         ]
824.
825.         if not allow_empty_yards:
826.             conditions.append(self.df['院落住户数'] != 0)
827.
828.         valid_plots = self.df[np.logical_and.reduce(conditions)].copy()
829.
830.         # 如果允许空院落, 需要检查搬迁效益
831.         if allow_empty_yards:
832.             re_area = self.calculate_relocation_benefit(original_yard_id)
833.             new_yard_ids = valid_plots[valid_plots['院落住户数']==0]['院落ID'].tolist()
834.             for new_yard_id in new_yard_ids:
835.                 new_area = self.calculate_relocation_benefit(new_yard_id)
836.                 if new_area > re_area:
837.                     valid_plots = valid_plots[valid_plots['院落ID'] != new_yard_id]
838.
839.         # 计算排序指标
840.         valid_plots['主排序'] = valid_plots['院落ID'].map(self.yard_info.set_index('院落ID')['院落住户数'])
841.         valid_plots['次排序'] = valid_plots['院落ID'].apply(lambda x: self.calculate_adjacent_complete_area(x))
842.         valid_plots['次次排序'] = abs(valid_plots['地块面积'] - original_area)
843.
844.         # 排序: 主降序, 次升序
845.         return valid_plots.sort_values(
846.             by=['主排序', '次排序', '次次排序'],
847.             ascending=[False, True, True])
848.
849.     def calculate_adjacent_complete_area(self, yard_id):
850.         """计算只与该院落毗邻的完整院落面积"""
851.         adjacent_yards = np.where(self.adj_matrix[yard_id-1] == 1)[0] + 1
852.         total_area = 0
853.
854.         for adj_yard in adjacent_yards:
855.             if adj_yard in self.complete_yards:
856.                 other_adjacent = np.where(self.adj_matrix[adj_yard-1] == 1)[0] + 1
857.                 if len(other_adjacent) == 1 and other_adjacent[0] == yard_id:
858.                     total_area += self.yard_info[self.yard_info['院落ID'] == adj_yard]['院落面积'].values[0]
859.         return total_area

```

```

861.     def calculate_relocation_cost(self, from_plot, to_plot):
862.         """计算搬迁成本
863.         Args:
864.             from_plot: 原居住地块信息
865.             to_plot: 新搬迁地块信息
866.         Returns:
867.             总搬迁成本 (万元)
868.         """
869.         # 1. 沟通成本 (固定)
870.         communication_cost = self.communicate # 万元
871.
872.         # 2. 修缮成本 (这里设为0, 可根据实际情况调整)
873.         renovation_cost = self.renovation # 万元
874.
875.         # 3. 面积损失成本 (按10年计算)
876.         days_per_year = 365
877.         years = 10 # 按10年期计算
878.
879.         # 计算面积差 (新地块面积必须>原地块面积)
880.         area_diff = max(0, to_plot['地块面积'] - from_plot.地块面积)
881.
882.         # 根据朝向确定租金单价
883.         if to_plot.地块方位 in ['东', '西']:
884.             rent_rate = self.nn # 东西厢8元/平米/天
885.         else:
886.             rent_rate = self.ew # 南北厢15元/平米/天
887.
888.         if from_plot.地块方位 in ['东', '西']:
889.             rent_rate_new = self.nn # 东西厢8元/平米/天
890.         else:
891.             rent_rate_new = self.ew # 南北厢15元/平米/天
892.
893.         # 面积损失成本 = 面积差 × 租金 × 365天 × 10年
894.         area_loss_cost = area_diff * rent_rate * days_per_year * years / 10000 # 转换为万元
895.
896.         # 4. 时间损失成本 (4个月无法出租)
897.         months = 4
898.         days = months * 30 # 按每月30天计算
899.         time_loss_cost = from_plot.地块面积 * rent_rate_new * days / 10000 # 万元
900.
901.         # 总成本
902.         total_cost = communication_cost + renovation_cost + area_loss_cost + time_loss_cost
903.
904.         return total_cost
905.
906.     def update_data(self, from_plot_id, to_plot_id):
907.         """更新数据表"""
908.         if self.df.loc[self.df['地块ID'] == to_plot_id, '是否有住户'].values[0] == 1:
909.             raise ValueError(f"地块ID {to_plot_id} 已有住户, 无法搬迁")
910.
911.         # 更新住户状态
912.         self.df.loc[self.df['地块ID'] == from_plot_id, '是否有住户'] = 0
913.         self.df.loc[self.df['地块ID'] == to_plot_id, '是否有住户'] = 1
914.
915.         # 更新院落住户数
916.         self.df['院落住户数'] = self.df.groupby('院落ID')['是否有住户'].transform('sum')
917.         self.yard_info = self.df.groupby('院落ID').agg({
918.             '院落面积': 'first',
919.             '院落住户数': 'first'
920.         }).reset_index()
921.
922.     # def resolve_duplicate_plots(self, best_plots, valid_plots_list):
923.     #     """
924.     #     解决地块重复问题
925.     #     :param best_plots: 当前最佳地块列表
926.     #     :param valid_plots_list: 每个地块对应的备选列表
927.     #     :return: 处理后的无重复地块列表或None(无法解决)
928.     #     """
929.     #     n = len(best_plots)
930.     #     for i in range(n):
931.     #         for j in range(i + 1, n):
932.     #             if best_plots[i]['地块ID'] == best_plots[j]['地块ID']:
933.     #                 # 尝试替换后面的地块
934.     #                 if len(valid_plots_list[j]) > 1:
935.     #                     best_plots[j] = valid_plots_list[j].iloc[1]
936.     #                 # 如果不行, 尝试替换前面的地块
937.     #                 elif len(valid_plots_list[i]) > 1:
938.     #                     best_plots[i] = valid_plots_list[i].iloc[1]
939.     #             else:

```



```

938.         # else:
939.         #     return None
940.         # return best_plots
941.     def resolve_duplicate_plots(self, best_plots, valid_plots_list):
942.         """
943.         彻底解决地块重复问题（递归检查+优先队列）
944.
945.         :param best_plots: 当前最佳地块列表（pd.Series对象列表）
946.         :param valid_plots_list: 每个地块对应的备选DataFrame列表
947.         :return: 无重复的地块列表或None（无法解决）
948.         """
949.         # 转换为字典列表方便处理
950.         current_plots = [plot.to_dict() for plot in best_plots]
951.         candidate_pools = [df.to_dict('records') for df in valid_plots_list]
952.
953.         def backtrack(selected, index):
954.             if index == len(current_plots):
955.                 return selected
956.
957.             current_id = current_plots[index]['地块ID']
958.
959.             # 检查当前选择是否与已选地块冲突
960.             for i in range(index):
961.                 if selected[i]['地块ID'] == current_id:
962.                     # 尝试从候选池中选择下一个最佳方案
963.                     if len(candidate_pools[index]) > 1:
964.                         next_option = candidate_pools[index][1] # 取次优选项
965.                         candidate_pools[index] = candidate_pools[index][1:] # 移除已尝试选项
966.                         return backtrack(selected[:index] + [next_option], index)
967.                     else:
968.                         return None # 无备选方案

```

```

973.         # 首次尝试使用最优解
974.         result = backtrack([], 0)
975.
976.         if result is None:
977.             # 如果首次回溯失败，尝试交换处理顺序（解决AB-BA式死锁）
978.             for i in range(len(current_plots)):
979.                 for j in range(i+1, len(current_plots)):
980.                     if current_plots[i]['地块ID'] == current_plots[j]['地块ID']:
981.                         # 尝试交换优先级
982.                         swapped_plots = current_plots.copy()
983.                         swapped_plots[i], swapped_plots[j] = swapped_plots[j], swapped_plots[i]
984.                         swapped_pools = candidate_pools.copy()
985.                         swapped_pools[i], swapped_pools[j] = swapped_pools[j], swapped_pools[i]
986.
987.                         result = backtrack([], 0)
988.                         if result is not None:
989.                             return [pd.Series(plot) for plot in result]
990.
991.             return None # 所有尝试都失败
992.
993.         return [pd.Series(plot) for plot in result]
994.
995.     def optimize_relocation(self):
996.         """执行搬迁优化"""
997.         self.preprocess_data()
998.
999.         while True:
1000.             ranked_yards = self.rank_target_yards()
1001.             if not ranked_yards:
1002.                 break
1003.
1004.             for yard_id, total_adj_area in ranked_yards:
1005.                 # 获取该院落的所有住户地块
1006.                 household_plots = self.df[(self.df['院落ID'] == yard_id) &
1007.                                             (self.df['是否有住户'] == 1)].iloc[:self.household_num]
1008.
1009.                 # 为每个住户地块寻找有效搬迁地块
1010.                 valid_plots_list = []

```

```

1011.         for _, plot in household_plots.iterrows():
1012.             # 先尝试不允许空院落的查找
1013.             valid_plots = self.find_valid_plots(plot, allow_empty_yards=False)
1014.             if valid_plots.empty:
1015.                 # 如果找不到, 尝试允许空院落的查找
1016.                 valid_plots = self.find_valid_plots(plot, allow_empty_yards=False)
1017.                 if valid_plots.empty:
1018.                     break
1019.             valid_plots_list.append(valid_plots)
1020.
1021.         # 如果未能为所有住户找到有效地块, 跳过该院落
1022.         if len(valid_plots_list) != self.household_num:
1023.             continue
1024.
1025.         # 选择每个住户的最佳地块
1026.         best_plots = [plots.iloc[0] for plots in valid_plots_list]
1027.
1028.         # 解决地块重复问题
1029.         resolved_plots = self.resolve_duplicate_plots(best_plots, valid_plots_list)
1030.         if resolved_plots is None:
1031.             continue
1032.
1033.         # 准备搬迁记录
1034.         relocation_info = {'from_yard': yard_id}
1035.         cost = 0
1036.
1037.         # 处理每个搬迁
1038.         for i, (from_plot, to_plot) in enumerate(zip(household_plots.itertuples(), resolved_plots)):
1039.             relocation_info.update({
1040.                 f'from_plot_{i+1}': from_plot.地块ID,
1041.                 f'to_yard_{i+1}': to_plot['院落ID'],
1042.                 f'to_plot_{i+1}': to_plot['地块ID']
1043.             })
1044.             cost += self.calculate_relocation_cost(from_plot, to_plot)
1045.
1046.         # 记录搬迁信息
1047.         self.relocation_log.append(relocation_info)
1048.         self.current_cost += cost
1049.         self.cost_records.append({
1049.
1050.             self.cost_records.append({
1051.                 'relocation': len(self.relocation_log),
1052.                 'cost': cost,
1053.                 'cumulative_cost': self.current_cost
1054.             })
1055.
1056.             # 执行数据更新
1057.             for from_plot, to_plot in zip(household_plots.itertuples(), resolved_plots):
1058.                 self.update_data(from_plot.地块ID, to_plot['地块ID'])
1059.
1060.
1061.
1062.             # 一次只处理一个院落的搬迁
1063.             break
1064.         else:
1065.             # 没有找到符合条件的搬迁
1066.             break
1067.
1068.         return self.relocation_log, self.cost_records, self.df, self.current_cost+self.cost
1069.
1070.     ###
1071.     if __name__ == "__main__":
1072.         # 初始化系统
1073.         system = RelocationOptimizationSystem(housing_num=[1,2,3,4,5,6,7,8], adjacency_price=36, complete_price=30, communicate=3, renovation=0,
1074.
1075.         # 加载输入数据
1076.         system.load_data(
1077.             adjacency_path="毗邻矩阵.xlsx",
1078.             plot_info_path="附件一: 老城街区地块信息.xlsx"
1079.         )
1080.
1081.         # 执行优化
1082.         system.run_optimization()

```

```

1081. # 执行优化
1082. system.run_optimization()
1083.
1084. # 生成并打印完整报告
1085. full_report = system.generate_full_report()
1086.
1087. # 打印常规优化结果
1088. print("\n===== 常规优化结果 =====")
1089. reg = full_report['regular_optimization']
1090. print(f"总投入成本: {reg['total_cost']:.2f} 万元")
1091. print(f"搬迁腾出的完整院落ID: {reg['new_complete_yards']}")
1092. print(f"最终完整院落总面积: {reg['final_complete_yards_area']:.2f} m²")
1093. print(f"毗邻完整院落总面积: {reg['final_adjacent_area']:.2f} m²")
1094. print(f"预计总收入: {reg['total_income']:.2f} 万元")
1095. print(f"预计总盈利: {reg['total_profit']:.2f} 万元")
1096. print(f"完整院落数: {reg['optimization_steps']['完整院落数']}")
1097.
1098. # 打印拐点分析结果
1099. print("\n===== 拐点分析结果 =====")
1100. infl = full_report['inflection_point_analysis']
1101. print(f"拐点位置: 第{infl['inflection_point']}次搬迁")
1102. print(f"拐点处总成本: {infl['cost']:.2f} 万元")
1103. print(f"拐点处总收入: {infl['income']:.2f} 万元")
1104. print(f"拐点处总利润: {infl['profit']:.2f} 万元")
1105. print(f"拐点处腾空院落ID: {infl['complete_yards']}")
1106. print(f"拐点处完整院落面积: {infl['complete_area']:.2f} m²")
1107. print(f"拐点处毗邻面积: {infl['adjacent_area']:.2f} m²")
1108.
1109. # 绘制m值变化曲线
1110. plt.figure(figsize=(10, 6))
1111. plt.plot(full_report['inflection_point_analysis']['m_history'], marker='o')
1112. plt.title('m-value change')
1113. plt.xlabel('step')
1114. plt.ylabel('m-value')
1115. plt.grid(True)
1116. plt.show()

```